INFORME FINAL

Docker & Docker Compose
Trabajo realizado

JHONATAN BRICEÑO CUÑACHI CANAQUIRI

Curso CURS-000662

Repositorio: https://github.com/jhobri26/docker-lab.git

Generado: 02 nov. 2026, 19:46 (Lima)

100 % de avance declarado · 10 de 10 casos listos para revisión
70 evidencias adjuntas

El estado corresponde al avance registrado por el alumno. La evaluación y la calificación corresponden al docente.

Contenido

01. Docker Cheat Sheet personalizado	8 evidencias
02. Contenedores simples	8 evidencias
03. Conectando tres contenedores	9 evidencias
04. Redes y conectividad	8 evidencias
05. Creación de imágenes	8 evidencias
06. Registro de imágenes	12 evidencias
07. Docker Compose	2 evidencias
08. Docker Compose con Build	3 evidencias
09. Secrets y volúmenes	6 evidencias
10. Investigación y complementos	6 evidencias

CASO 01

Docker Cheat Sheet personalizado

Listo para revisión · Avance guardado: 31 oct. 2026, 21:48

Lo solicitado

Crear una hoja de ayuda personalizada de Docker y Docker Compose con los principales comandos usados en los casos y otros relevantes para el trabajo.

Desarrollo y resultados

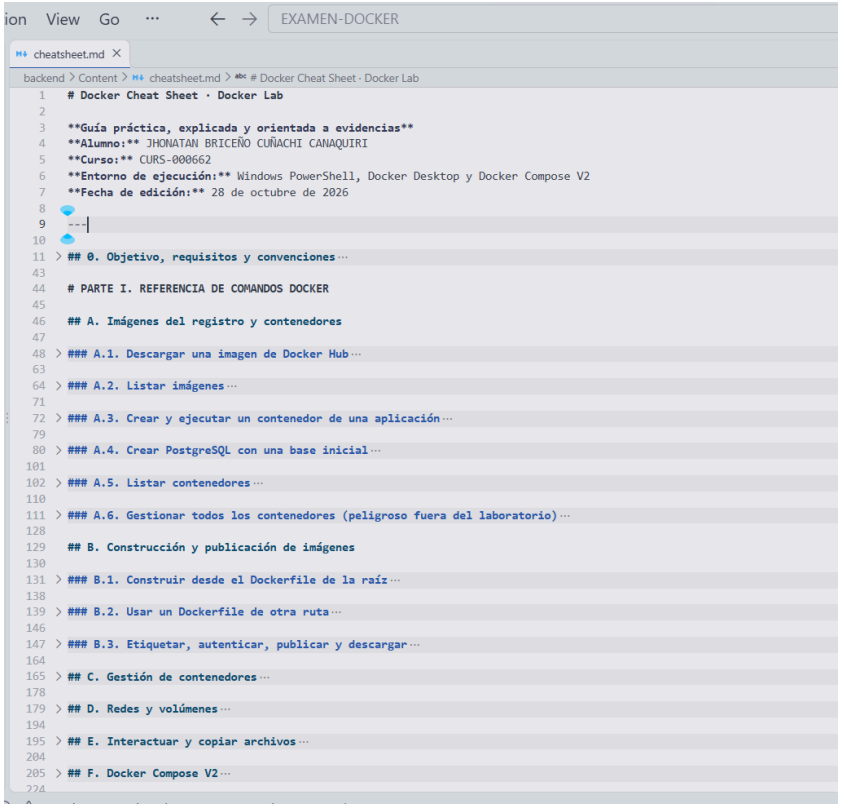
se evindencias de algunos comandos del Docker Sheet

Evidencias adjuntas: 8. Las capturas y sus descripciones se presentan a continuación.

CASO 01 · EVIDENCIA 6

Hoja de ayuda Docker – Consulta y ejecución de comandos

Cargada: 28 oct. 2026, 21:49 (Lima)



Descripción de la evidencia

Se presenta la hoja de ayuda (Docker Cheat Sheet), que contiene los principales comandos utilizados para la administración de imágenes, contenedores, redes, volúmenes y servicios mediante Docker Compose.

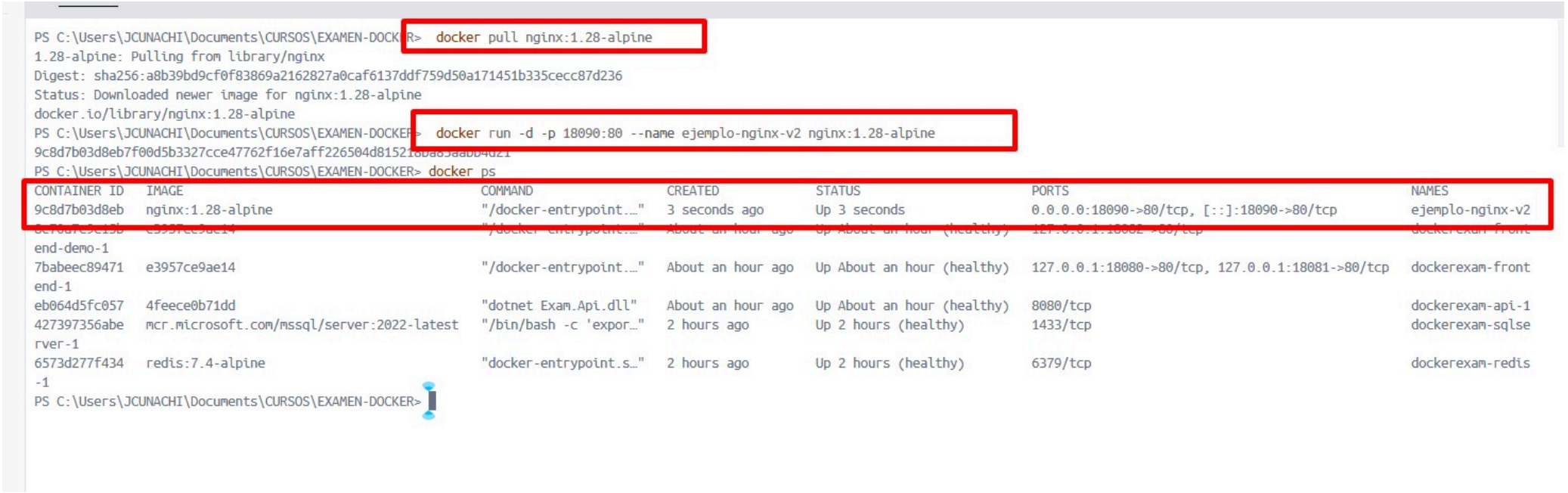
El documento incluye la descripción, sintaxis, finalidad y ejemplos de ejecución de cada comando, facilitando su aplicación durante el desarrollo del laboratorio.

Esta evidencia permite demostrar la documentación y preparación técnica de los comandos Docker utilizados en las actividades del curso CURS-000662.

CASO 01 · EVIDENCIA 9

A.1. Descargar una imagen de Docker Hub

Cargada: 28 oct. 2026, 22:20 (Lima)



```
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER>  docker pull nginx:1.28-alpine
1.28-alpine: Pulling from library/nginx
Digest: sha256:a8b39bd9cf0f83869a2162827a0caf6137ddf759d50a171451b335cecc87d236
Status: Downloaded newer image for nginx:1.28-alpine
docker.io/library/nginx:1.28-alpine
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER>  docker run -d -p 18090:80 --name ejemplo-nginx-v2 nginx:1.28-alpine
9c8d7b03d8eb7f00d5b3327cce47762f16e7aff226504d815218baa3aabb4d21
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker ps
CONTAINER ID   IMAGE                                   COMMAND                  CREATED          STATUS                    PORTS                                                  NAMES
9c8d7b03d8eb   nginx:1.28-alpine                       "/docker-entrypoint.…"   3 seconds ago    Up 3 seconds              0.0.0.0:18090->80/tcp, [::]:18090->80/tcp              ejemplo-nginx-v2
[illegible]                                                                                                                                                                    dockerexam-front
end-demo-1
7babeec89471   e3957ce9ae14                            "/docker-entrypoint.…"   About an hour ago   Up About an hour (healthy)   127.0.0.1:18080->80/tcp, 127.0.0.1:18081->80/tcp   dockerexam-front
end-1
eb064d5fc057   4feece0b71dd                            "dotnet Exam.Api.dll"    About an hour ago   Up About an hour (healthy)   8080/tcp                                           dockerexam-api-1
427397356abe   mcr.microsoft.com/mssql/server:2022-latest   "/bin/bash -c 'expor…"   2 hours ago      Up 2 hours (healthy)      1433/tcp                                           dockerexam-sqlse
rver-1
6573d277f434   redis:7.4-alpine                        "docker-entrypoint.s…"   2 hours ago      Up 2 hours (healthy)      6379/tcp                                           dockerexam-redis
-1
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER>
```

Descripción de la evidencia

docker pull nginx:1.28-alpine

Qué hace: Descarga la imagen `nginx` con la etiqueta `1.28-alpine` al almacenamiento local, sin crear ni iniciar contenedores. **Resultado esperado:** Docker informa que descargó o ya tiene las capas de la imagen. **Verificación:** `docker image ls nginx`.

```
docker run -d -p 8088:80 --name ejemplo-nginx nginx:1.28-alpine
```

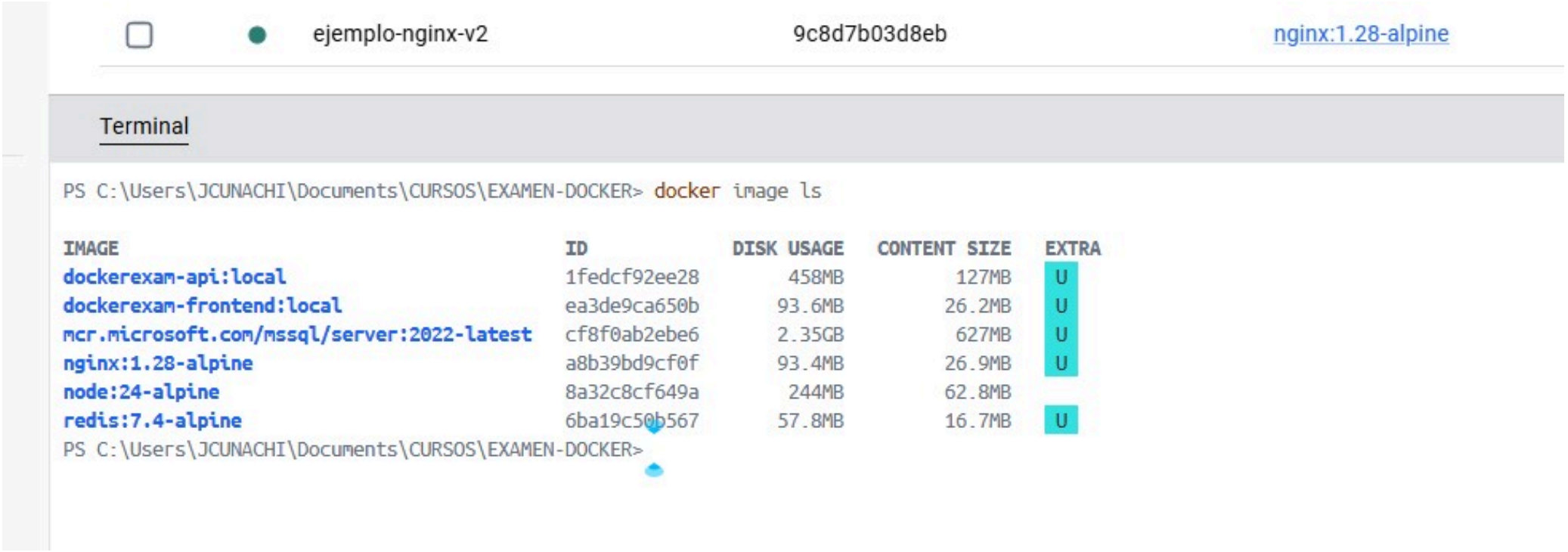
Qué hace: Si es necesario descarga la imagen, crea un contenedor llamado `ejemplo-nginx`, lo inicia en segundo plano (`-d`) y publica su puerto interno 80 en el puerto 8088 del equipo. **Verificación:** `curl.exe http://localhost:8088` y `docker ps`.

> `docker pull` y `docker run` no son equivalentes: el primero solo obtiene la imagen; el segundo crea e inicia un contenedor.

CASO 01 · EVIDENCIA 14

A.2. Listar imágenes

Cargada: 28 oct. 2026, 22:49 (Lima)



Descripción de la evidencia

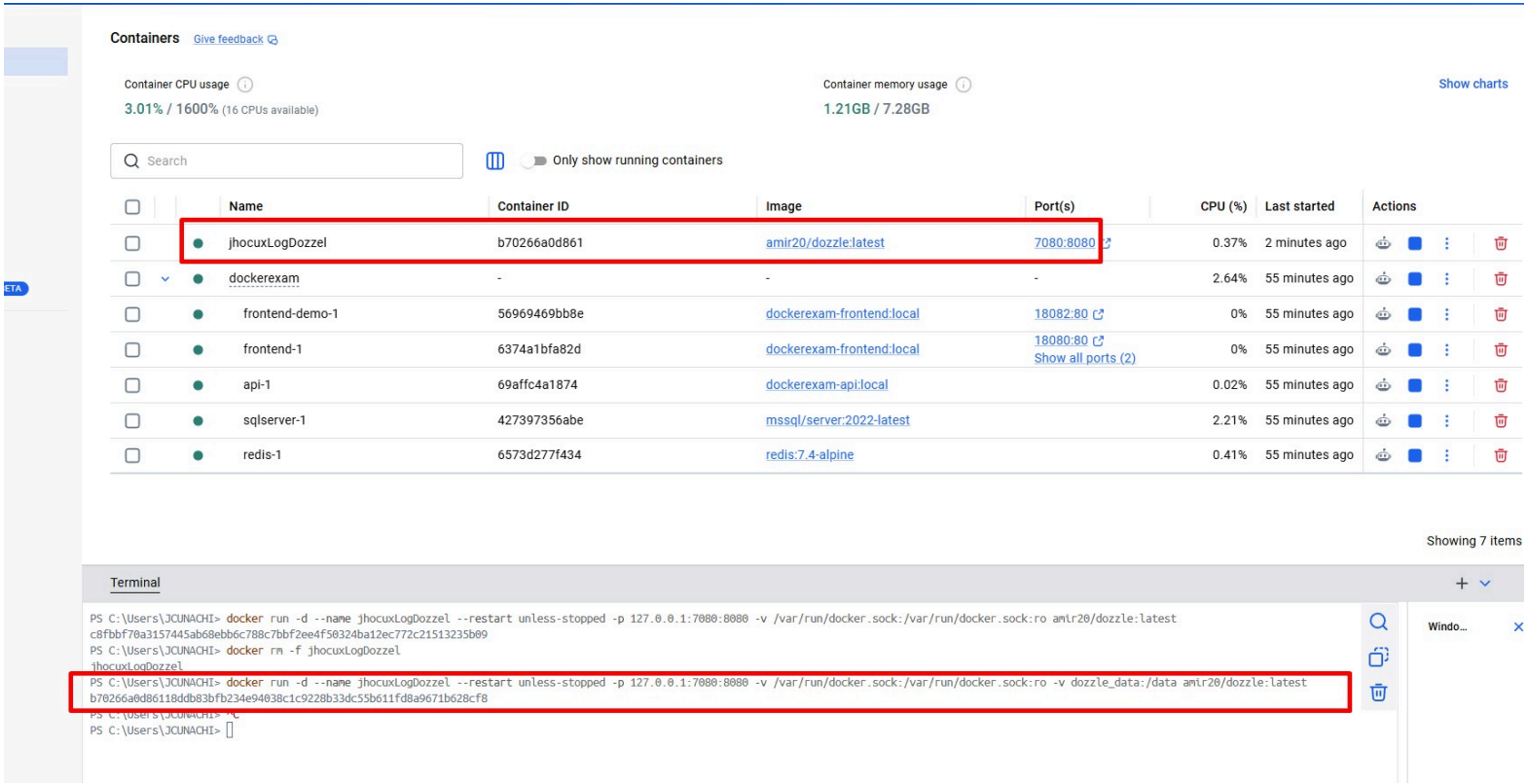
docker image ls

Qué hace: Lista repositorio, etiqueta, identificador, fecha y tamaño de imágenes locales. **Evidencia:** salida donde figure la imagen descargada.

CASO 01 · EVIDENCIA 1010

A.3. Crear y ejecutar un contenedor de una aplicación

Cargada: 31 oct. 2026, 13:30 (Lima)



Descripción de la evidencia

```powershell
 docker run -d --name jhocuxLogDozzel --restart unless-stopped -p 127.0.0.1:7080:8080 -v /var/run/docker.sock:/var/run/docker.sock:ro -v dozzle_data:/data amir20/dozzle:latest
```

Qué hace: Crea e inicia `jhocuxLogDozzel` desde `amir20/dozzle:latest`, publicando dos puertos. Al no usar `-d`, el proceso queda adjunto a la terminal. **Precaución:** ambos puertos deben estar libres; la disponibilidad de la imagen y sus puertos efectivos depende de esa aplicación. **Verificación:** `docker ps` y `docker logs jhocuxLogDozzel`.

CASO 01 · EVIDENCIA 1011

A.5. Listar contenedores

Cargada: 31 oct. 2026, 21:18 (Lima)

Terminal

```
PS C:\Users\JCUNACHI> docker ps
CONTAINER ID   IMAGE                                        COMMAND                  CREATED       STATUS                    PORTS                                                  NAMES
b07cbed565d4   louislam/uptime-kuma:2                       "/usr/bin/dumb-init …"   7 hours ago   Up 7 hours (healthy)      127.0.0.1:3001->3001/tcp                               jhocuxUptimeKuma
b70266a0d861   amir20/dozzle:latest                         "/dozzle"                8 hours ago   Up 8 hours                127.0.0.1:7080->8080/tcp                               jhocuxLogDozzel
56969469bb8e   dockerexam-frontend:local                    "/docker-entrypoint.…"   2 days ago    Up 9 hours (healthy)      127.0.0.1:18082->80/tcp                                dockerexam-frontend-demo-1
6374a1bfa82d   dockerexam-frontend:local                    "/docker-entrypoint.…"   2 days ago    Up 9 hours (healthy)      127.0.0.1:18080->80/tcp, 127.0.0.1:18081->80/tcp       dockerexam-frontend-1
69affc4a1874   dockerexam-api:local                         "dotnet Exam.Api.dll"    2 days ago    Up 9 hours (healthy)      8080/tcp                                               dockerexam-api-1
427397356abe   mcr.microsoft.com/mssql/server:2022-latest   "/bin/bash -c 'expor…"   3 days ago    Up 9 hours (healthy)      1433/tcp                                               dockerexam-sqlserver-1
6573d277f434   redis:7.4-alpine                             "docker-entrypoint.s…"   3 days ago    Up 9 hours (healthy)      6379/tcp                                               dockerexam-redis-1
PS C:\Users\JCUNACHI> docker ps -a
CONTAINER ID   IMAGE                                        COMMAND                  CREATED       STATUS                    PORTS                                                  NAMES
b07cbed565d4   louislam/uptime-kuma:2                       "/usr/bin/dumb-init …"   7 hours ago   Up 7 hours (healthy)      127.0.0.1:3001->3001/tcp                               jhocuxUptimeKuma
b70266a0d861   amir20/dozzle:latest                         "/dozzle"                8 hours ago   Up 8 hours                127.0.0.1:7080->8080/tcp                               jhocuxLogDozzel
56969469bb8e   dockerexam-frontend:local                    "/docker-entrypoint.…"   2 days ago    Up 9 hours (healthy)      127.0.0.1:18082->80/tcp                                dockerexam-frontend-demo-1
6374a1bfa82d   dockerexam-frontend:local                    "/docker-entrypoint.…"   2 days ago    Up 9 hours (healthy)      127.0.0.1:18080->80/tcp, 127.0.0.1:18081->80/tcp       dockerexam-frontend-1
69affc4a1874   dockerexam-api:local                         "dotnet Exam.Api.dll"    2 days ago    Up 9 hours (healthy)      8080/tcp                                               dockerexam-api-1
427397356abe   mcr.microsoft.com/mssql/server:2022-latest   "/bin/bash -c 'expor…"   3 days ago    Up 9 hours (healthy)      1433/tcp                                               dockerexam-sqlserver-1
6573d277f434   redis:7.4-alpine                             "docker-entrypoint.s…"   3 days ago    Up 9 hours (healthy)      6379/tcp                                               dockerexam-redis-1
PS C:\Users\JCUNACHI>
```

Descripción de la evidencia

```powershell
docker ps
docker ps -a
```

```

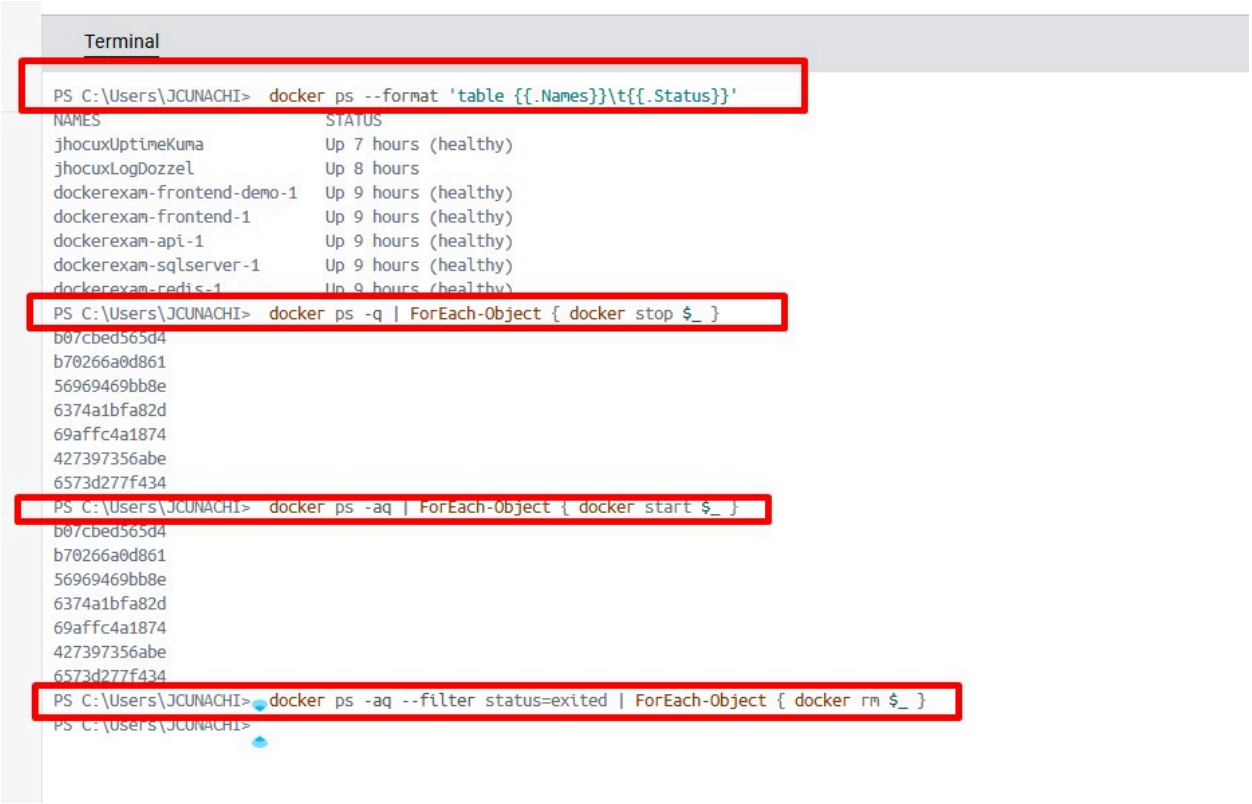
```

`**docker ps**` muestra únicamente contenedores en ejecución; `**docker ps -a**` incluye los detenidos. **Evidencia:** comparar ambos resultados después de detener un contenedor.

CASO 01 · EVIDENCIA 1012

A.6. Gestionar todos los contenedores (peligroso fuera del laboratorio)

Cargada: 31 oct. 2026, 21:26 (Lima)



Descripción de la evidencia

```powershell
# Primero, consultar cuáles se verían afectados
docker ps --format 'table {{.Names}}\t{{.Status}}'
```

```
# Detener todos los contenedores en ejecución (NO ejecutar si hay otros servicios)
docker ps -q | ForEach-Object { docker stop $_ }

# Iniciar todos los contenedores existentes
docker ps -aq | ForEach-Object { docker start $_ }

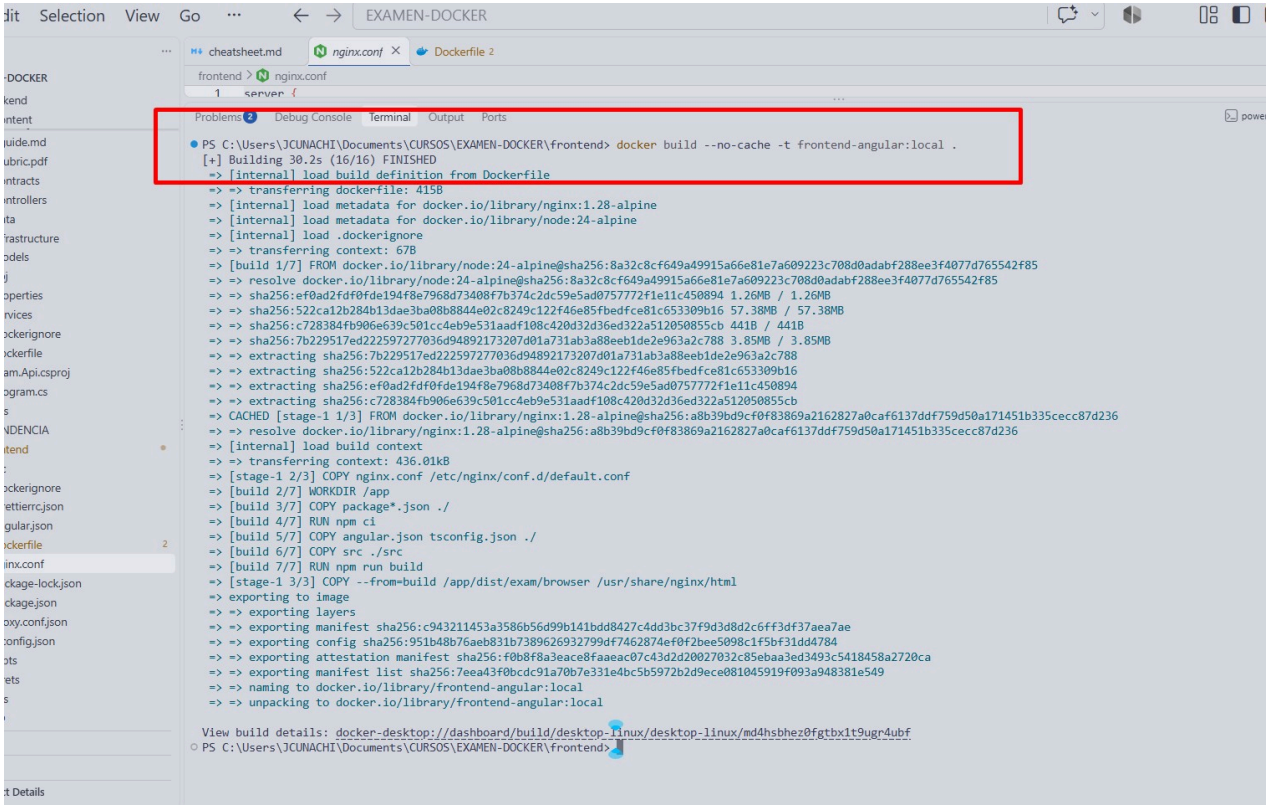
# Eliminar todos los contenedores detenidos (destructivo)
docker ps -aq --filter status=exited | ForEach-Object { docker rm $_ }
```

Interpretación: `-q` devuelve identificadores; `-a` incluye detenidos; `ForEach-Object` invoca Docker por cada ID. En PowerShell, `$(docker ps -q)` no debe asumirse como una solución segura para el caso sin resultados. **Recomendación para el laboratorio:** usar nombres explícitos o `docker compose` para no afectar proyectos ajenos.

CASO 01 · EVIDENCIA 1013

B.1. Construir desde el Dockerfile de la raíz

Cargada: 31 oct. 2026, 21:39 (Lima)



Descripción de la evidencia

```powershell
docker build --no-cache --build-arg JAR_FILE='target/*.jar' -t billingapp:local .

docker build --no-cache -t frontend-angular:local .
```

```

**Qué hace:** `docker build` construye una imagen, `--no-cache` omite la caché de pasos reutilizables, `--build-arg` entrega el argumento `JAR_FILE` al Dockerfile si éste lo declara mediante `ARG`, `-t` asigna el nombre/etiqueta y `.` define el contexto de construcción.

**Requisito:** deben existir el Dockerfile y el artefacto JAR esperado dentro del contexto.

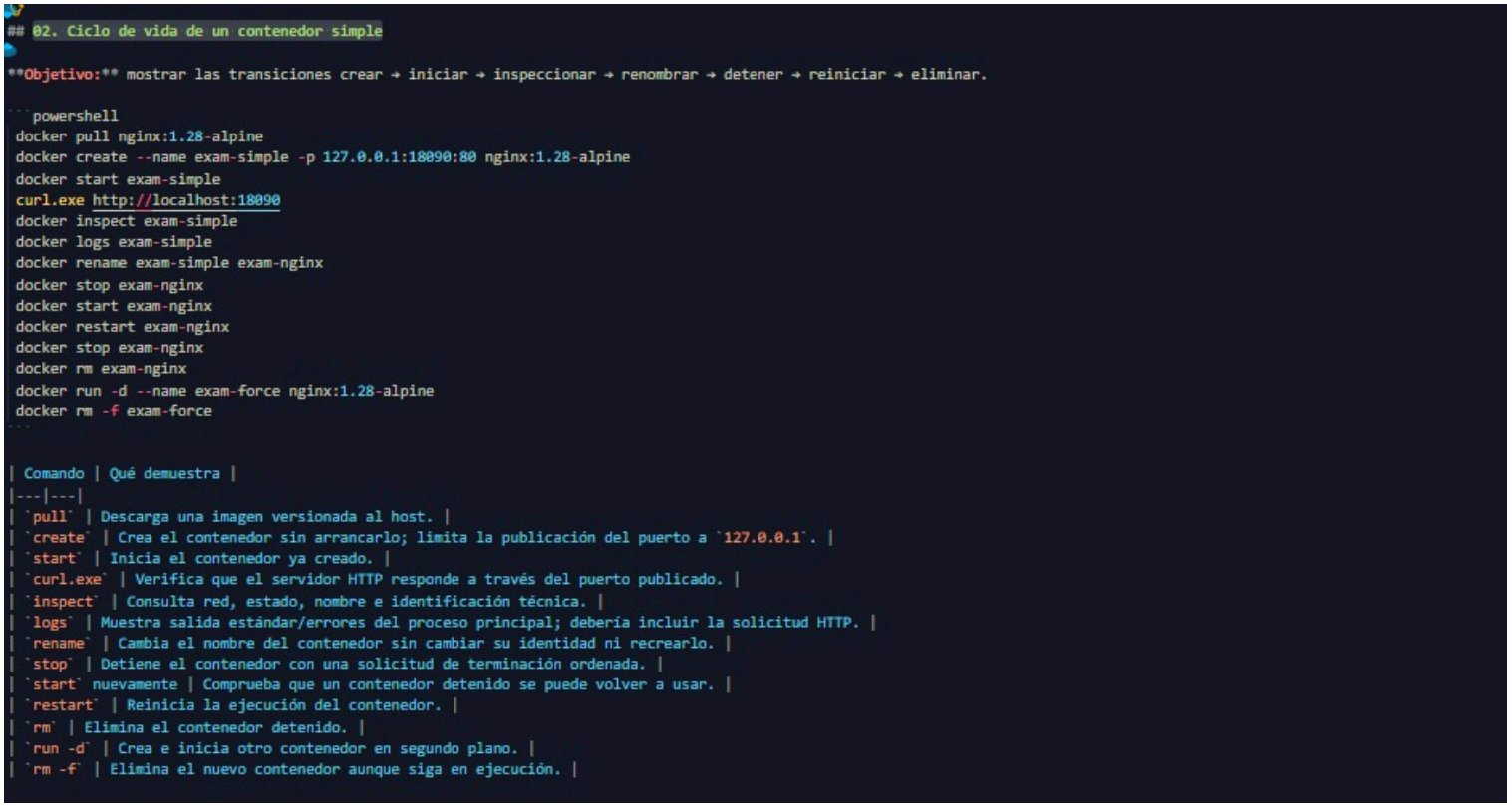
**Verificación:** `docker image ls billingapp`.

**Parámetro | Descripción**
docker build | Construye la imagen Docker
--no-cache | Reconstruye sin utilizar caché
-t frontend-angular:local | Nombre y etiqueta de la imagen
. | Utiliza la carpeta actual como contexto
```

CASO 01 · EVIDENCIA 1022

02. Ciclo de vida de un contenedor simple

Cargada: 01 nov. 2026, 00:17 (Lima)



````
## 02. Ciclo de vida de un contenedor simple

**Objetivo:** mostrar las transiciones crear → iniciar → inspeccionar → renombrar → detener → reiniciar → eliminar.

```powershell
docker pull nginx:1.28-alpine
docker create --name exam-simple -p 127.0.0.1:18090:80 nginx:1.28-alpine
docker start exam-simple
curl.exe http://localhost:18090
docker inspect exam-simple
docker logs exam-simple
docker rename exam-simple exam-nginx
docker stop exam-nginx
docker start exam-nginx
docker restart exam-nginx
docker stop exam-nginx
docker rm exam-nginx
docker run -d --name exam-force nginx:1.28-alpine
docker rm -f exam-force
```

| Comando | Qué demuestra |
|---|---|
| `pull` | Descarga una imagen versionada al host. |
| `create` | Crea el contenedor sin arrancarlo; limita la publicación del puerto a `127.0.0.1`. |
| `start` | Inicia el contenedor ya creado. |
| `curl.exe` | Verifica que el servidor HTTP responde a través del puerto publicado. |
| `inspect` | Consulta red, estado, nombre e identificación técnica. |
| `logs` | Muestra salida estándar/errores del proceso principal; debería incluir la solicitud HTTP. |
| `rename` | Cambia el nombre del contenedor sin cambiar su identidad ni recrearlo. |
| `stop` | Detiene el contenedor con una solicitud de terminación ordenada. |
| `start` nuevamente | Comprueba que un contenedor detenido se puede volver a usar. |
| `restart` | Reinicia la ejecución del contenedor. |
| `rm` | Elimina el contenedor detenido. |
| `run -d` | Crea e inicia otro contenedor en segundo plano. |
| `rm -f` | Elimina el nuevo contenedor aunque siga en ejecución. |
````

Descripción de la evidencia

````
```powershell
docker pull nginx:1.28-alpine
docker create --name exam-simple -p 127.0.0.1:18090:80 nginx:1.28-alpine
docker start exam-simple
````
````

```
curl.exe http://localhost:18090
docker inspect exam-simple
docker logs exam-simple
docker rename exam-simple exam-nginx
docker stop exam-nginx
docker start exam-nginx
docker restart exam-nginx
docker stop exam-nginx
docker rm exam-nginx
docker run -d --name exam-force nginx:1.28-alpine
docker rm -f exam-force
```

Comando	Qué demuestra
`pull`	Descarga una imagen versionada al host.
`create`	Crea el contenedor sin arrancarlo; limita la publicación del puerto a `127.0.0.1`.
`start`	Inicia el contenedor ya creado.
`curl.exe`	Verifica que el servidor HTTP responde a través del puerto publicado.
`inspect`	Consulta red, estado, nombre e identificación técnica.
`logs`	Muestra salida estándar/errores del proceso principal; debería incluir la solicitud HTTP.
`rename`	Cambia el nombre del contenedor sin cambiar su identidad ni recrearlo.
`stop`	Detiene el contenedor con una solicitud de terminación ordenada.
`start` nuevamente	Comprueba que un contenedor detenido se puede volver a usar.
`restart`	Reinicia la ejecución del contenedor.
`rm`	Elimina el contenedor detenido.
`run -d`	Crea e inicia otro contenedor en segundo plano.
`rm -f`	Elimina el nuevo contenedor aunque siga en ejecución.

CASO 02

Contenedores simples

Listo para revisión · Avance guardado: 01 nov. 2026, 00:18

Lo solicitado

Usar imágenes públicas y demostrar creación, eliminación estándar y forzada, inspección, logs, renombre, parada y reinicio de contenedores.

Desarrollo y resultados

Se culmino con la rubrita punto 2, con repositorios git Docker Hub

Evidencias adjuntas: 8. Las capturas y sus descripciones se presentan a continuación.

CASO 02 · EVIDENCIA 1014

Descarga de imagen Nginx 1.28 Alpine mediante Docker.

Cargada: 31 oct. 2026, 23:21 (Lima)



Descripción de la evidencia

docker pull nginx:1.28-alpine

Descripción:

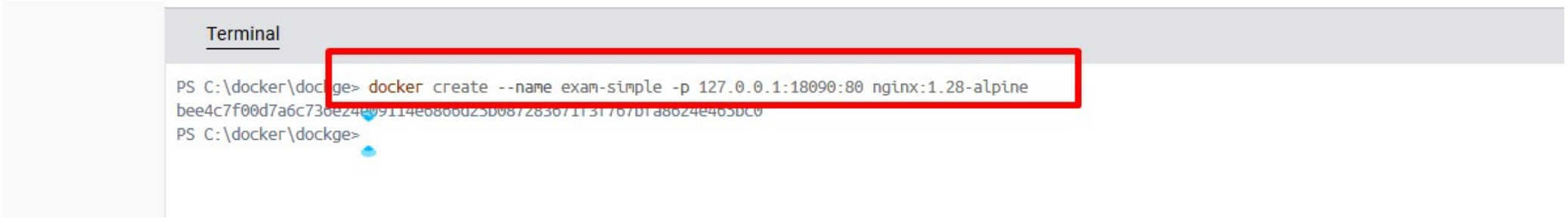
- `pull`: Comando que permite descargar imágenes desde un registro como Docker Hub.
- `nginx`: Nombre de la imagen oficial del servidor web que se desea descargar.
- `1.28`: Especifica la versión de Nginx.
- `alpine`: Indica que la imagen utiliza Alpine Linux, una distribución ligera y optimizada.

Resultado: El comando descarga y almacena la imagen de Nginx en Docker, dejándola disponible para crear y ejecutar contenedores posteriormente.

CASO 02 · EVIDENCIA 1015

Creación de contenedor Nginx con configuración de puerto local.

Cargada: 31 oct. 2026, 23:25 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker create --name exam-simple -p 127.0.0.1:18090:80 nginx:1.28-alpine
```

Descripción:

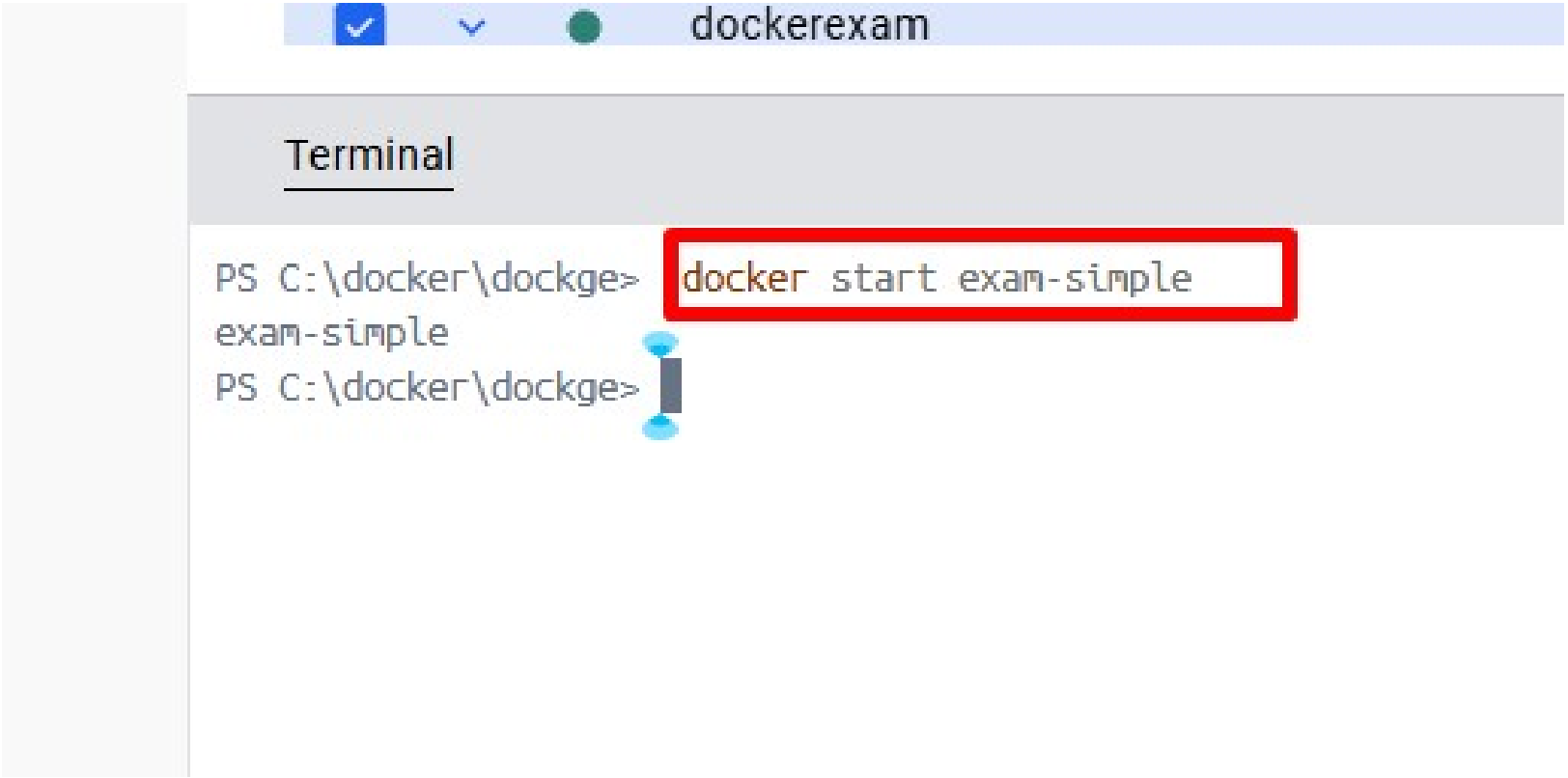
- `create`: Comando que permite crear un contenedor sin iniciarlo.
- `--name`: Parámetro que permite asignar un nombre al contenedor.
- `exam-simple`: Nombre asignado al contenedor creado.
- `-p`: Parámetro que permite configurar la publicación de puertos entre el equipo anfitrión y el contenedor.
- `127.0.0.1`: Dirección IP local que limita el acceso al propio equipo anfitrión.
- `18090`: Puerto del equipo anfitrión utilizado para acceder al servicio.
- `80`: Puerto interno del contenedor donde Nginx escucha las solicitudes HTTP.
- `nginx:1.28-alpine`: Imagen de Nginx versión 1.28 basada en Alpine Linux, utilizada para crear el contenedor.

Resultado: Se crea el contenedor `exam-simple` con el puerto `18090` vinculado al puerto interno `80`, quedando pendiente su inicio mediante `docker start exam-simple`. Una vez iniciado, se podrá acceder desde `http://127.0.0.1:18090`.

CASO 02 · EVIDENCIA 1016

Inicio y ejecución del contenedor Nginx mediante Docker.

Cargada: 31 oct. 2026, 23:27 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker start exam-simple
```

Descripción:

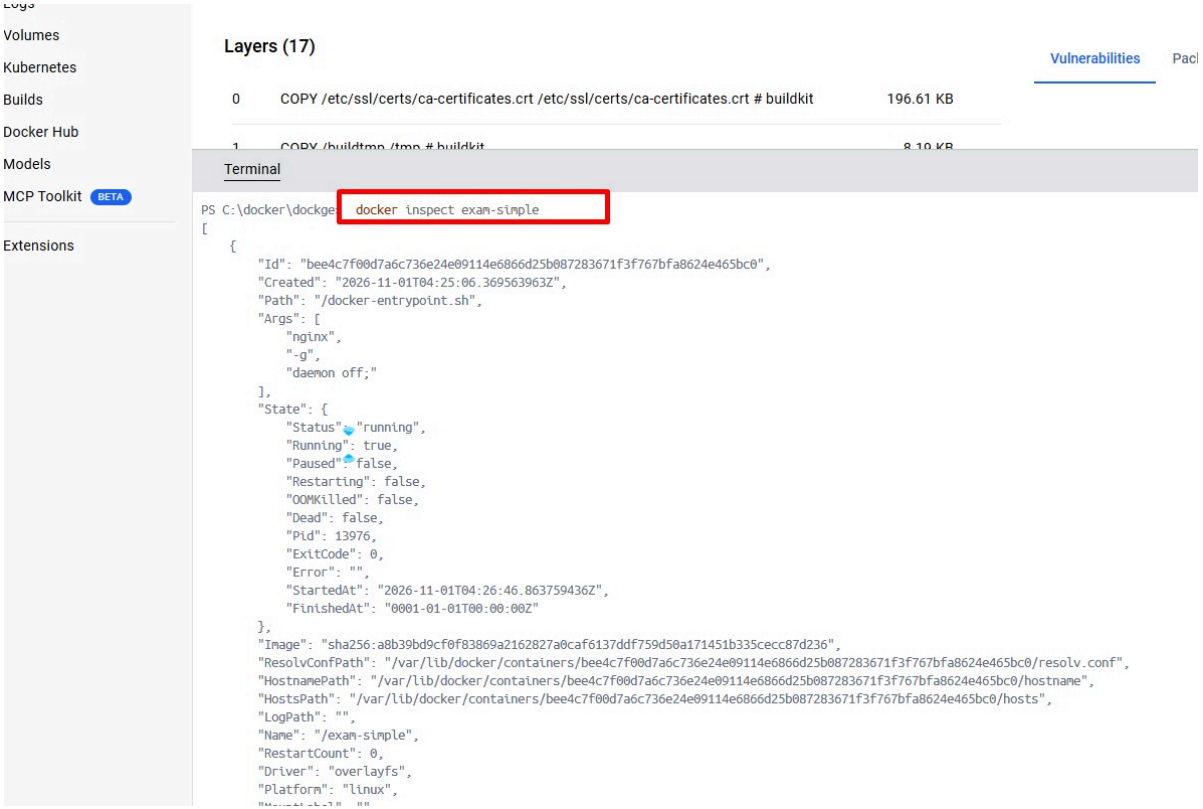
- `docker`: Herramienta que permite administrar imágenes y contenedores.
- `start`: Comando que permite iniciar un contenedor previamente creado que se encuentra detenido.
- `exam-simple`: Nombre del contenedor que se desea iniciar, creado anteriormente con la imagen `nginx:1.28-alpine`.

Resultado: Se inicia el contenedor `exam-simple`, poniendo en funcionamiento el servidor web Nginx. Una vez iniciado, se puede acceder al servicio desde el navegador mediante `http://127.0.0.1:18090`.

CASO 02 · EVIDENCIA 1017

Inspección y consulta de la configuración del contenedor Nginx mediante Docker.

Cargada: 31 oct. 2026, 23:53 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker inspect exam-simple
```

Descripción:

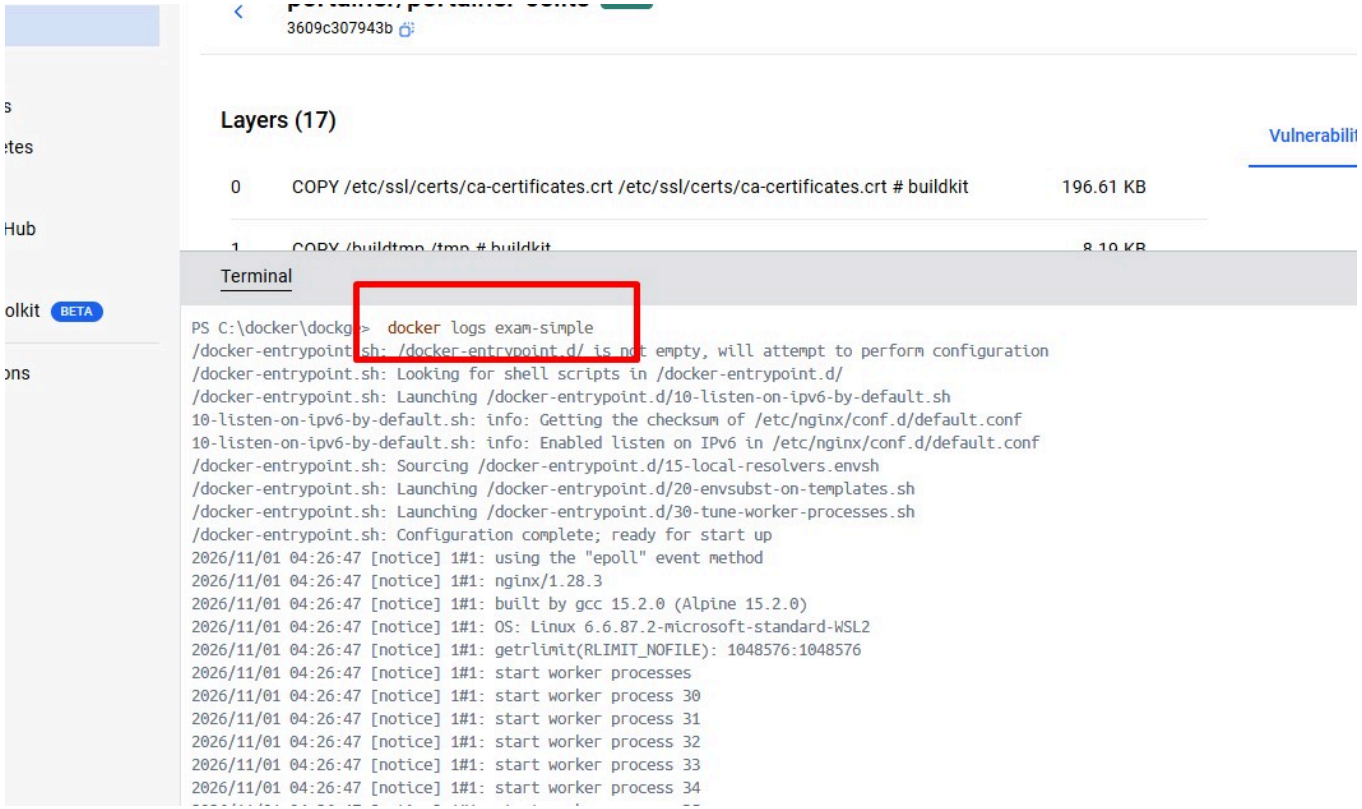
- `inspect`: Comando que permite consultar información detallada sobre la configuración, estado y propiedades de un contenedor en formato JSON.
- `exam-simple`: Nombre del contenedor del cual se desea obtener información detallada.

Resultado: Se muestra la información técnica del contenedor `exam-simple`, incluyendo su identificador, estado de ejecución, dirección IP, puertos configurados, imagen utilizada, redes, volúmenes y demás parámetros de configuración, permitiendo verificar su correcta creación y funcionamiento.

CASO 02 · EVIDENCIA 1018

Consulta y visualización de registros (logs) del contenedor Nginx mediante Docker.

Cargada: 31 oct. 2026, 23:56 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker logs exam-simple
```

Descripción:

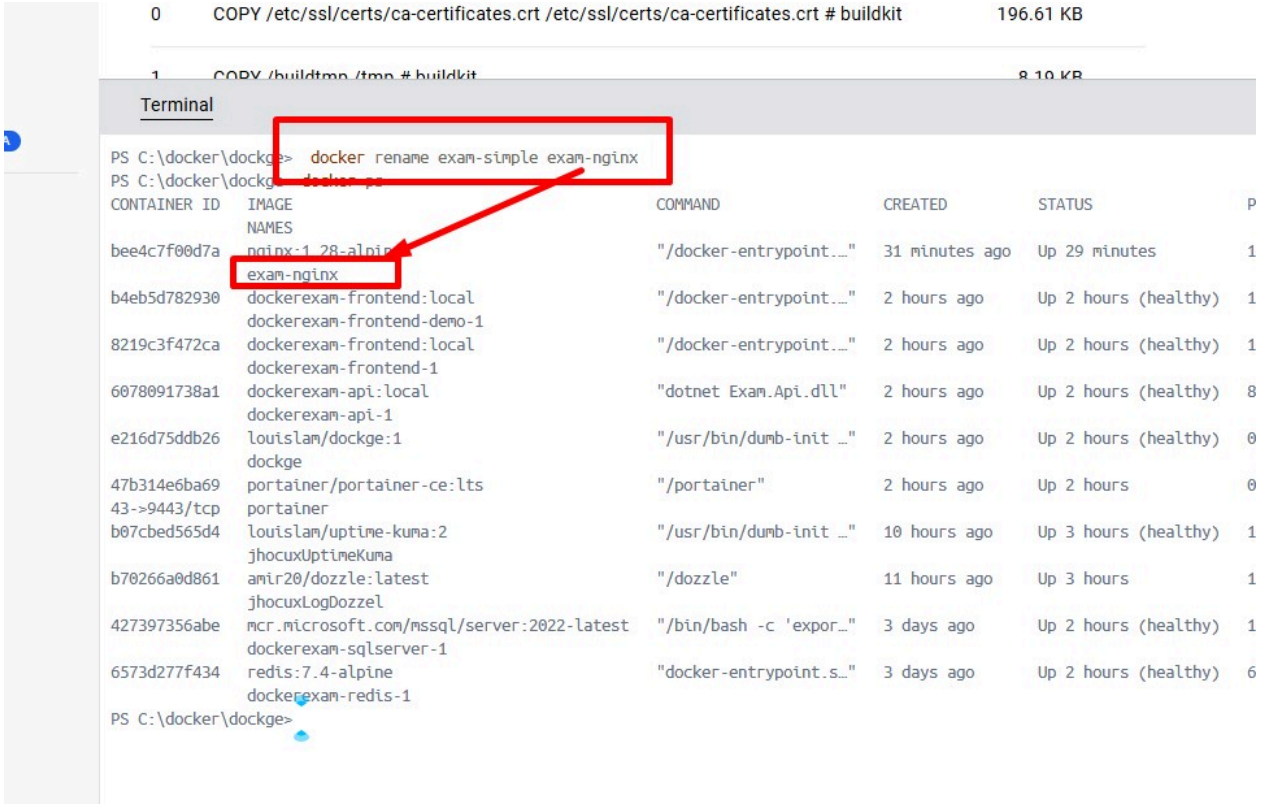
- `logs`: Comando que permite consultar y visualizar los registros generados por un contenedor durante su ejecución.
- `exam-simple`: Nombre del contenedor del cual se desea consultar los registros de actividad.

Resultado: Se muestran los registros del contenedor `exam-simple`, incluyendo mensajes de inicio, solicitudes HTTP, advertencias y posibles errores del servidor Nginx, permitiendo supervisar su funcionamiento e identificar problemas durante su ejecución.

CASO 02 · EVIDENCIA 1019

Cambio de nombre de un contenedor Nginx mediante Docker.

Cargada: 31 oct. 2026, 23:58 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker rename exam-simple exam-nginx
```

Descripción:

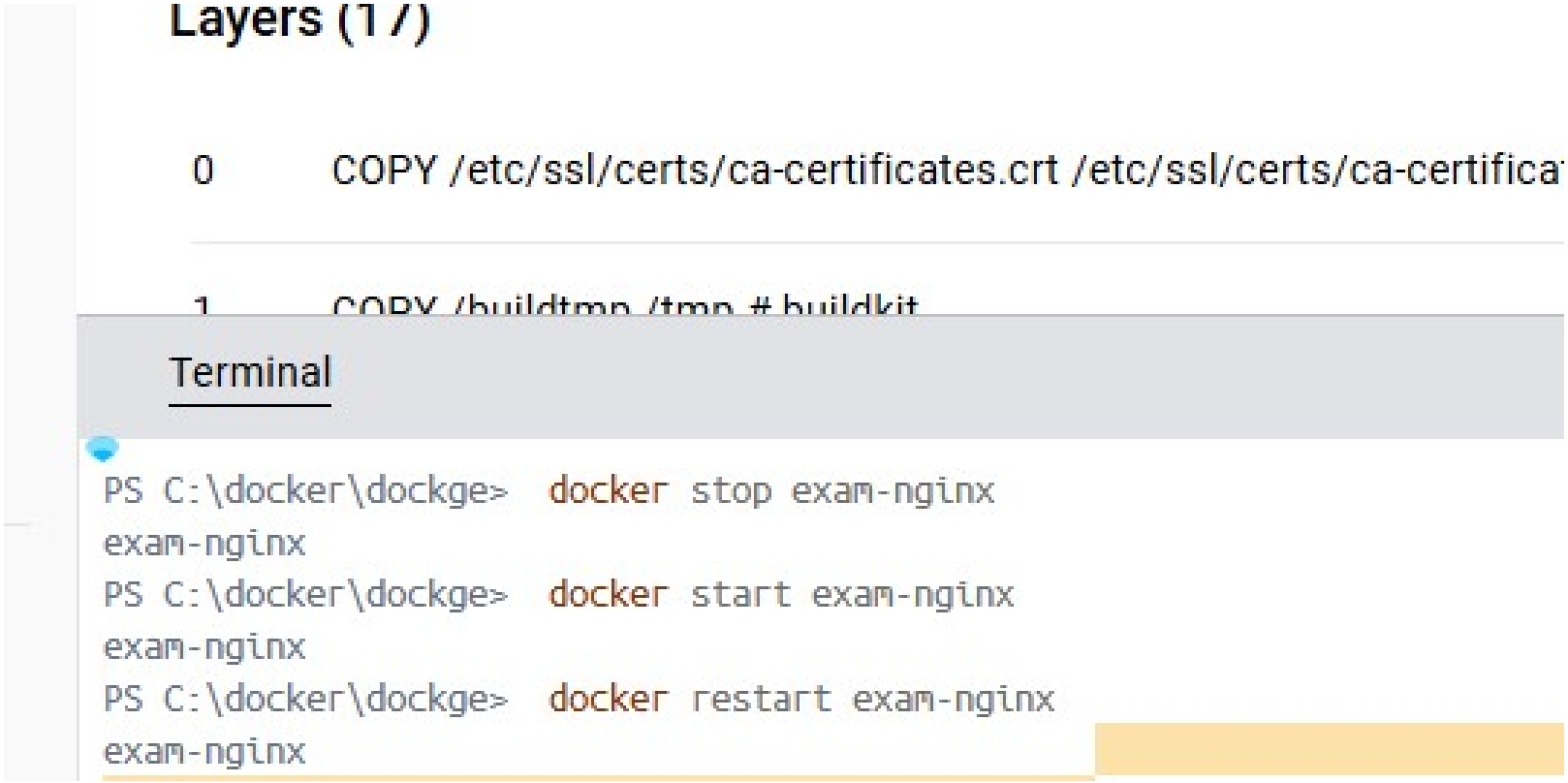
- `rename`: Comando que permite cambiar el nombre de un contenedor existente sin modificar su configuración ni afectar su estado de ejecución.
- `exam-simple`: Nombre actual del contenedor que se desea modificar.
- `exam-nginx`: Nuevo nombre que se asignará al contenedor.

Resultado: Se cambia el nombre del contenedor de `exam-simple` a `exam-nginx`, conservando su configuración, puertos, imagen y estado de ejecución. A partir de este momento, el contenedor podrá administrarse utilizando su nuevo nombre.

CASO 02 · EVIDENCIA 1020

Administración del ciclo de vida del contenedor Nginx mediante Docker.

Cargada: 01 nov. 2026, 00:07 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker stop exam-nginx
```

```
docker start exam-nginx
docker restart exam-nginx
docker stop exam-nginx
```

Descripción:

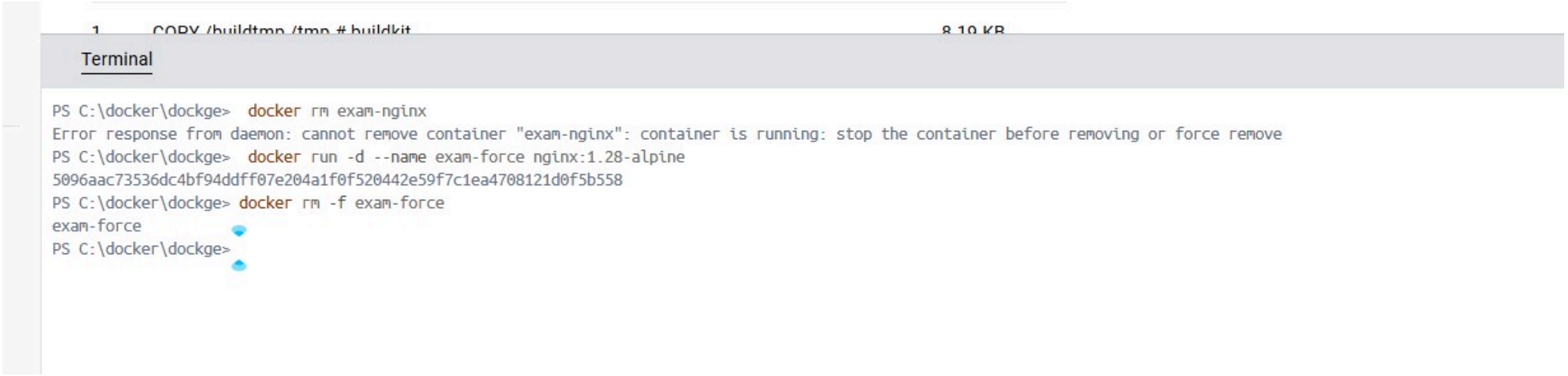
- `stop`: Comando que permite detener de manera controlada la ejecución de un contenedor, sin eliminarlo ni perder su configuración.
- `start`: Comando que permite iniciar nuevamente un contenedor previamente creado y que se encuentra detenido.
- `restart`: Comando que permite reiniciar un contenedor, deteniendo y volviendo a iniciar su ejecución automáticamente.
- `stop`: Comando utilizado nuevamente para detener el contenedor al finalizar las pruebas.
- `exam-nginx`: Nombre del contenedor sobre el cual se ejecutan las operaciones de administración.

Resultado: Se realizaron operaciones de detención, inicio y reinicio del contenedor `exam-nginx`, verificando la administración de su ciclo de vida. Finalmente, el contenedor quedó en estado detenido, conservando su configuración y permaneciendo disponible para futuras ejecuciones.

CASO 02 · EVIDENCIA 1021

Eliminación de contenedores Docker mediante comandos normales y forzados.

Cargada: 01 nov. 2026, 00:15 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker rm exam-nginx
```

```
docker run -d --name exam-force nginx:1.28-alpine
docker rm -f exam-force
```

Descripción:

- `rm`: Comando que permite eliminar un contenedor existente. Si el contenedor se encuentra en ejecución, primero debe detenerse.
- `exam-nginx`: Nombre del contenedor que se intenta eliminar. La operación genera un error porque el contenedor se encuentra en ejecución.
- `run`: Comando que permite crear e iniciar un nuevo contenedor a partir de una imagen Docker.
- `-d`: Parámetro que permite ejecutar el contenedor en segundo plano (modo detached).
- `--name`: Parámetro que permite asignar un nombre específico al contenedor.
- `exam-force`: Nombre asignado al nuevo contenedor para realizar las pruebas de eliminación forzada.
- `nginx:1.28-alpine`: Imagen de Nginx versión 1.28 basada en Alpine Linux, utilizada para crear el contenedor.
- `-f`: Parámetro que permite forzar la eliminación de un contenedor, incluso si se encuentra en ejecución.

Resultado: Se evidenció que Docker impide eliminar un contenedor en ejecución mediante `docker rm`. Posteriormente, se creó el contenedor `exam-force` y se eliminó exitosamente utilizando `docker rm -f`, demostrando la diferencia entre la eliminación normal y forzada de contenedores.

CASO 03

Conectando tres contenedores

Listo para revisión · Avance guardado: 01 nov. 2026, 02:08

Lo solicitado

Integrar tres contenedores usando sus IP internas obtenidas con inspect. Seleccionar imágenes diferentes de las usadas en clase.

Desarrollo y resultados

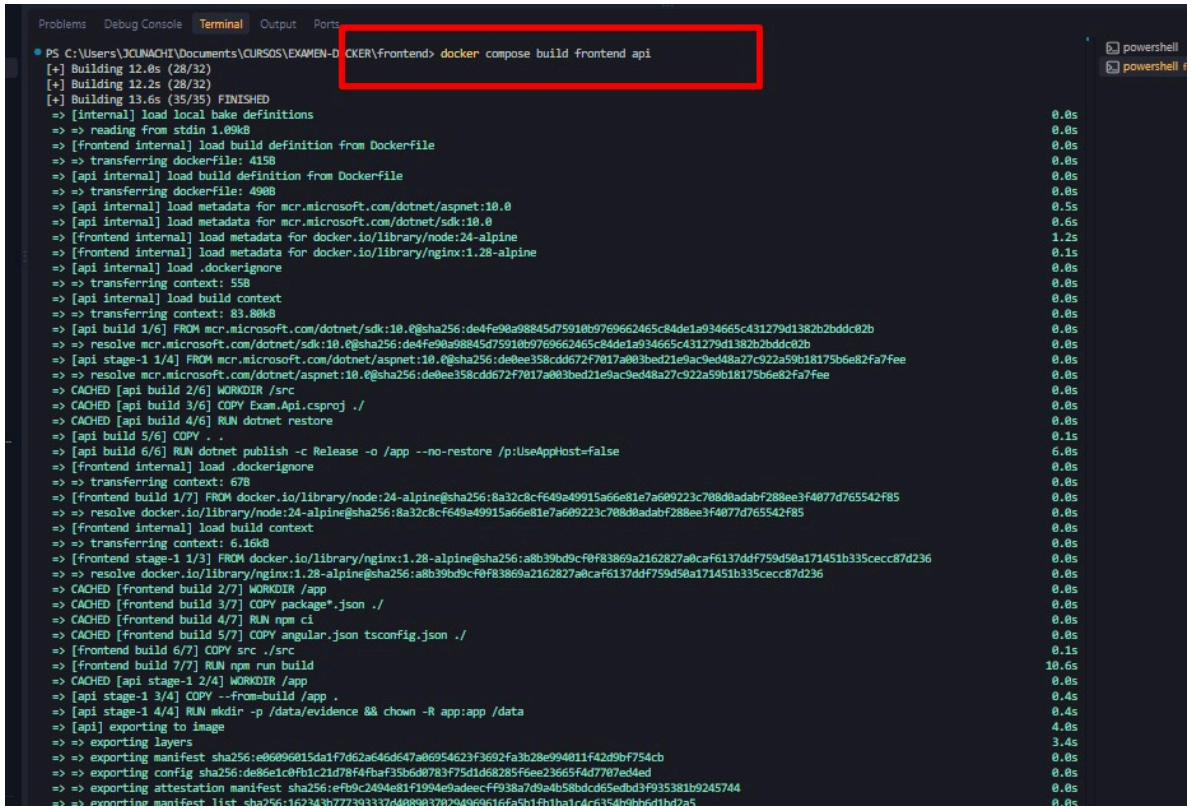
CASO 3 COMPLETADO CON EXITO

Evidencias adjuntas: 9. Las capturas y sus descripciones se presentan a continuación.

CASO 03 · EVIDENCIA 1023

Construcción de imágenes Docker para los servicios Frontend y API.

Cargada: 01 nov. 2026, 00:44 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker compose build frontend api
```

Descripción:

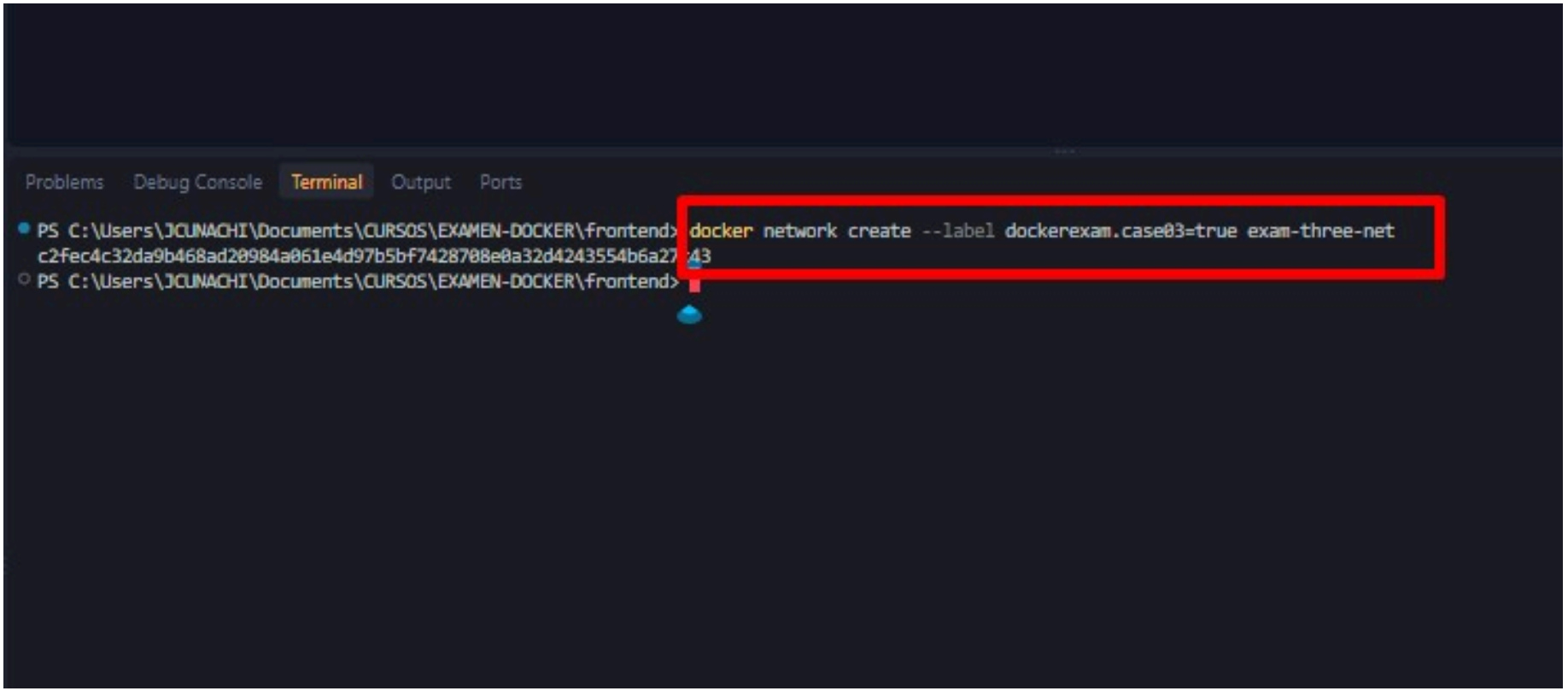
- `compose`: Permite administrar aplicaciones compuestas por múltiples servicios definidos en un archivo Docker Compose.
- `build`: Comando que permite construir imágenes Docker utilizando las instrucciones de sus respectivos Dockerfile.
- `frontend`: Nombre del servicio correspondiente a la aplicación web, cuya imagen Docker será construida.
- `api`: Nombre del servicio correspondiente al backend o API, cuya imagen Docker será construida.

Resultado: Se construyen las imágenes Docker de los servicios `frontend` y `api`, procesando sus respectivos Dockerfile y dependencias. Las imágenes quedan disponibles para crear y ejecutar contenedores posteriormente, sin iniciarlos automáticamente.

CASO 03 · EVIDENCIA 1024

Creación de una red Docker personalizada para la comunicación entre contenedores.

Cargada: 01 nov. 2026, 00:49 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker network create --label dockerexam.case03=true exam-three-net
```

Descripción:

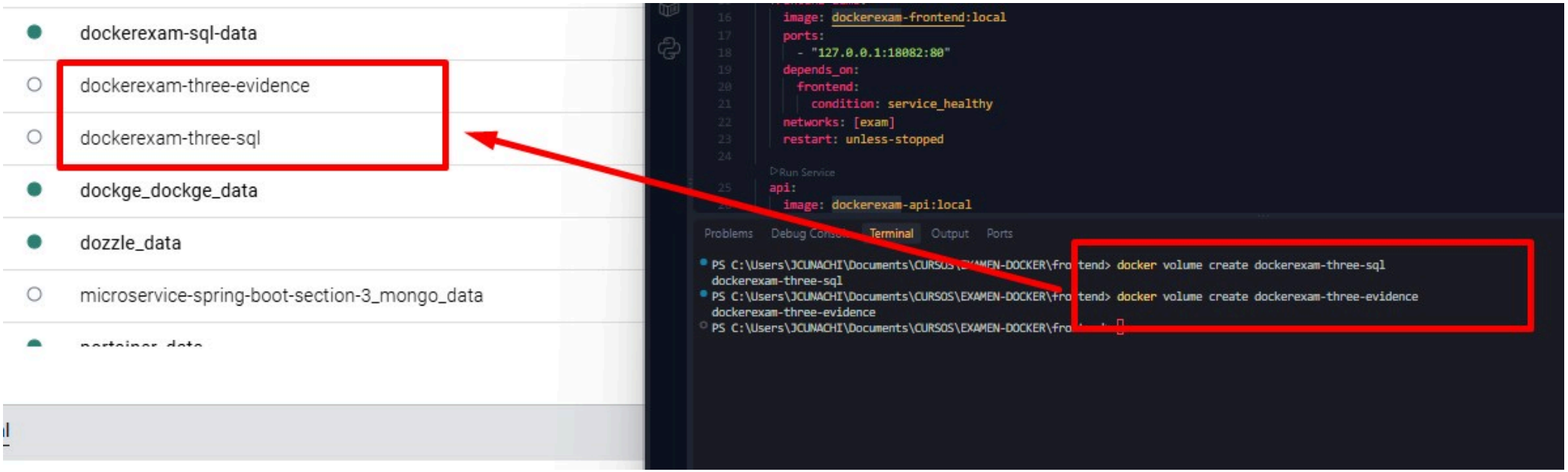
- `network`: Permite administrar las redes Docker utilizadas para la comunicación entre contenedores.
- `create`: Comando que permite crear una nueva red Docker.
- `--label`: Parámetro que permite asignar etiquetas personalizadas para identificar y organizar recursos Docker.
- `dockerexam.case03=true`: Etiqueta que identifica la red como parte del caso práctico 03.
- `exam-three-net`: Nombre asignado a la nueva red Docker, que permitirá conectar los diferentes servicios de la aplicación.

Resultado: Se crea la red personalizada `exam-three-net`, utilizando el controlador `bridge` por defecto, permitiendo la comunicación entre los contenedores que se conecten a ella. La red queda identificada mediante una etiqueta personalizada para facilitar su administración.

CASO 03 · EVIDENCIA 1025

Creación de volúmenes Docker para el almacenamiento persistente de datos y evidencias.

Cargada: 01 nov. 2026, 01:00 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker volume create dockerexam-three-sql
```

```
docker volume create dockerexam-three-evidence
```

Descripción:

- `volume`: Permite administrar los volúmenes Docker utilizados para almacenar información de manera persistente.
- `create`: Comando que permite crear nuevos volúmenes de almacenamiento.
- `dockerexam-three-sql`: Nombre del volumen destinado al almacenamiento persistente de los datos de SQL Server.
- `dockerexam-three-evidence`: Nombre del volumen destinado al almacenamiento persistente de evidencias y archivos generados por la aplicación.

Resultado: Se crean dos volúmenes Docker destinados al almacenamiento de datos de SQL Server y evidencias de la aplicación. Ambos quedan disponibles para ser utilizados por los contenedores, permitiendo conservar la información incluso cuando estos sean eliminados o recreados, siempre que los volúmenes no se eliminen.

CASO 03 · EVIDENCIA 1026

Creación y ejecución de un contenedor SQL Server 2022 con red personalizada y almacenamiento persistente mediante Docker.

Cargada: 01 nov. 2026, 01:08 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker run -d --name exam-three-sql --network exam-three-net -e ACCEPT_EULA=Y -e MSSQL_PID=Developer --env-file ./secrets/three-sql.txt -v dockerexam-three-sql:/var/opt/mssql mcr.microsoft.com/mssql/server:2022-latest
```

Descripción:

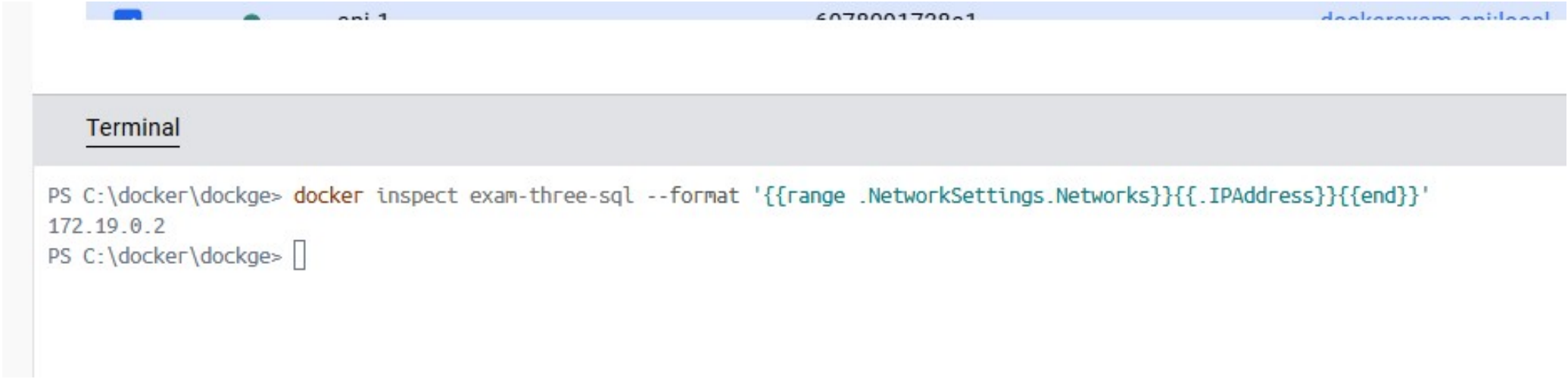
- `run`: Comando que permite crear e iniciar un contenedor a partir de una imagen Docker.
- `-d`: Ejecuta el contenedor en segundo plano (modo detached).
- `--name`: Permite asignar un nombre específico al contenedor.
- `exam-three-sql`: Nombre asignado al contenedor de SQL Server.
- `--network`: Permite conectar el contenedor a una red Docker existente.
- `exam-three-net`: Nombre de la red personalizada utilizada para la comunicación con otros contenedores.
- `-e`: Permite definir variables de entorno dentro del contenedor.
- `ACCEPT_EULA=Y`: Indica la aceptación de los términos de licencia de Microsoft SQL Server.
- `MSSQL_PID=Developer`: Establece la edición Developer de SQL Server, destinada a desarrollo y pruebas.
- `--env-file`: Permite cargar variables de entorno desde un archivo externo.
- `./secrets/three-sql.txt`: Ruta del archivo que contiene variables de configuración, como la contraseña del administrador de SQL Server.
- `-v`: Permite montar un volumen Docker para almacenar información de manera persistente.
- `dockerexam-three-sql`: Nombre del volumen Docker utilizado para almacenar los datos de SQL Server.
- `/var/opt/mssql`: Directorio interno del contenedor donde SQL Server almacena sus archivos de datos.
- `mcr.microsoft.com/mssql/server`: Imagen oficial de Microsoft SQL Server alojada en Microsoft Container Registry.
- `2022-latest`: Etiqueta que identifica la versión más reciente disponible de la imagen SQL Server 2022.

Resultado: Se crea e inicia el contenedor `exam-three-sql` utilizando SQL Server 2022 Developer, conectado a la red `exam-three-net` y configurado con variables de entorno externas. Además, se utiliza el volumen `dockerexam-three-sql` para conservar los datos de la base de datos, incluso cuando el contenedor sea eliminado o recreado, siempre que se reutilice el mismo volumen.

CASO 03 · EVIDENCIA 1028

Consulta de la dirección IP interna del contenedor SQL Server mediante Docker.

Cargada: 01 nov. 2026, 01:15 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker inspect exam-three-sql --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
```

Descripción:

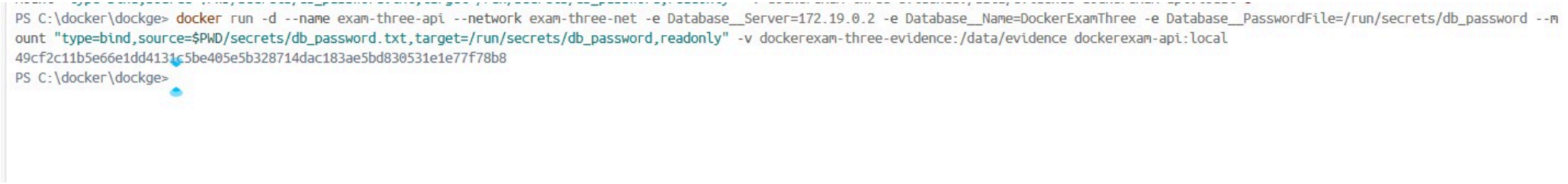
- `inspect`: Comando que permite consultar información detallada sobre la configuración, estado y propiedades de un contenedor Docker.
- `exam-three-sql`: Nombre del contenedor SQL Server del cual se desea obtener la dirección IP.
- `--format`: Parámetro que permite personalizar y filtrar la información mostrada mediante plantillas de Go.
- `range`: Instrucción que permite recorrer las redes asociadas al contenedor.
- `.NetworkSettings.Networks`: Propiedad que contiene la información de las redes Docker a las que está conectado el contenedor.
- `.IPAddress`: Propiedad que permite obtener la dirección IP interna asignada al contenedor en cada red.
- `end`: Indica el cierre de la instrucción `range`.

Resultado: Se obtiene la dirección IP interna del contenedor `exam-three-sql` dentro de la red Docker `exam-three-net`, permitiendo verificar su configuración de red y facilitar la comunicación con otros servicios o contenedores conectados a la misma red.

CASO 03 · EVIDENCIA 1030

Creación y ejecución de un contenedor API con conexión a SQL Server, gestión de credenciales y almacenamiento persistente mediante Docker.

Cargada: 01 nov. 2026, 01:40 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker run -d --name exam-three-api --network exam-three-net -e Database__Server=IP_SQL_REAL -e Database__Name=DockerExamThree -e Database__PasswordFile=/run/secrets/db_password --mount "type=bind,source=$PWD/secrets/db_password.txt,target=/run/secrets/db_password,readonly" -v dockerexam-three-evidence:/data/evidence dockerexam-api:local
```

Descripción:

- `run`: Permite crear e iniciar un contenedor a partir de una imagen Docker.
- `-d`: Ejecuta el contenedor en segundo plano (modo detached).
- `--name`: Permite asignar un nombre específico al contenedor.
- `exam-three-api`: Nombre asignado al contenedor que ejecutará el servicio backend o API.
- `--network`: Permite conectar el contenedor a una red Docker existente.
- `exam-three-net`: Red personalizada que permite la comunicación entre la API y el contenedor SQL Server.
- `-e`: Permite definir variables de entorno para configurar el funcionamiento de la aplicación.
- `Database__Server=IP_SQL_REAL`: Establece la dirección IP del servidor SQL Server al que se conectará la API. `IP_SQL_REAL` debe reemplazarse por la IP interna obtenida anteriormente.
- `Database__Name=DockerExamThree`: Especifica el nombre de la base de datos que utilizará la aplicación.
- `Database__PasswordFile=/run/secrets/db_password`: Define la ruta interna del archivo que contiene la contraseña de conexión a la base de datos.
- `--mount`: Permite montar archivos o directorios del equipo anfitrión dentro del contenedor.
- `type=bind`: Indica que se utilizará un montaje directo desde el sistema de archivos del equipo anfitrión.
- `source=$PWD/secrets/db_password.txt`: Especifica la ruta del archivo local que contiene la contraseña de la base de datos. `$PWD` representa el directorio actual en PowerShell.
- `target=/run/secrets/db_password`: Define la ubicación donde estará disponible el archivo de contraseña dentro del contenedor.
- `readonly`: Establece el archivo montado en modo de solo lectura para evitar modificaciones desde el contenedor.
- `-v`: Permite montar un volumen Docker para almacenar información de manera persistente.
- `dockerexam-three-evidence`: Nombre del volumen destinado al almacenamiento de evidencias generadas por la aplicación.
- `/data/evidence`: Directorio interno del contenedor donde se almacenarán las evidencias.
- `dockerexam-api:local`: Imagen Docker de la API, identificada mediante la etiqueta `local`, utilizada para crear el contenedor.

Resultado: Se crea e inicia el contenedor `exam-three-api`, conectado a la red `exam-three-net` para comunicarse con SQL Server. Se configuran los parámetros de conexión mediante variables de entorno, se monta un archivo de contraseña en modo de solo lectura y se incorpora un volumen persistente para almacenar las evidencias generadas por la

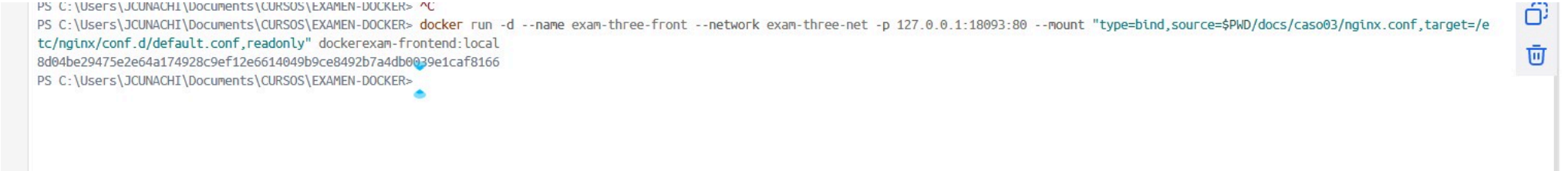
aplicación.

Nota: El comando no publica puertos hacia el equipo anfitrión, por lo que la API queda accesible principalmente desde otros contenedores conectados a la misma red. Además, la aplicación debe estar preparada para leer la contraseña desde el archivo configurado.

CASO 03 · EVIDENCIA 1032

Creación y ejecución de un contenedor Frontend con Nginx, configuración personalizada y publicación de puerto local mediante Docker.

Cargada: 01 nov. 2026, 01:55 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker run -d --name exam-three-front --network exam-three-net -p 127.0.0.1:18093:80 --mount "type=bind,source=$PWD/docs/caso03/nginx.conf,target=/etc/nginx/conf.d/default.conf,readonly" dockerexam-fron
tend:local
```

Descripción:

- `run`: Permite crear e iniciar un contenedor a partir de una imagen Docker.
- `-d`: Ejecuta el contenedor en segundo plano (modo detached).
- `--name`: Permite asignar un nombre específico al contenedor.
- `exam-three-front`: Nombre asignado al contenedor que ejecutará la aplicación Frontend.
- `--network`: Permite conectar el contenedor a una red Docker existente.
- `exam-three-net`: Red personalizada que permite la comunicación entre el Frontend y los demás servicios de la aplicación.
- `-p`: Permite publicar y vincular puertos entre el equipo anfitrión y el contenedor.
- `127.0.0.1`: Dirección IP local que restringe el acceso al servicio al propio equipo anfitrión.
- `18093`: Puerto del equipo anfitrión utilizado para acceder a la aplicación web.
- `80`: Puerto interno del contenedor donde Nginx atiende las solicitudes HTTP.
- `--mount`: Permite montar archivos o directorios del equipo anfitrión dentro del contenedor.
- `type=bind`: Indica que se utilizará un montaje directo de un archivo local.
- `source=$PWD/docs/caso03/nginx.conf`: Ruta del archivo de configuración personalizada de Nginx ubicado en el equipo anfitrión.
- `target=/etc/nginx/conf.d/default.conf`: Ruta interna donde se utilizará el archivo de configuración de Nginx.
- `readonly`: Establece el archivo montado en modo de solo lectura, evitando modificaciones desde el contenedor.
- `dockerexam-frontend:local`: Imagen Docker de la aplicación Frontend, identificada con la etiqueta `local`, utilizada para crear el contenedor.

Resultado: Se crea e inicia el contenedor `exam-three-front`, conectado a la red `exam-three-net`, utilizando una configuración personalizada de Nginx para publicar la aplicación web. El servicio queda disponible desde el navegador mediante `http://127.0.0.1:18093`, permitiendo acceder al Frontend y, si la configuración de Nginx lo contempla, comunicarse con la API desplegada en la misma red Docker.

CASO 03 · EVIDENCIA 1033

CASO 3 RESULTADO :Verificación de conectividad entre contenedores Docker y funcionamiento de la API mediante Nginx.

Cargada: 01 nov. 2026, 02:08 (Lima)

Terminal

```
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker inspect exam-three-sql --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
172.19.0.2
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker inspect exam-three-api --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
172.19.0.3
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker inspect exam-three-front --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
172.19.0.4
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker exec exam-three-api printenv Database__Server
172.19.0.2
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker exec exam-three-front cat /etc/nginx/conf.d/default.conf
server {
    listen 80;
    root /usr/share/nginx/html;
    client_max_body_size 9m;
    location /api/ { proxy_pass http://172.19.0.3:8080; }
    location / { try_files $uri $uri/ /index.html; }
}
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> curl.exe http://localhost:18093/api/health
{"status":"ok","api":".NET 10","database":"SQL Server 2022","version":"1.0.0"}
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER>
```

Descripción de la evidencia

Descripción:

- `docker inspect --format`: Permite consultar las direcciones IP internas de los contenedores SQL Server (`172.19.0.2`), API (`172.19.0.3`) y Frontend (`172.19.0.4`).

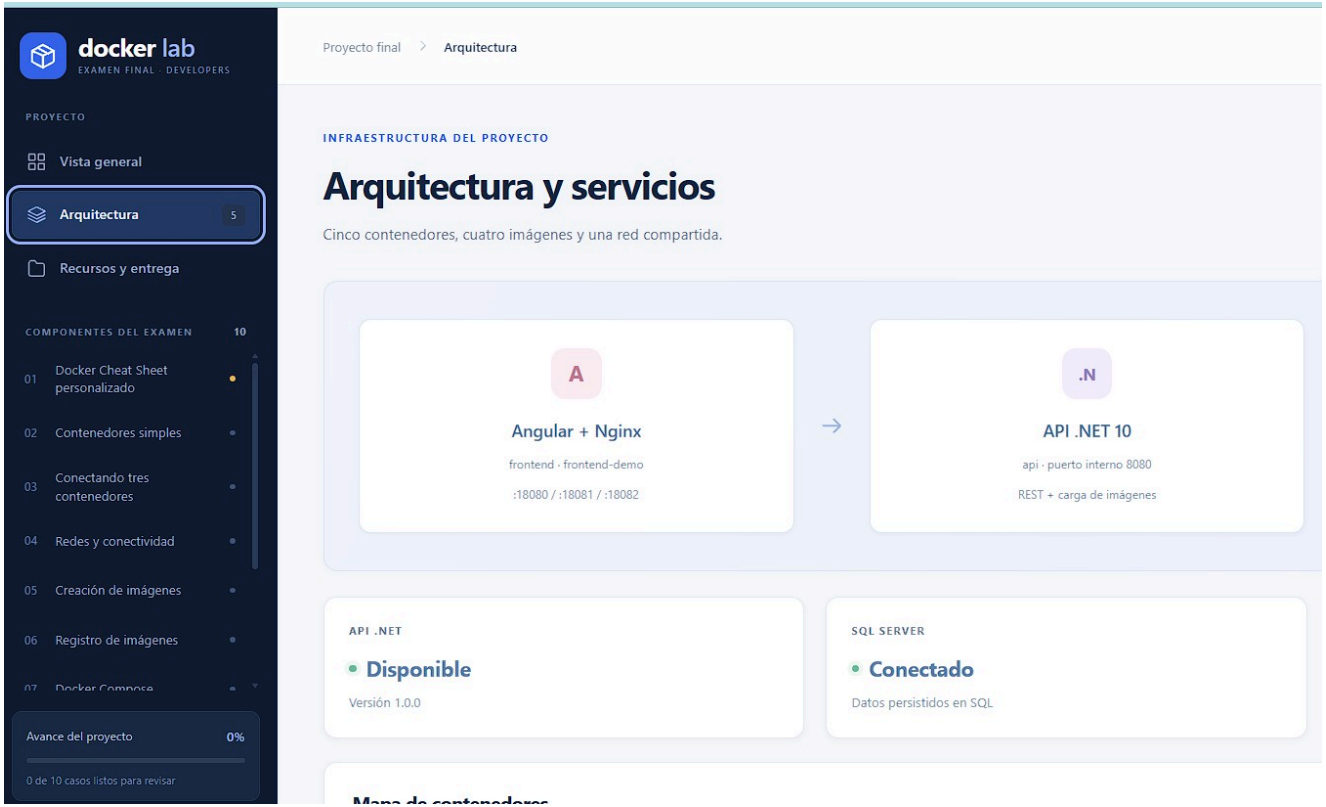
- `docker exec exam-three-api printenv Database__Server`: Verifica que la API tenga configurada correctamente la dirección IP del servidor SQL Server (`172.19.0.2`).
- `docker exec exam-three-front cat /etc/nginx/conf.d/default.conf`: Permite visualizar la configuración de Nginx y comprobar que las solicitudes `/api/` se redirigen hacia la API mediante el puerto `8080`.
- `curl.exe http://localhost:18093/api/health`: Realiza una solicitud HTTP para comprobar la disponibilidad y respuesta del servicio API a través del Frontend.

Resultado: Se verificó la configuración de red y comunicación entre los tres contenedores. La API respondió correctamente con el estado `"status":"ok"`, confirmando el funcionamiento del proxy inverso Nginx y la conectividad con SQL Server 2022, según la información reportada por el endpoint.

CASO 03 · EVIDENCIA 1034

EVIDENCIA FUNCIONAL DEL CASO 3

Cargada: 01 nov. 2026, 02:11 (Lima)



Descripción de la evidencia

CASO 3 RESUELTO

CASO 04

Redes y conectividad

Listo para revisión · Avance guardado: 01 nov. 2026, 02:51

Lo solicitado

Gestionar redes personalizadas: creación, inspección, renombre y eliminación; conectar y desconectar contenedores existentes, asignar una red al crear y limpiar redes sin uso.

Desarrollo y resultados

Se completaron satisfactoriamente las actividades del caso 4. Sin embargo, se identificó una precisión técnica respecto a la rúbrica de evaluación: Docker no permite renombrar directamente las redes existentes.

Para cumplir con este requerimiento, es necesario crear una nueva red con el nombre solicitado, desconectar los contenedores de la red original y conectarlos a la nueva red. Finalmente, se puede eliminar la red anterior si ya no está en uso.

Conclusión: Caso 4 completado exitosamente, aplicando el procedimiento alternativo para el cambio de nombre de redes en Docker.

Evidencias adjuntas: 8. Las capturas y sus descripciones se presentan a continuación.

CASO 04 · EVIDENCIA 1035

Creación de red Docker personalizada y despliegue de contenedores Nginx para pruebas de conectividad.

Cargada: 01 nov. 2026, 02:45 (Lima)

Terminal

```
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker network create --label dockerexam.case04=true exam-red-a
6506324b0b4dede93b7e4455c2e07c01c77bfc5f2d7b40586c12658c51d705a7
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker run -d --name exam-network --network exam-red-a nginx:1.28-alpine
b7d40b22cb034c0cf9f1c76da9ab1d2b4da8baab4f7a6af3e012d06cb0dc7ab8
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker run -d --name exam-probe --network exam-red-a nginx:1.28-alpine
5f1918caf23d783a04c369b5a33e40eab8d182c7ba08bcd00f79284c5657ea49
```

Descripción de la evidencia

Comandos ejecutados:

```
docker network create --label dockerexam.case04=true exam-red-a
docker run -d --name exam-network --network exam-red-a nginx:1.28-alpine
docker run -d --name exam-probe --network exam-red-a nginx:1.28-alpine
```

Descripción:

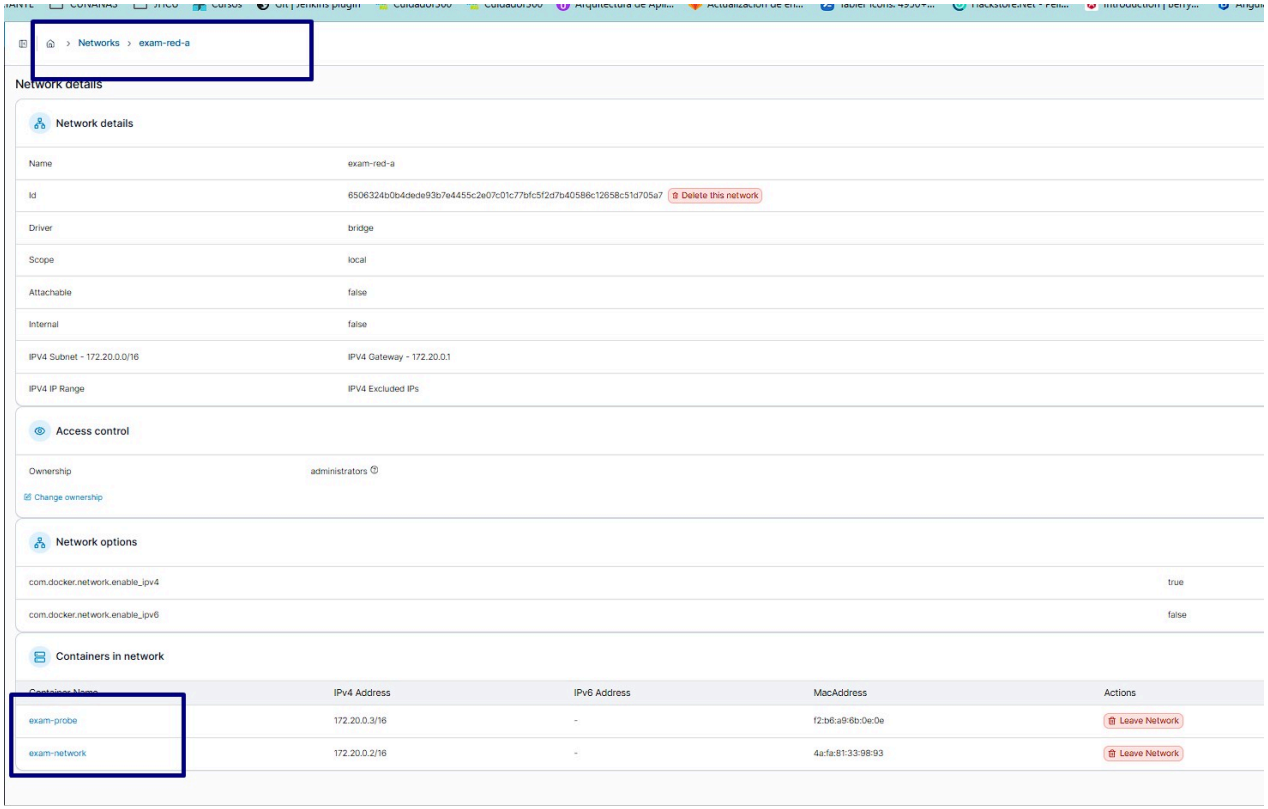
- `network create`: Permite crear una red Docker personalizada para facilitar la comunicación entre contenedores.
- `--label dockerexam.case04=true`: Asigna una etiqueta que identifica la red como parte del caso práctico 04.
- `exam-red-a`: Nombre de la red Docker creada.
- `run`: Permite crear e iniciar un contenedor utilizando una imagen Docker.
- `-d`: Ejecuta los contenedores en segundo plano.
- `--name`: Permite asignar un nombre específico a cada contenedor.
- `exam-network`: Nombre del primer contenedor Nginx utilizado para las pruebas de red.
- `exam-probe`: Nombre del segundo contenedor Nginx utilizado para verificar la comunicación entre servicios.
- `--network exam-red-a`: Conecta ambos contenedores a la misma red Docker.
- `nginx:1.28-alpine`: Imagen oficial de Nginx versión 1.28 basada en Alpine Linux.

Resultado: Se crea la red personalizada `exam-red-a` y se despliegan dos contenedores Nginx (`exam-network` y `exam-probe`) conectados a ella, estableciendo un entorno para realizar pruebas de conectividad, resolución de nombres y comunicación interna entre contenedores Docker.

CASO 04 · EVIDENCIA 1036

CONTENEDORES CONECTADOS A LA MISMA RED

Cargada: 01 nov. 2026, 02:46 (Lima)



Descripción de la evidencia

docker network inspect exam-red-a

CASO 04 · EVIDENCIA 1037

Inspección y verificación de la configuración de una red Docker personalizada.

Cargada: 01 nov. 2026, 02:46 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker network inspect exam-red-a
```

Descripción:

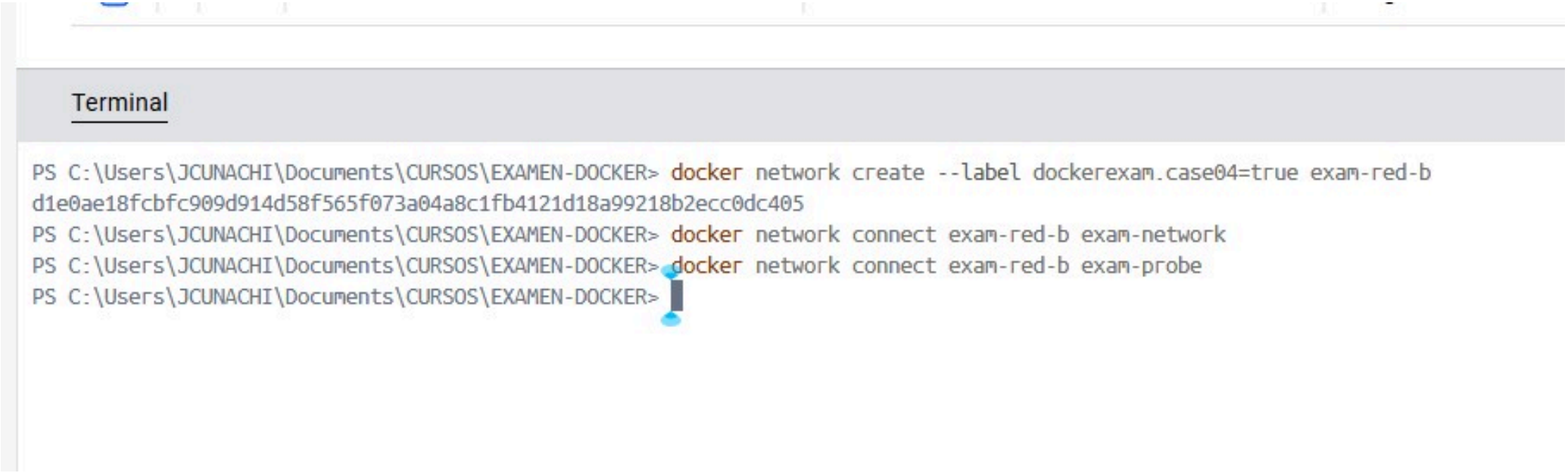
- `network`: Permite administrar y consultar las redes Docker utilizadas para la comunicación entre contenedores.
- `inspect`: Comando que permite visualizar información detallada sobre la configuración y propiedades de una red Docker.
- `exam-red-a`: Nombre de la red personalizada que se desea inspeccionar.

Resultado: Se muestra la información detallada de la red `exam-red-a`, incluyendo su identificador, controlador de red (`bridge`), rango de direcciones IP, puerta de enlace, etiquetas y contenedores conectados (`exam-network` y `exam-probe`). Esto permite verificar que ambos contenedores pertenecen a la misma red y están configurados para comunicarse internamente.

CASO 04 · EVIDENCIA 1038

Creación de una segunda red Docker y conexión de contenedores existentes a múltiples redes.

Cargada: 01 nov. 2026, 02:47 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker network create --label dockerexam.case04=true exam-red-b
```

```
docker network connect exam-red-b exam-network
docker network connect exam-red-b exam-probe
```

Descripción:

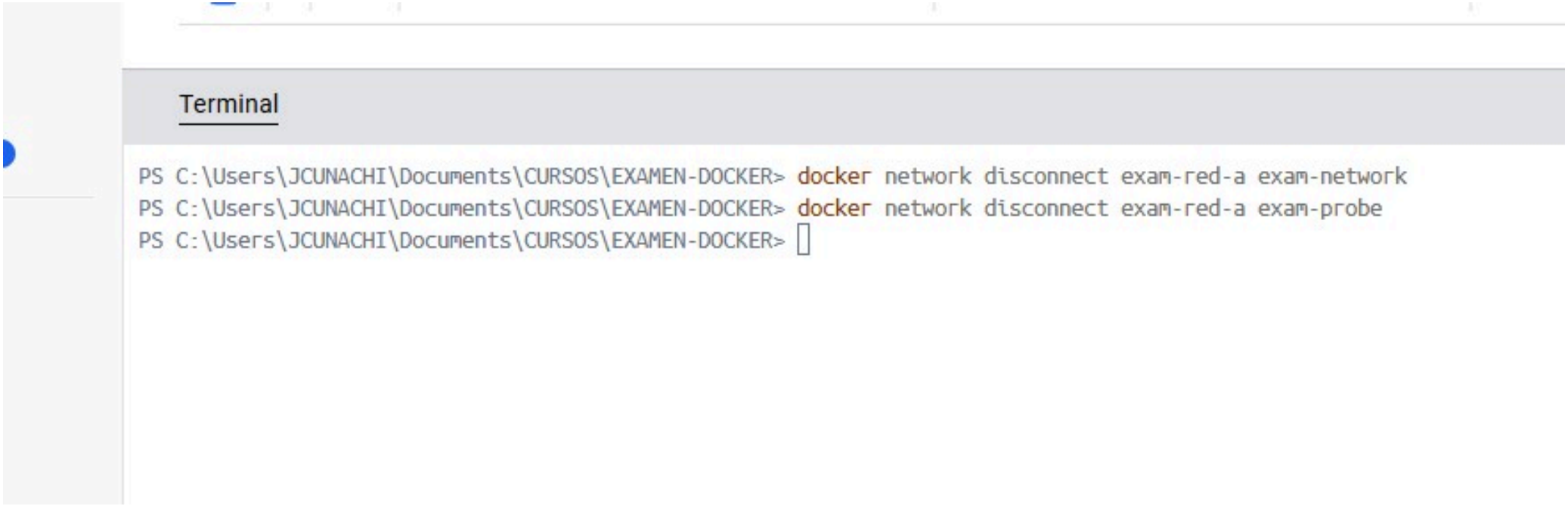
- `network create`: Permite crear una nueva red Docker personalizada.
- `--label dockerexam.case04=true`: Asigna una etiqueta para identificar la red como parte del caso práctico 04.
- `exam-red-b`: Nombre asignado a la segunda red Docker.
- `network connect`: Comando que permite conectar contenedores existentes a una red Docker adicional sin necesidad de recrearlos.
- `exam-network`: Primer contenedor Nginx que se conecta a la nueva red.
- `exam-probe`: Segundo contenedor Nginx que se conecta a la nueva red.

Resultado: Se creó exitosamente la red `exam-red-b` y se conectaron los contenedores `exam-network` y `exam-probe`, manteniendo su conexión original a `exam-red-a`. De esta manera, ambos contenedores pertenecen a dos redes Docker simultáneamente, permitiendo realizar pruebas de conectividad y comunicación entre múltiples redes.

CASO 04 · EVIDENCIA 1039

Desconexión de contenedores de una red Docker personalizada.

Cargada: 01 nov. 2026, 02:48 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker network disconnect exam-red-a exam-network
```

```
docker network disconnect exam-red-a exam-probe
```

Descripción:

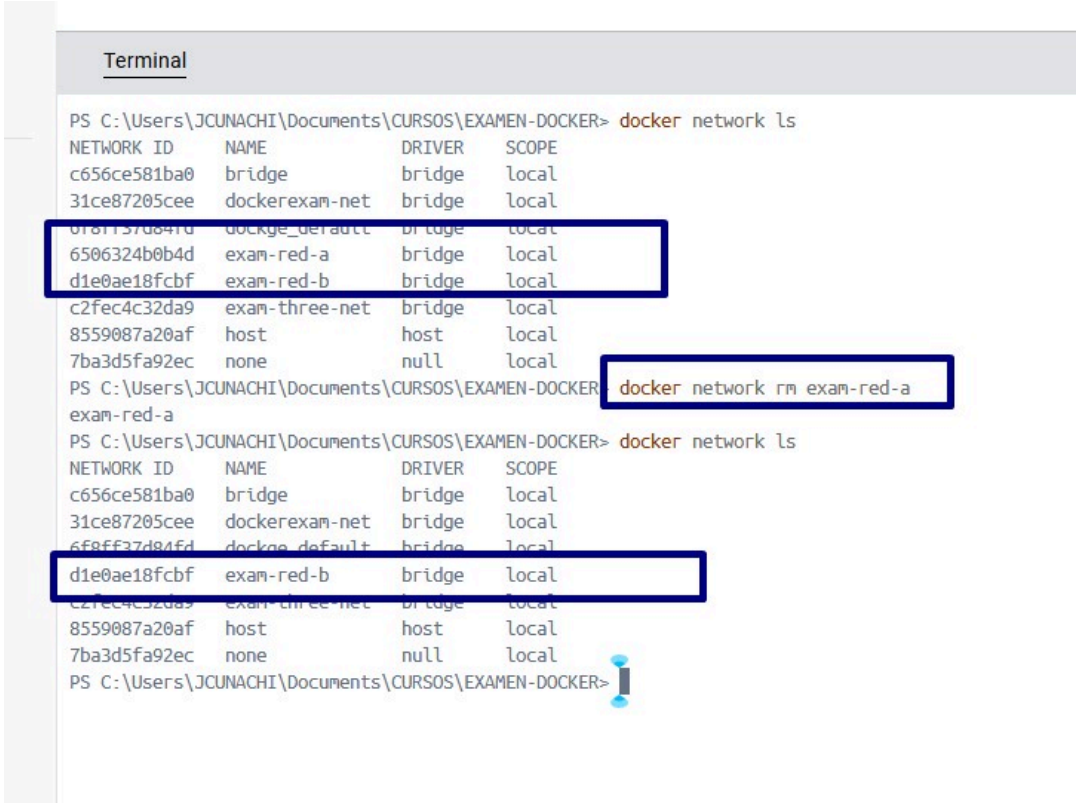
- `network`: Permite administrar las redes Docker utilizadas para la comunicación entre contenedores.
- `disconnect`: Comando que permite desconectar un contenedor de una red Docker sin eliminarlo ni detener su ejecución.
- `exam-red-a`: Nombre de la red Docker de la cual se desconectarán los contenedores.
- `exam-network`: Primer contenedor Nginx que será desconectado de la red.
- `exam-probe`: Segundo contenedor Nginx que será desconectado de la red.

Resultado: Se desconectaron los contenedores `exam-network` y `exam-probe` de la red `exam-red-a`, conservando su conexión a `exam-red-b`. Esto permite demostrar la administración dinámica de conexiones de red en Docker, sin necesidad de detener ni recrear los contenedores.

CASO 04 · EVIDENCIA 1040

Listado, eliminación y verificación de redes Docker mediante comandos de administración.

Cargada: 01 nov. 2026, 02:49 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker network ls
```

```
docker network rm exam-red-a
docker network ls
```

Descripción:

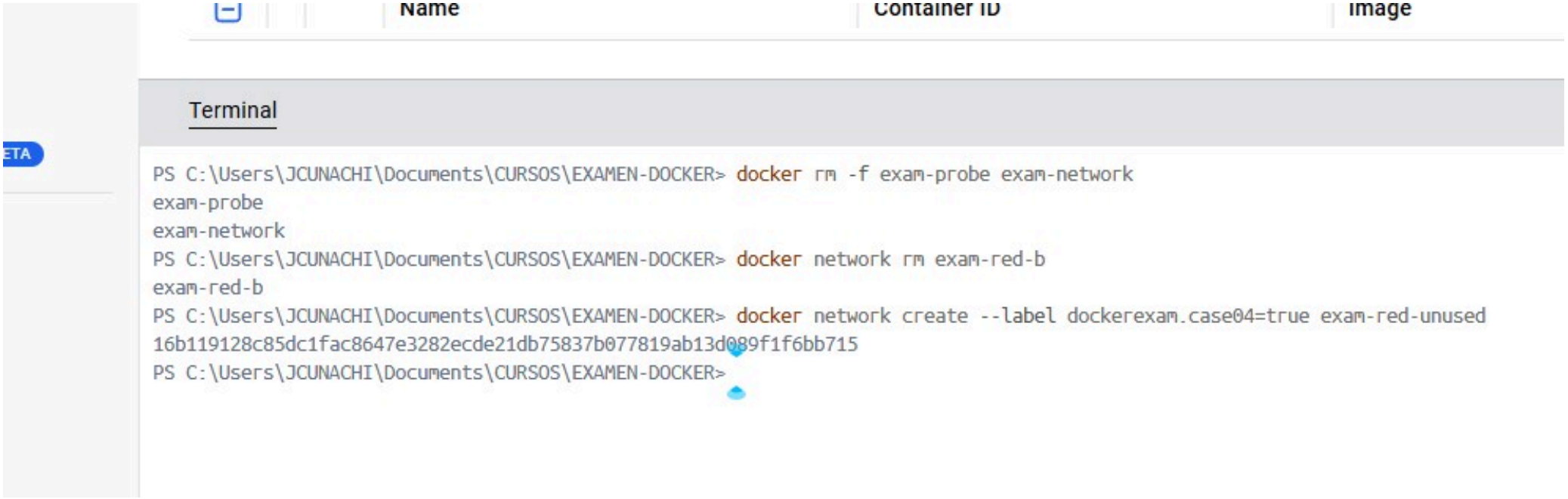
- `network ls`: Permite listar todas las redes Docker existentes, mostrando su identificador, nombre, controlador y ámbito de funcionamiento.
- `network rm`: Comando que permite eliminar una red Docker existente, siempre que no tenga contenedores conectados.
- `exam-red-a`: Nombre de la red personalizada que se desea eliminar.
- `network ls` (segunda ejecución): Permite verificar que la red eliminada ya no aparece en el listado de redes disponibles.

Resultado: Se verificaron las redes Docker existentes y se eliminó correctamente la red `exam-red-a`, después de desconectar sus contenedores. Finalmente, se consultó nuevamente el listado, confirmando que la red fue eliminada y que `exam-red-b` y las demás redes permanecen disponibles.

CASO 04 · EVIDENCIA 1041

Eliminación de contenedores y redes Docker, y creación de una red sin contenedores asociados.

Cargada: 01 nov. 2026, 02:50 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker rm -f exam-probe exam-network
```

```
docker network rm exam-red-b
docker network create --label dockerexam.case04=true exam-red-unused
```

Descripción:

- `rm`: Comando que permite eliminar contenedores Docker existentes.
- `-f`: Parámetro que permite forzar la eliminación de contenedores, incluso si se encuentran en ejecución.
- `exam-probe exam-network`: Nombres de los dos contenedores que serán eliminados.
- `network rm`: Permite eliminar una red Docker existente que no tenga contenedores conectados.

CASO 04 · EVIDENCIA 1042

Filtrado, limpieza y verificación de redes Docker mediante etiquetas personalizadas.

Cargada: 01 nov. 2026, 02:50 (Lima)

Terminal

```
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker network ls --filter label=dockerexam.case04=true
NETWORK ID     NAME              DRIVER    SCOPE
16b119128c85   exam-red-unused   bridge    local
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker network prune --filter label=dockerexam.case04=true
WARNING! This will remove all custom networks not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Networks:
exam-red-unused

PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER> docker network ls --filter label=dockerexam.case04=true
NETWORK ID   NAME      DRIVER    SCOPE
PS C:\Users\JCUNACHI\Documents\CURSOS\EXAMEN-DOCKER>
```

Descripción de la evidencia

Comandos ejecutados:

```
docker network ls --filter label=dockerexam.case04=true
```

```
docker network prune --filter label=dockerexam.case04=true
docker network ls --filter label=dockerexam.case04=true
```

Descripción:

- `network ls`: Permite listar las redes Docker existentes.
- `--filter`: Parámetro que permite filtrar los resultados según criterios específicos.
- `label=dockerexam.case04=true`: Filtra las redes que contienen la etiqueta correspondiente al caso práctico 04.
- `network prune`: Comando que permite eliminar las redes Docker personalizadas que no están siendo utilizadas por ningún contenedor.
- `--filter label=dockerexam.case04=true`: Limita la limpieza a las redes sin uso que tengan la etiqueta especificada.
- `y`: Confirma la operación de eliminación cuando Docker solicita autorización.
- `network ls` (segunda ejecución): Permite verificar que las redes filtradas hayan sido eliminadas correctamente.

Resultado: Se identificó la red `exam-red-unused` mediante su etiqueta personalizada y se eliminó utilizando `docker network prune`. Finalmente, se ejecutó nuevamente el listado filtrado, confirmando que no quedan redes con la etiqueta `dockerexam.case04=true`, demostrando la correcta limpieza y administración de recursos de red Docker.

CASO 05

Creación de imágenes

Listo para revisión · Avance guardado: 01 nov. 2026, 03:44

Lo solicitado

Crear imágenes propias con Dockerfile y Dockerignore personalizados, generar contenedores y verificar que las aplicaciones funcionen.

Desarrollo y resultados

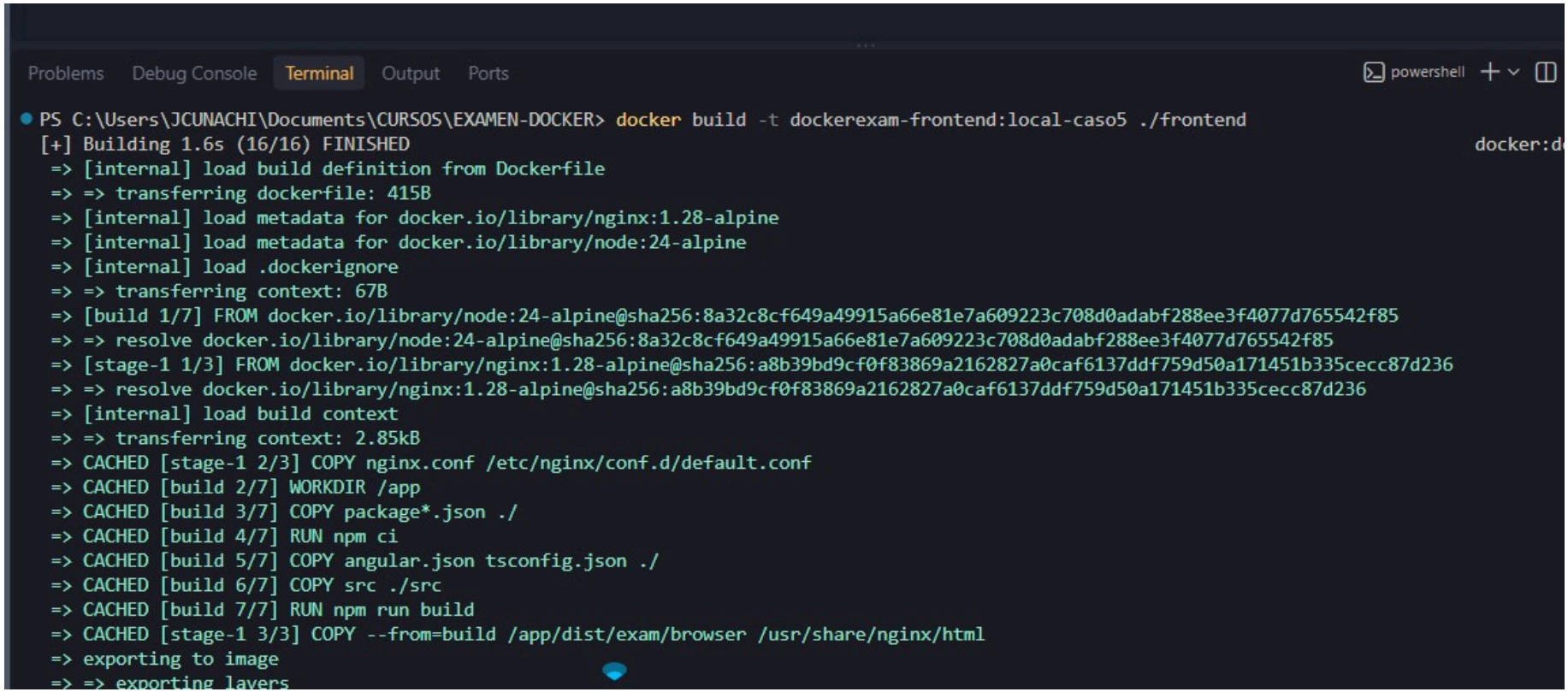
Concluido

Evidencias adjuntas: 8. Las capturas y sus descripciones se presentan a continuación.

CASO 05 · EVIDENCIA 1043

Construcción de imagen Docker personalizada para el Frontend del caso práctico 05.

Cargada: 01 nov. 2026, 02:58 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker build -t dockerexam-frontend:local-caso5 ./frontend
```

Descripción:

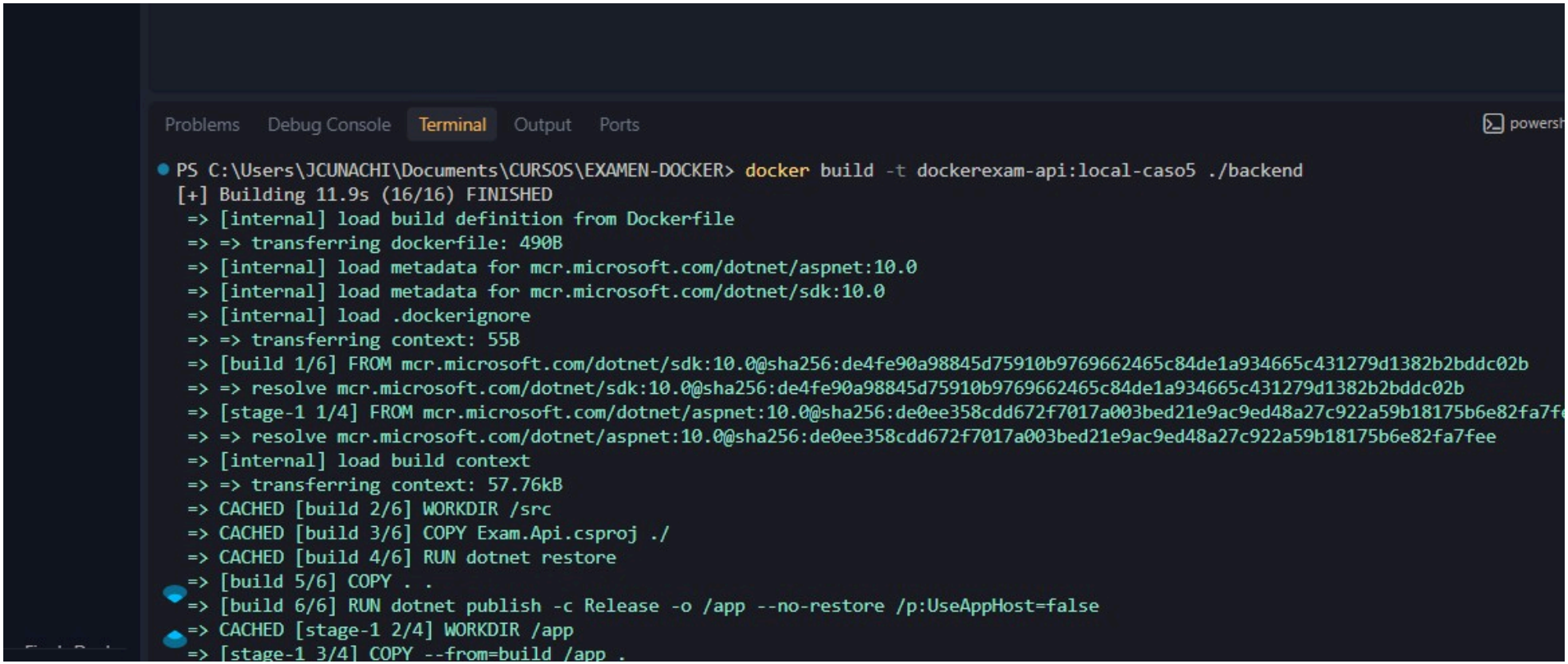
- `build`: Comando que permite construir una imagen Docker utilizando las instrucciones definidas en un Dockerfile.
- `-t`: Parámetro que permite asignar un nombre y una etiqueta a la imagen construida.
- `dockerexam-frontend`: Nombre asignado a la imagen Docker correspondiente a la aplicación Frontend.
- `local-caso5`: Etiqueta que permite identificar la imagen como parte del caso práctico 05.
- `./frontend`: Ruta relativa del directorio que contiene el código fuente y los archivos necesarios para construir la imagen, incluido el Dockerfile.

Resultado: Se construye la imagen Docker `dockerexam-frontend:local-caso5` utilizando los archivos del directorio `frontend`. La imagen queda almacenada localmente y disponible para crear y ejecutar contenedores posteriormente, sin iniciarlos automáticamente.

CASO 05 · EVIDENCIA 1044

Construcción de imagen Docker personalizada para el backend del caso práctico 05.

Cargada: 01 nov. 2026, 02:59 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker build -t dockerexam-api:local-caso5 ./backend
```

Descripción:

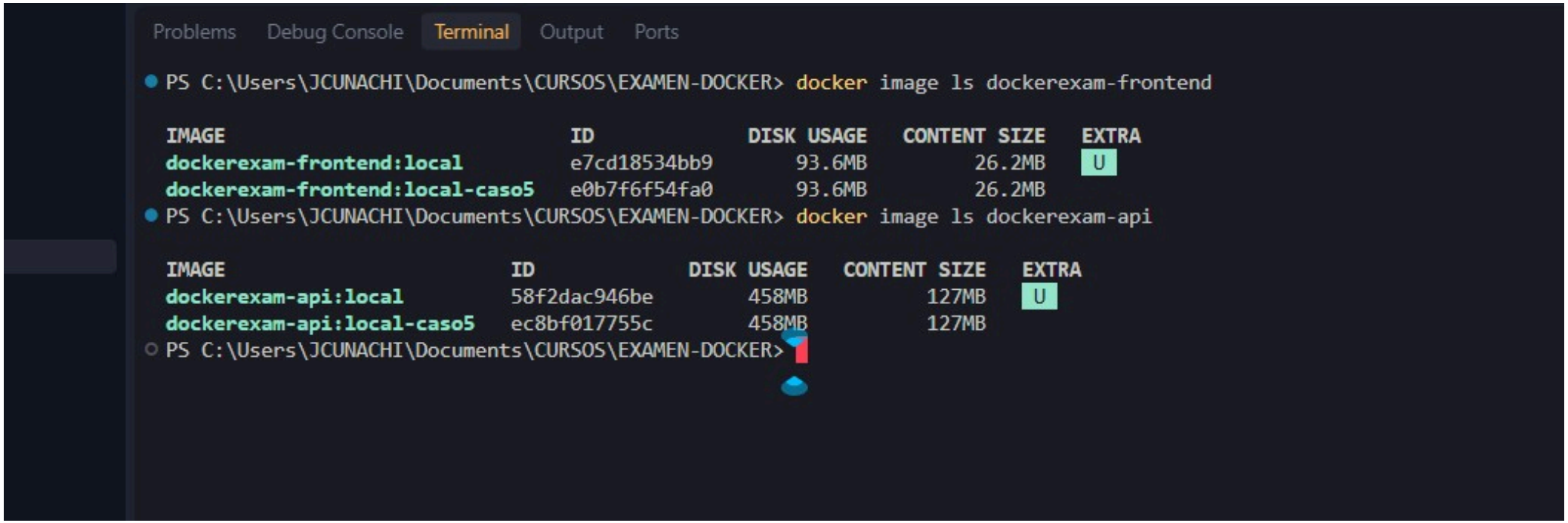
- `build`: Comando que permite construir una imagen Docker utilizando las instrucciones definidas en un Dockerfile.
- `-t`: Parámetro que permite asignar un nombre y una etiqueta a la imagen construida.
- `dockerexam-api`: Nombre asignado a la imagen Docker correspondiente al API.
- `local-caso5`: Etiqueta que permite identificar la imagen como parte del caso práctico 05.
- `./backend`: Ruta relativa del directorio que contiene el código fuente y los archivos necesarios para construir la imagen, incluido el Dockerfile.

Resultado: Se construye la imagen Docker `dockerexam-backend:local-caso5` utilizando los archivos del directorio `backend`. La imagen queda almacenada localmente y disponible para crear y ejecutar contenedores posteriormente, sin iniciarlos automáticamente.

CASO 05 · EVIDENCIA 1045

Listado y verificación de imágenes Docker del Frontend y API correspondientes al caso práctico 05.

Cargada: 01 nov. 2026, 03:03 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker image ls dockerexam-frontend
```

```
docker image ls dockerexam-api
```

Descripción:

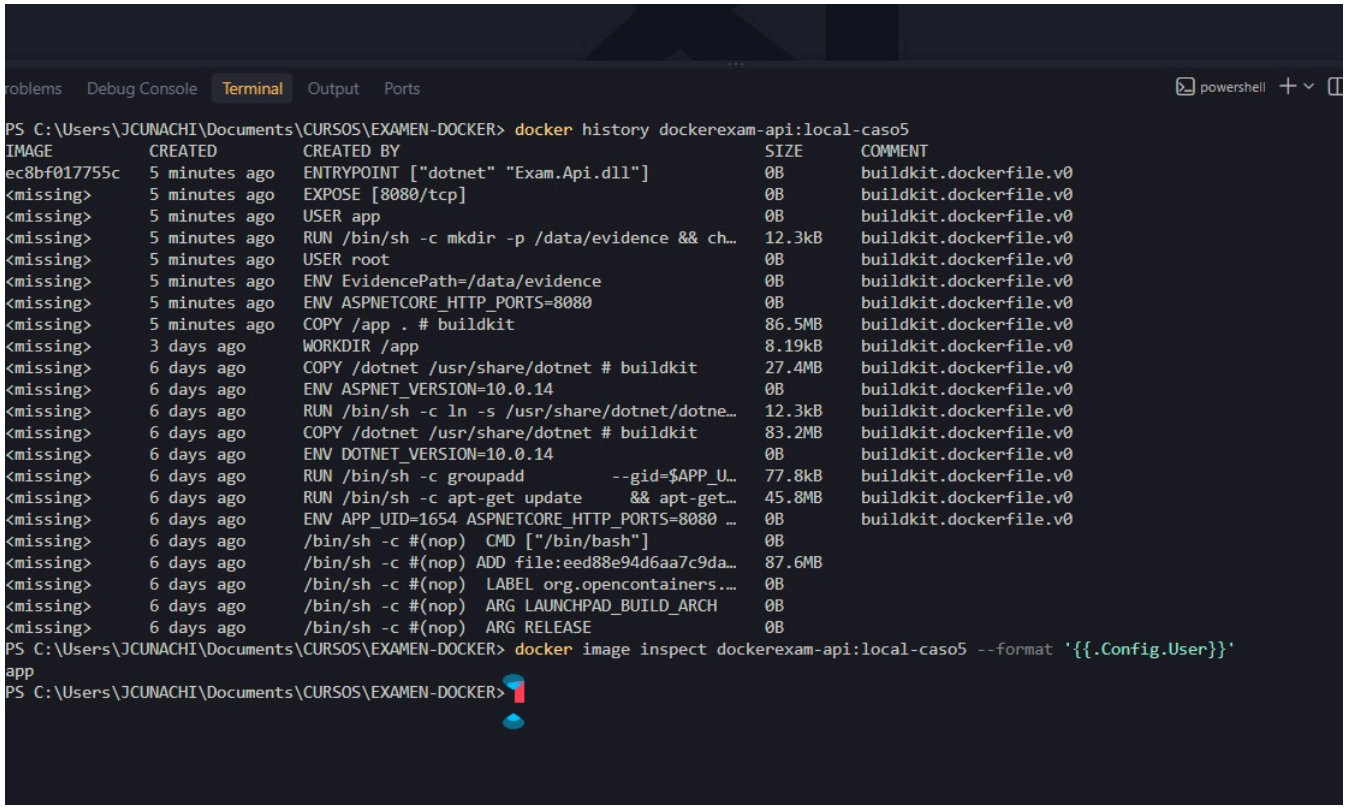
- `image`: Permite administrar las imágenes Docker almacenadas localmente.
- `ls`: Comando que permite listar las imágenes disponibles, mostrando sus etiquetas, identificadores y tamaños.
- `dockerexam-frontend`: Nombre de la imagen correspondiente a la aplicación Frontend que se desea consultar.
- `dockerexam-api`: Nombre de la imagen correspondiente al servicio Backend o API que se desea consultar.
- `local`: Etiqueta que identifica las imágenes locales existentes.
- `local-caso5`: Etiqueta que identifica las imágenes construidas para el caso práctico 05.

Resultado: Se verificó la existencia de las imágenes Docker del Frontend y API, ambas con las etiquetas `local` y `local-caso5`. El Frontend presenta un tamaño de disco de 93.6 MB y la API de 458 MB, confirmando que las imágenes se encuentran disponibles localmente para su posterior despliegue en contenedores.

CASO 05 · EVIDENCIA 1046

Inspección del historial de construcción y verificación del usuario configurado en la imagen Docker de la API.

Cargada: 01 nov. 2026, 03:06 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker history dockerexam-api:local-caso5
docker image inspect dockerexam-api:local-caso5 --format '{{.Config.User}}'
```

Descripción:

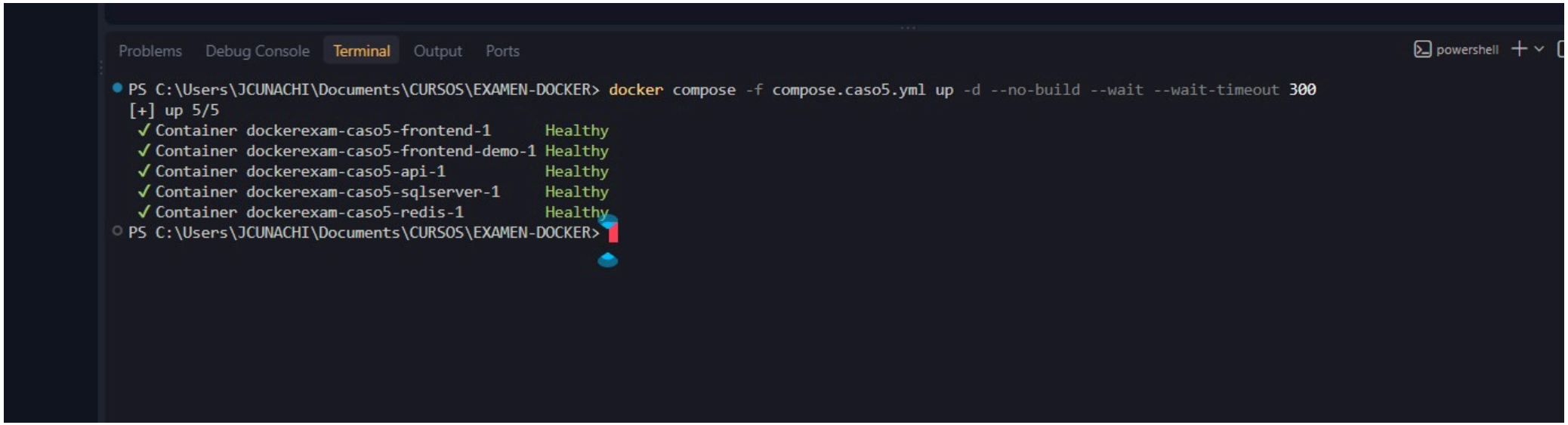
- `history`: Permite visualizar el historial de capas de una imagen Docker, mostrando las instrucciones ejecutadas durante su construcción, tamaños y fechas de creación.
- `dockerexam-api:local-caso5`: Nombre y etiqueta de la imagen Docker correspondiente a la API del caso práctico 05.
- `image inspect`: Permite consultar información detallada sobre la configuración y propiedades de una imagen Docker.
- `--format`: Parámetro que permite filtrar y mostrar información específica mediante plantillas de Go.
- `{{.Config.User}}`: Permite obtener el usuario configurado para ejecutar los procesos dentro del contenedor.

Resultado: Se verificó el historial de construcción de la imagen `dockerexam-api:local-caso5`, identificando las capas, instrucciones, configuración del puerto `8080` y ejecución de la aplicación .NET. Asimismo, se confirmó que la imagen tiene configurado el usuario `app`, permitiendo ejecutar la aplicación con un usuario distinto de `root`, como buena práctica de seguridad.

CASO 05 · EVIDENCIA 1048

Despliegue exitoso de cinco servicios Docker Compose con verificación de estado saludable.

Cargada: 01 nov. 2026, 03:36 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker compose -f compose.caso5.yml up -d --no-build --wait --wait-timeout 300
```

Descripción:

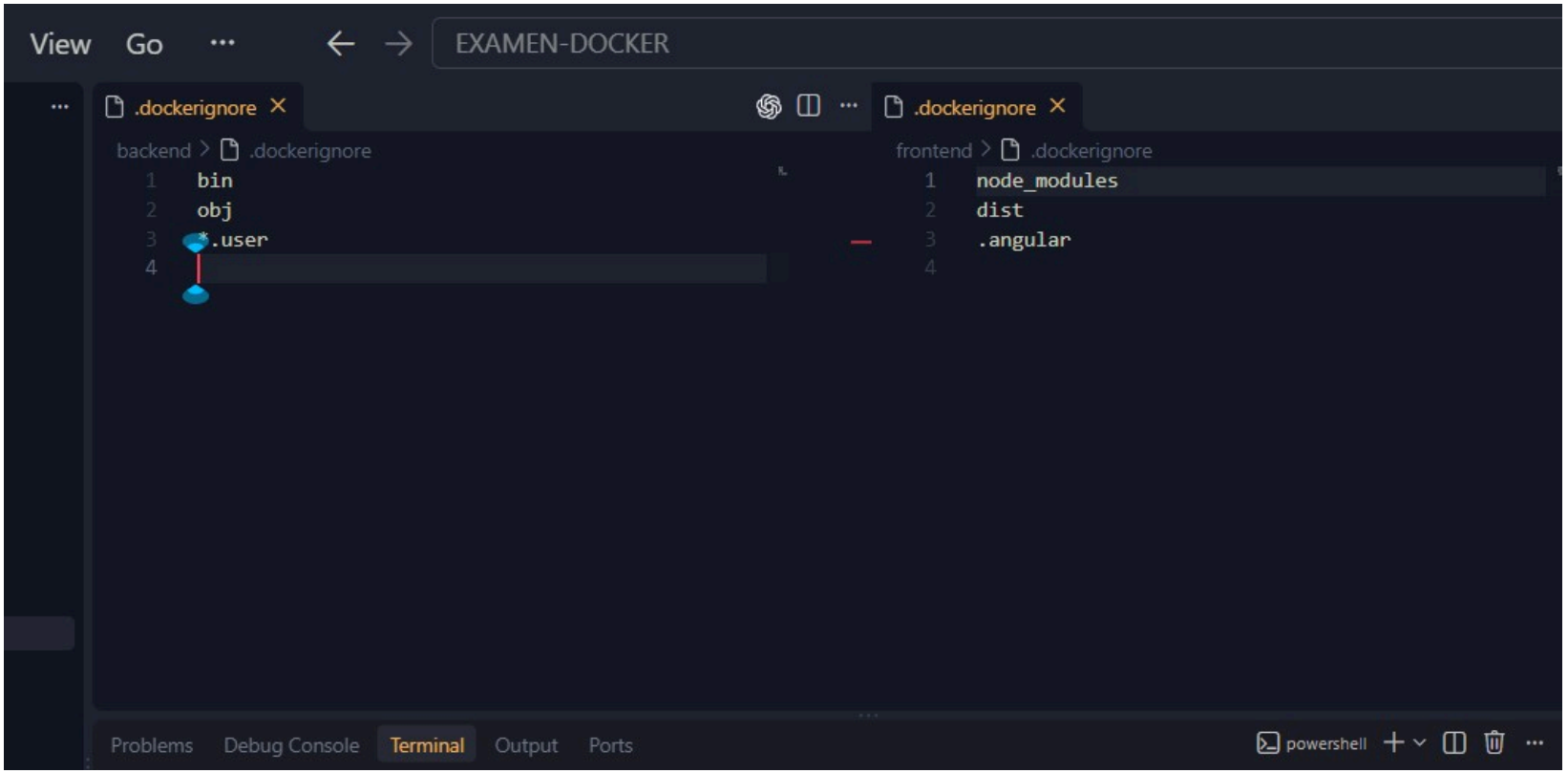
- `compose`: Permite administrar aplicaciones compuestas por múltiples servicios Docker.
- `-f compose.caso5.yml`: Especifica el archivo YAML que contiene la configuración de los servicios del caso práctico 05.
- `up`: Permite crear e iniciar los contenedores definidos en el archivo Compose.
- `-d`: Ejecuta los contenedores en segundo plano.
- `--no-build`: Utiliza las imágenes Docker existentes sin reconstruirlas.
- `--wait`: Espera hasta que los servicios alcancen el estado de ejecución o saludable según su configuración.
- `--wait-timeout 300`: Establece un tiempo máximo de espera de 300 segundos.

Resultado: Se desplegaron exitosamente los cinco contenedores del caso práctico 05: `frontend`, `frontend-demo`, `api`, `sqlserver` y `redis`. Todos alcanzaron el estado Healthy (5/5), confirmando que las verificaciones de salud configuradas se completaron satisfactoriamente y que los servicios están operativos según dichas comprobaciones.

CASO 05 · EVIDENCIA 1049

Docker Ignore Front y Backend

Cargada: 01 nov. 2026, 03:39 (Lima)



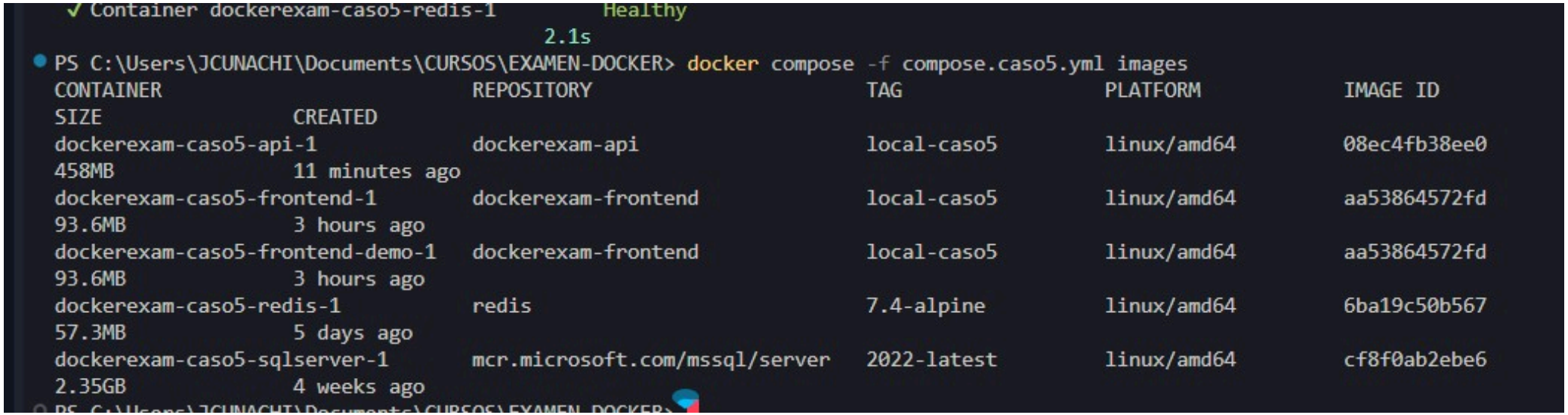
Descripción de la evidencia

Sin descripción registrada.

CASO 05 · EVIDENCIA 1050

Listado y verificación de imágenes Docker utilizadas por los servicios del caso práctico 05.

Cargada: 01 nov. 2026, 03:41 (Lima)



Descripción de la evidencia

Comando ejecutado:

```
docker compose -f compose.caso5.yml images
```

Descripción:

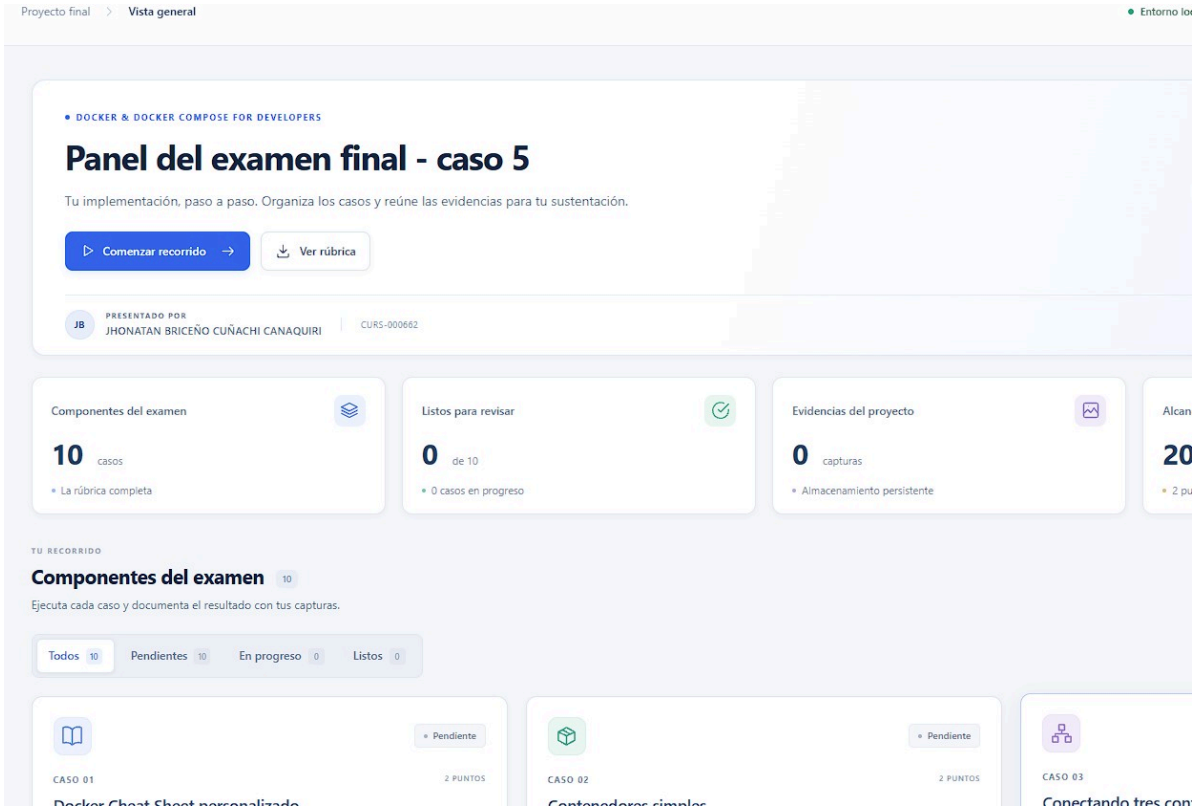
- `compose`: Permite administrar aplicaciones compuestas por múltiples servicios Docker.
- `-f compose.caso5.yml`: Especifica el archivo YAML que contiene la configuración de los servicios del caso práctico 05.
- `images`: Comando que permite listar las imágenes Docker utilizadas por los contenedores del proyecto.
- `REPOSITORY`: Muestra el nombre del repositorio de cada imagen.
- `TAG`: Identifica la versión o etiqueta de la imagen utilizada.
- `PLATFORM`: Indica la plataforma y arquitectura para la cual fue construida la imagen.
- `IMAGE ID`: Identificador único de cada imagen Docker.
- `SIZE`: Muestra el tamaño de las imágenes utilizadas.

Resultado: Se verificaron las imágenes utilizadas por los cinco contenedores del caso práctico 05: `api`, `frontend`, `frontend-demo`, `redis` y `sqlserver`. Se confirmó el uso de imágenes personalizadas con la etiqueta `local-caso5`, además de Redis `7.4-alpine` y SQL Server `2022-latest`, todas bajo la plataforma `linux/amd64`.

CASO 05 · EVIDENCIA 1051

caso 5 conluido

Cargada: 01 nov. 2026, 03:44 (Lima)



Descripción de la evidencia

Se concluyó satisfactoriamente

CASO 06

Registro de imágenes

Listo para revisión · Avance guardado: 02 nov. 2026, 00:44

Lo solicitado

Publicar imágenes propias en dos o más repositorios públicos y privados, usando Docker Hub y una nube específica. Descargar, crear contenedores y verificar su funcionamiento.

Desarrollo y resultados

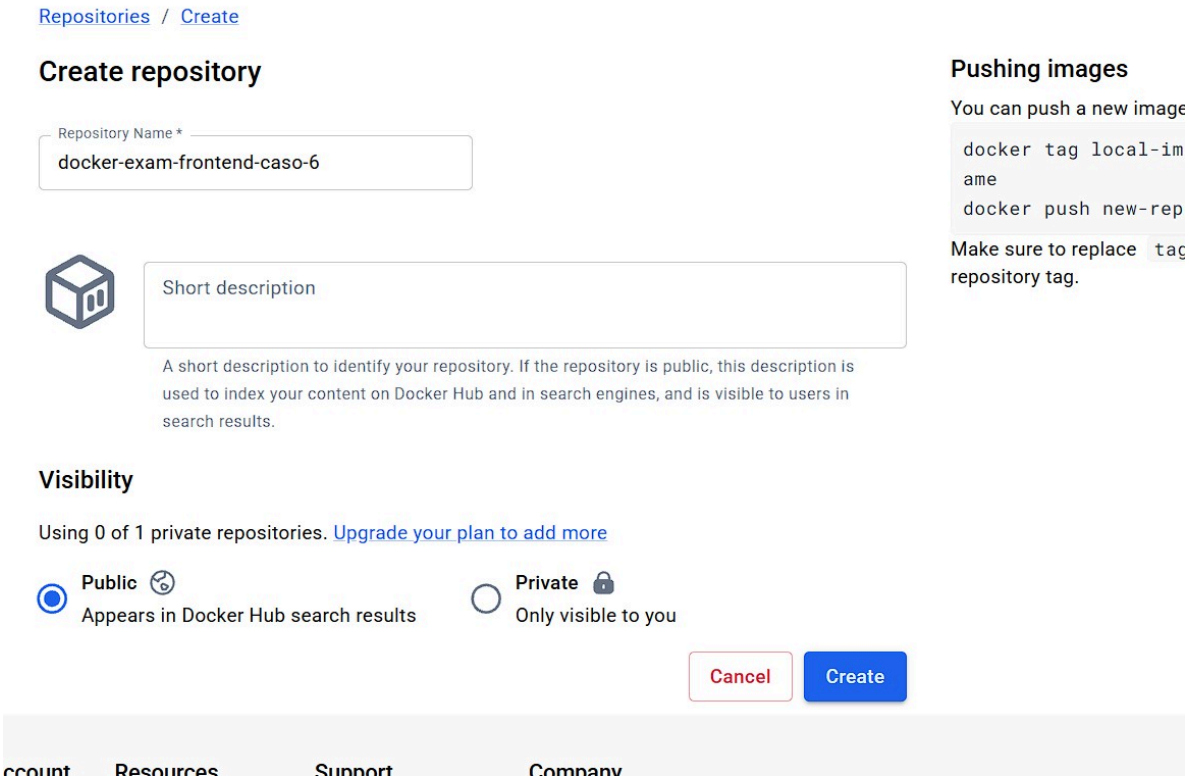
Ítem 6: Resuelto satisfactoriamente. Se completaron con éxito todas las actividades requeridas, utilizando **Docker Hub** y **Microsoft Azure Container Registry (ACR)** para la gestión, publicación y distribución de imágenes Docker, cumpliendo con los objetivos establecidos en la rúbrica de evaluación.

Evidencias adjuntas: 12. Las capturas y sus descripciones se presentan a continuación.

CASO 06 · EVIDENCIA 1052

Creación de repositorios

Cargada: 01 nov. 2026, 22:36 (Lima)



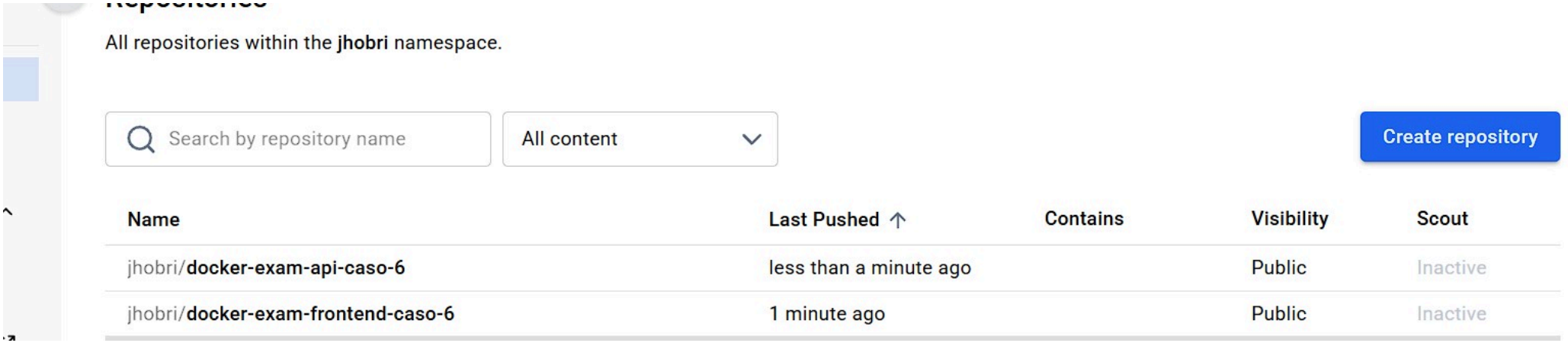
Descripción de la evidencia

Ejemplo de Creación de repositorios

CASO 06 · EVIDENCIA 1053

Repositorios Publicos Creados

Cargada: 01 nov. 2026, 22:37 (Lima)



Descripción de la evidencia

Repositorios creados front y backend

CASO 06 · EVIDENCIA 1054

Etiquetado y publicación de una imagen Docker personalizada en Docker Hub.

Cargada: 01 nov. 2026, 22:42 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker tag dockerexam-frontend:local-caso5 docker.io/jhobri/docker-exam-frontend-caso-6:1.0.0
```

```
docker push docker.io/jhobri/docker-exam-frontend-caso-6:1.0.0
```

Descripción:

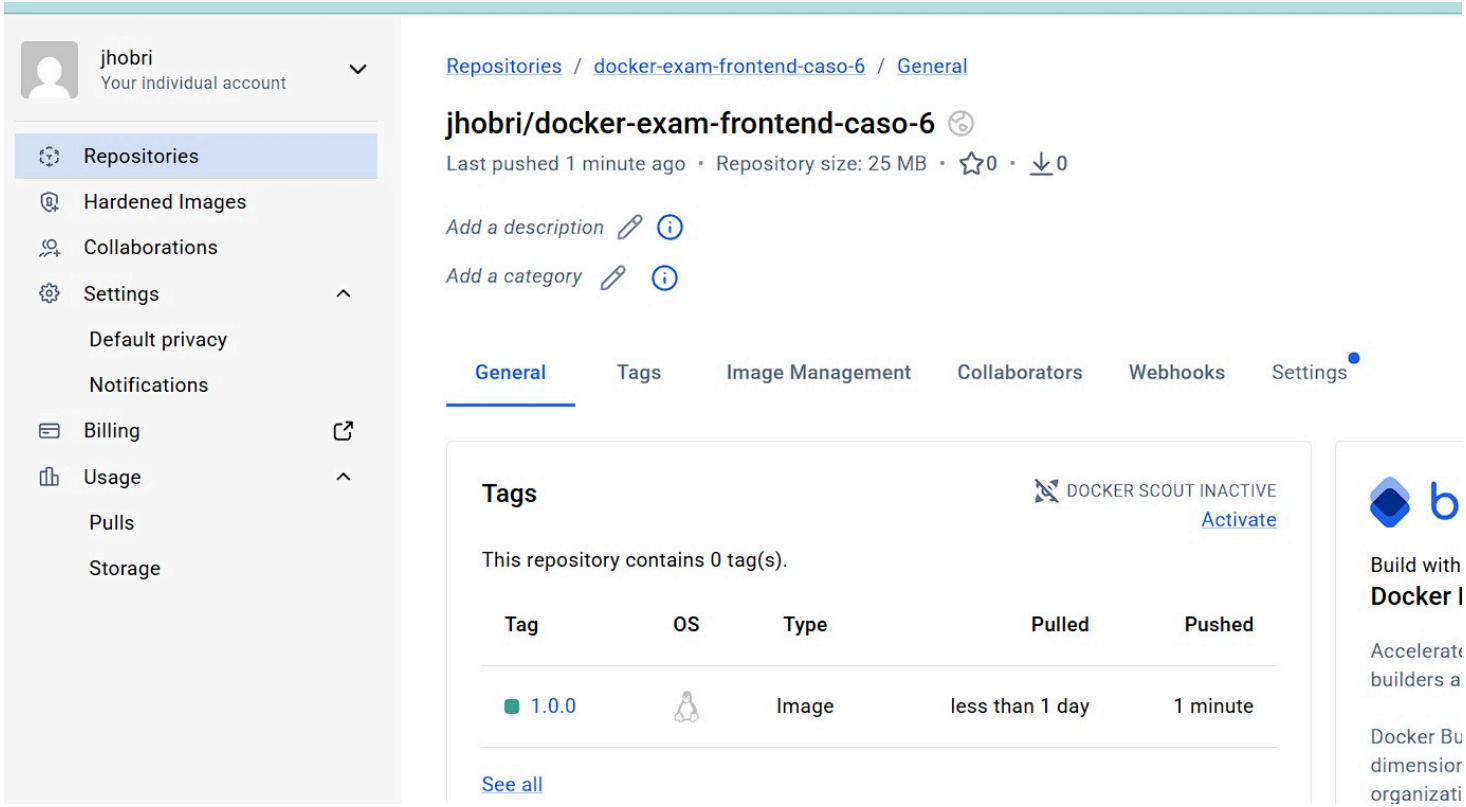
- tag: Comando que permite asignar una nueva etiqueta a una imagen Docker existente, sin reconstruirla ni duplicar sus capas.
- dockerexam-frontend:local-caso5: Nombre y etiqueta de la imagen local que se desea publicar.
- docker.io: Dirección del registro público Docker Hub, donde se almacenan y distribuyen imágenes Docker.
- jhobri: Nombre del usuario o espacio de repositorios de Docker Hub donde se publicará la imagen.
- docker-exam-frontend-caso-6: Nombre del repositorio que almacenará la imagen del Frontend.
- 1.0.0: Etiqueta que identifica la versión de la imagen publicada.
- push: Comando que permite subir una imagen Docker desde el equipo local hacia un registro remoto como Docker Hub.

Resultado: Se prepara la imagen del Frontend del caso práctico 06 con la etiqueta 1.0.0 y se ejecuta su publicación en Docker Hub, bajo el repositorio jhobri/docker-exam-frontend-caso-6. Si la operación finaliza correctamente, la imagen queda disponible para descargarse y utilizarse en otros entornos mediante docker pull, respetando los permisos de acceso del repositorio.

CASO 06 · EVIDENCIA 1055

Imagen Frontend Docker Hub

Cargada: 01 nov. 2026, 22:43 (Lima)



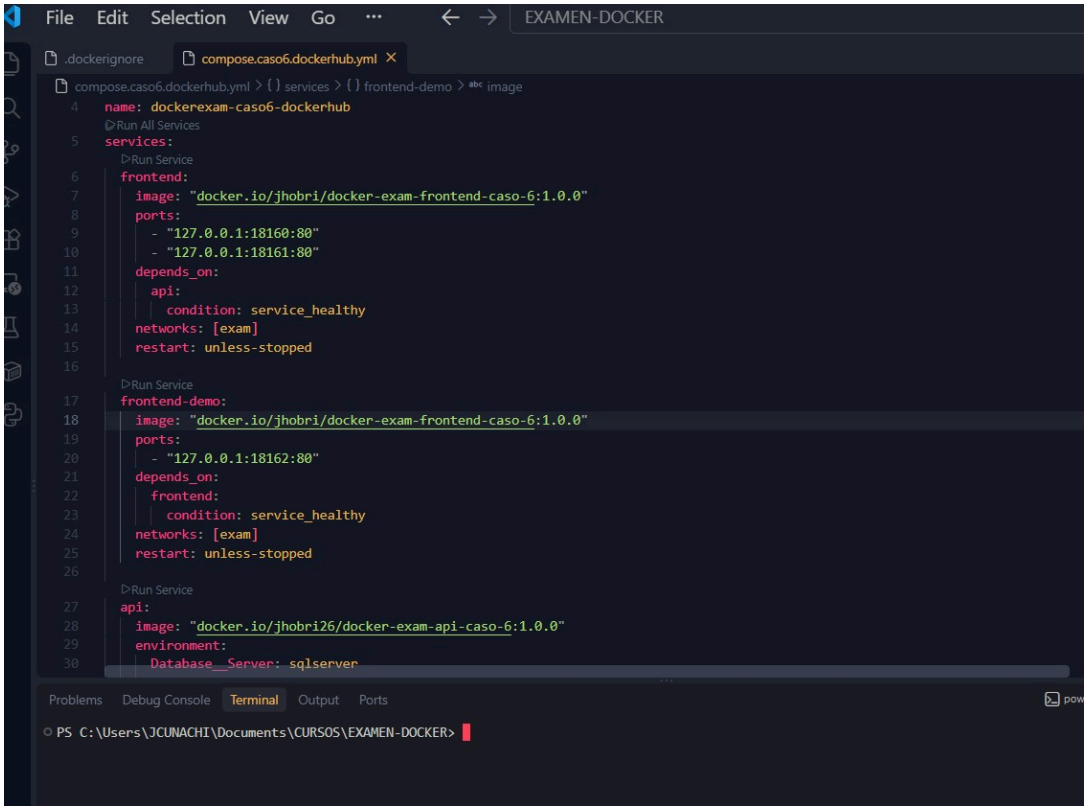
Descripción de la evidencia

Imagen subida al repositorio

CASO 06 · EVIDENCIA 1056

COMPOSE CONFIG DE IMAGENES PUBLICAS

Cargada: 01 nov. 2026, 22:53 (Lima)



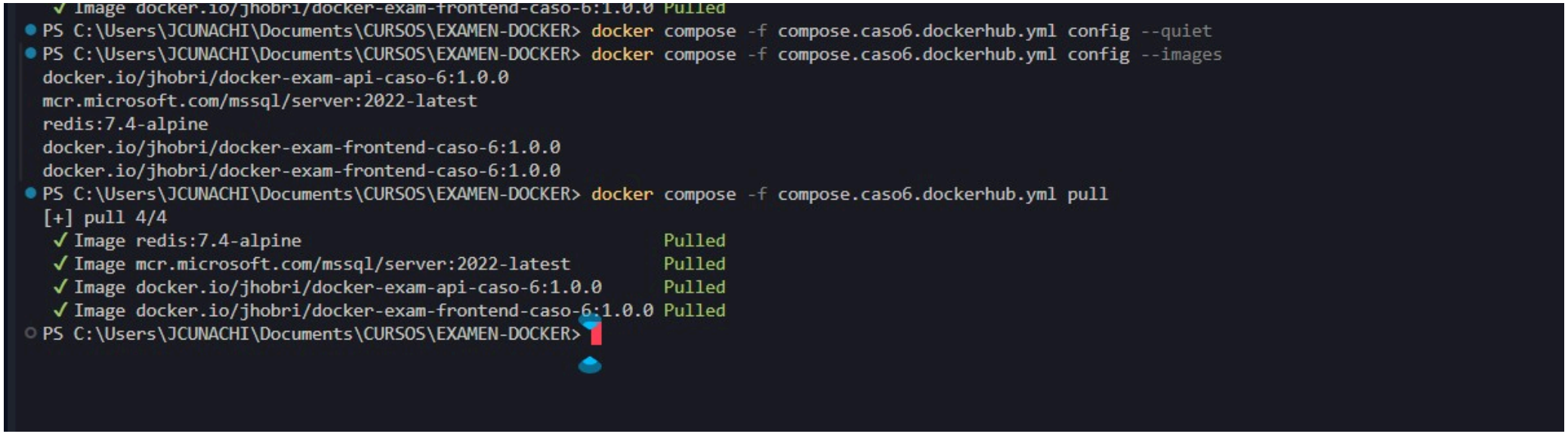
Descripción de la evidencia

SE ADJUNTA EVIDENCIA DE LA CONFIGURACION DE LAS IMAGENES A DESCARGAR CON COMPOSER

CASO 06 · EVIDENCIA 1057

Validación de configuración, verificación de imágenes y descarga de servicios Docker desde Docker Hub mediante Docker Compose.

Cargada: 01 nov. 2026, 22:59 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose -f compose.caso6.dockerhub.yml config --quiet
docker compose -f compose.caso6.dockerhub.yml config --images
docker compose -f compose.caso6.dockerhub.yml pull
```

Descripción:

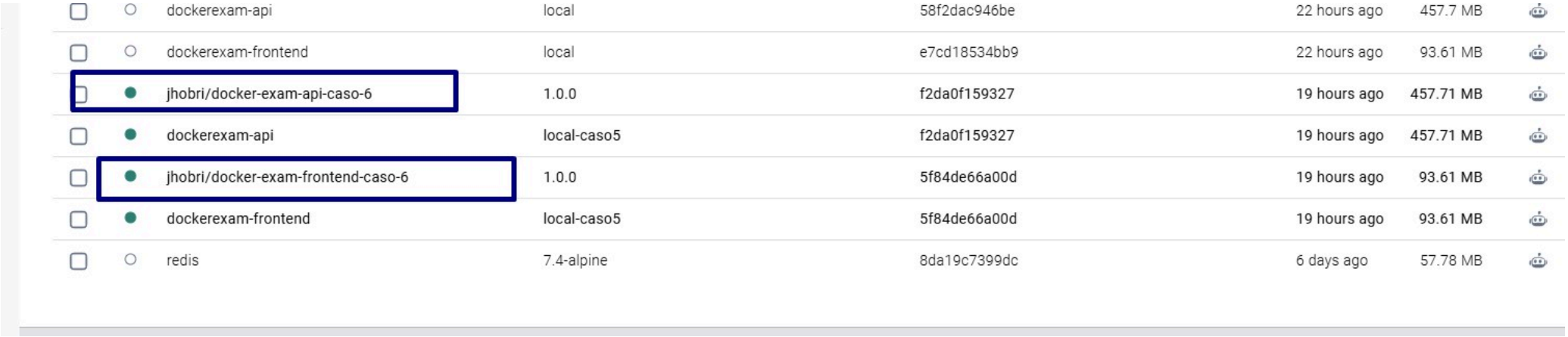
- `compose`: Permite administrar aplicaciones compuestas por múltiples servicios definidos en un archivo Docker Compose.
- `-f compose.caso6.dockerhub.yml`: Especifica el archivo YAML que contiene la configuración de los servicios del caso práctico 06.
- `config`: Permite procesar y validar la configuración del archivo Docker Compose.
- `--quiet`: Realiza la validación sin mostrar la configuración completa. Si no existen errores, finaliza sin generar salida.
- `--images`: Muestra los nombres y etiquetas de las imágenes Docker utilizadas por los servicios definidos en el archivo Compose.
- `pull`: Permite descargar desde Docker Hub u otros registros configurados las imágenes necesarias para ejecutar los servicios, sin crear ni iniciar contenedores.

Resultado: Se realiza la validación de la configuración del archivo `compose.caso6.dockerhub.yml`, se identifican las imágenes requeridas y se ejecuta su descarga desde los registros correspondientes. Esto permite preparar el entorno para el posterior despliegue de los servicios del caso práctico 06, utilizando imágenes previamente publicadas.

CASO 06 · EVIDENCIA 1058

Imagenes descargadas desde Docker Hub

Cargada: 01 nov. 2026, 23:01 (Lima)



dockerexam-api	local	58f2dac946be	22 hours ago	457.7 MB	
dockerexam-frontend	local	e7cd18534bb9	22 hours ago	93.61 MB	
jhobri/docker-exam-api-caso-6	1.0.0	f2da0f159327	19 hours ago	457.71 MB	
dockerexam-api	local-caso5	f2da0f159327	19 hours ago	457.71 MB	
jhobri/docker-exam-frontend-caso-6	1.0.0	5f84de66a00d	19 hours ago	93.61 MB	
dockerexam-frontend	local-caso5	5f84de66a00d	19 hours ago	93.61 MB	
redis	7.4-alpine	8da19c7399dc	6 days ago	57.78 MB	

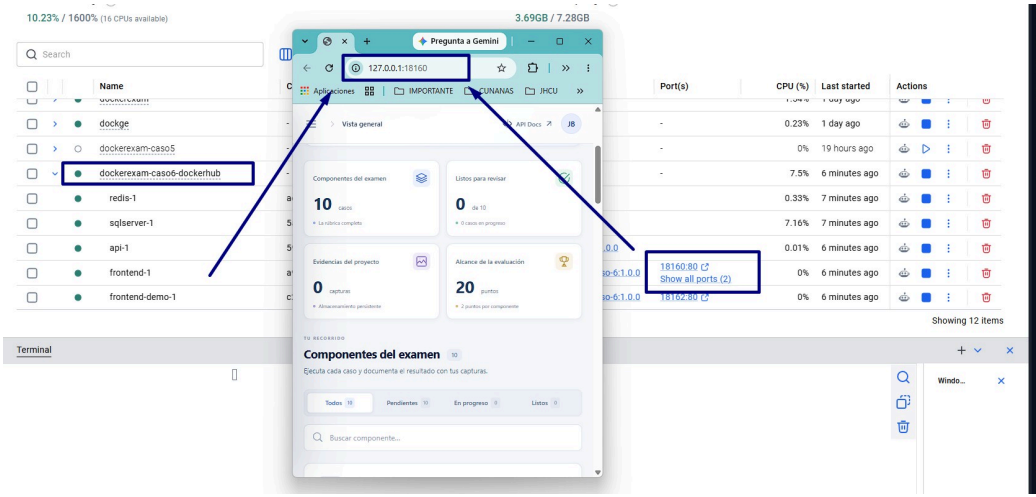
Descripción de la evidencia

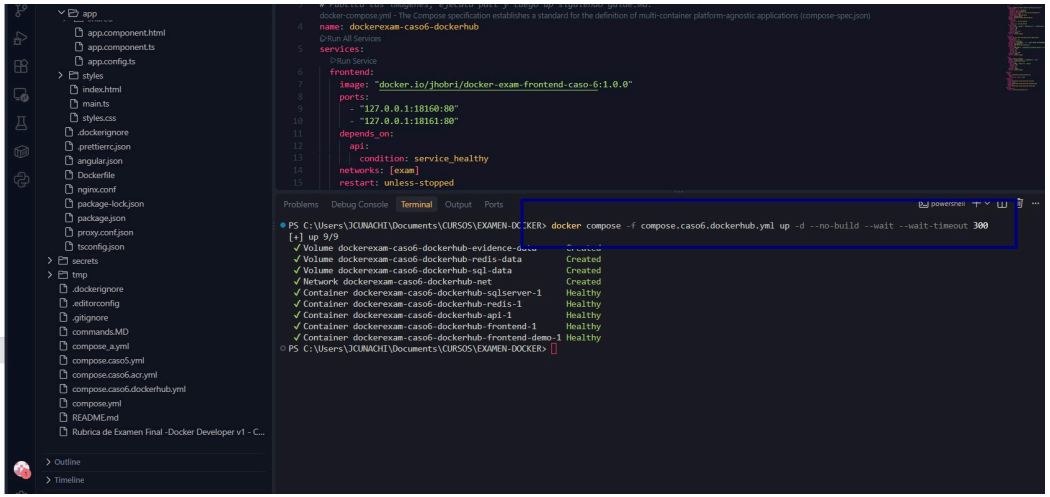
Evidencias de las imagenes descargadas desde Docker Hub

CASO 06 · EVIDENCIA 1059

Evidencia de Docker Hub

Cargada: 01 nov. 2026, 23:12 (Lima)





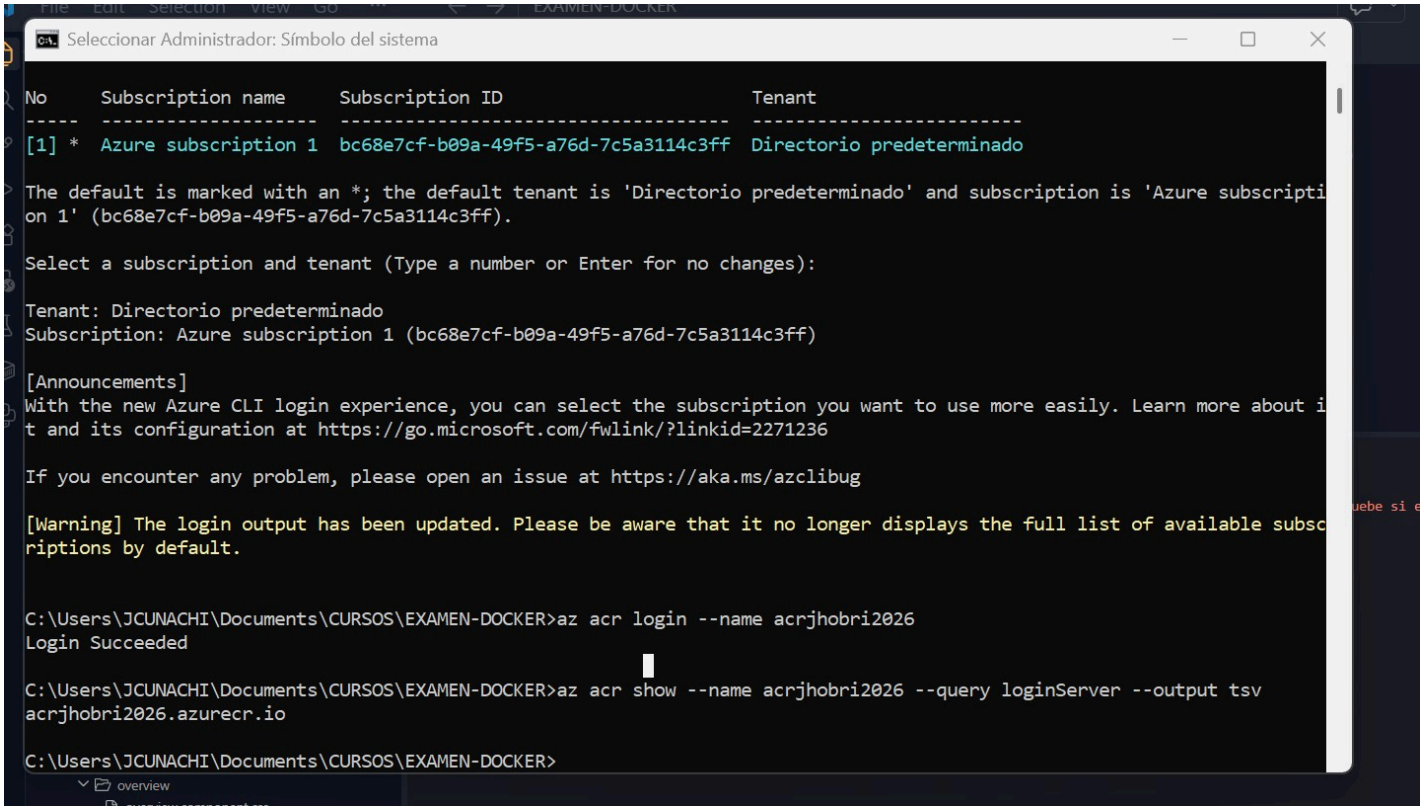
Descripción de la evidencia

Evidencia de descarga de imágenes, crear contenedores y verificar su funcionamiento.

CASO 06 · EVIDENCIA 1060

DOCKER AZURE CLI

Cargada: 02 nov. 2026, 00:20 (Lima)



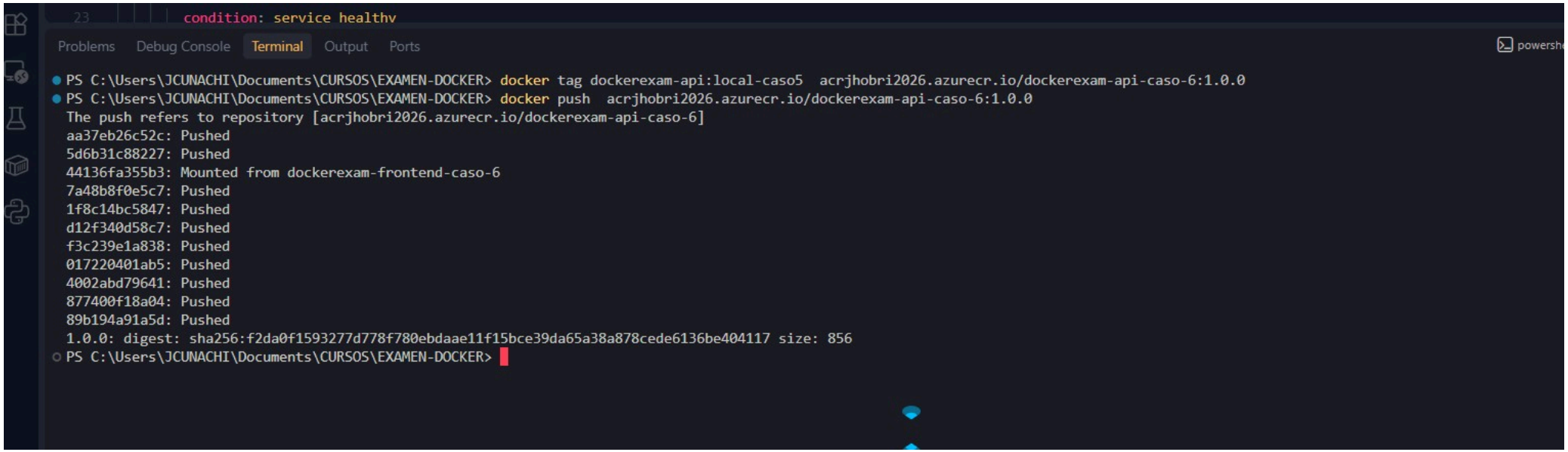
Descripción de la evidencia

LOGIN AZURE CLIENT

CASO 06 · EVIDENCIA 1061

Etiquetado y publicación de una imagen Docker de la API en Azure Container Registry (ACR).

Cargada: 02 nov. 2026, 00:31 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker tag dockerexam-api:local-caso5 acrjhobri2026.azurecr.io/dockerexam-api-caso-6:1.0.0
```

```
docker push acrjhobri2026.azurecr.io/dockerexam-api-caso-6:1.0.0
```

Descripción:

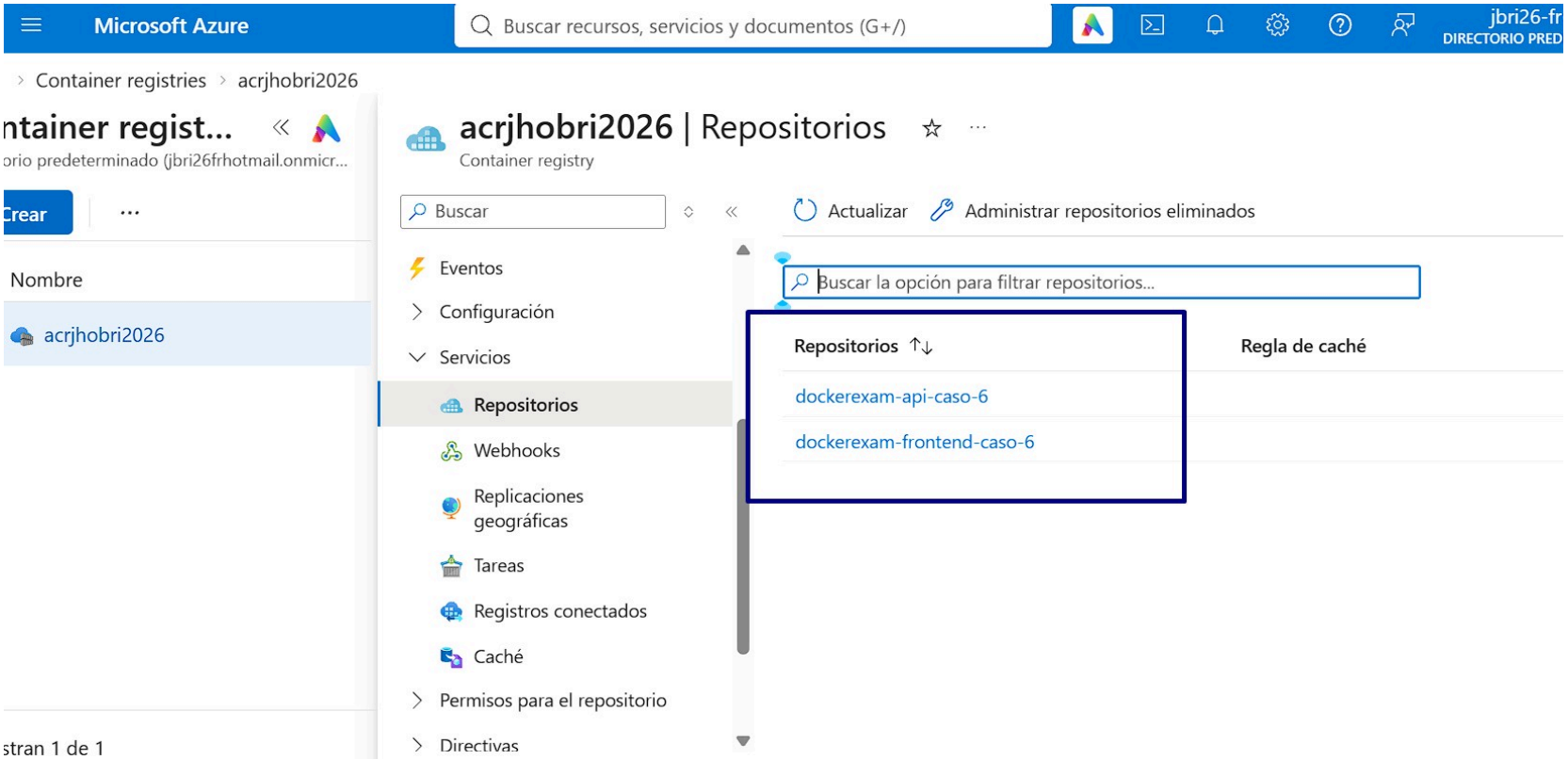
- `tag`: Comando que permite asignar una nueva etiqueta a una imagen Docker existente, sin reconstruirla ni duplicar sus capas.
- `dockerexam-api:local-caso5`: Nombre y etiqueta de la imagen local de la API que se desea publicar.
- `acrjhobri2026.azurecr.io`: Dirección del registro privado Azure Container Registry (ACR) donde se almacenará la imagen Docker.
- `dockerexam-api-caso-6`: Nombre del repositorio que almacenará la imagen de la API correspondiente al caso práctico 06.
- `1.0.0`: Etiqueta que identifica la versión de la imagen que será publicada.
- `push`: Comando que permite subir una imagen Docker desde el equipo local hacia el registro remoto Azure Container Registry.

Resultado: Se etiqueta la imagen local de la API con la versión `1.0.0` y se ejecuta su publicación en Azure Container Registry, bajo el repositorio `dockerexam-api-caso-6`. Si la operación finaliza correctamente, la imagen queda almacenada en el registro privado de Azure, disponible para su descarga y despliegue en otros entornos con la autenticación y los permisos correspondientes.

CASO 06 · EVIDENCIA 1062

IMAGENES CREADAS EN AZURE MICROSOFT

Cargada: 02 nov. 2026, 00:38 (Lima)



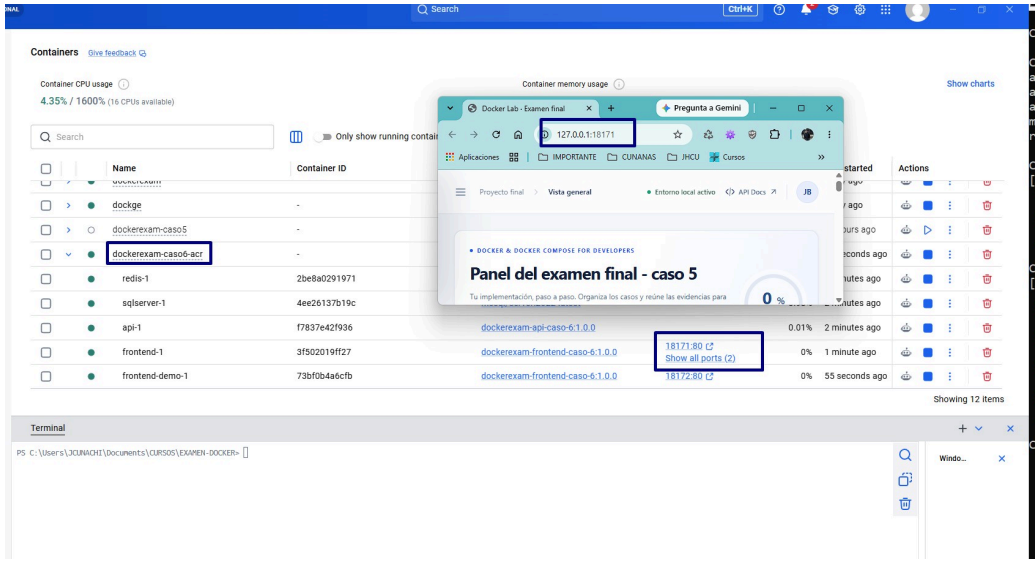
Descripción de la evidencia

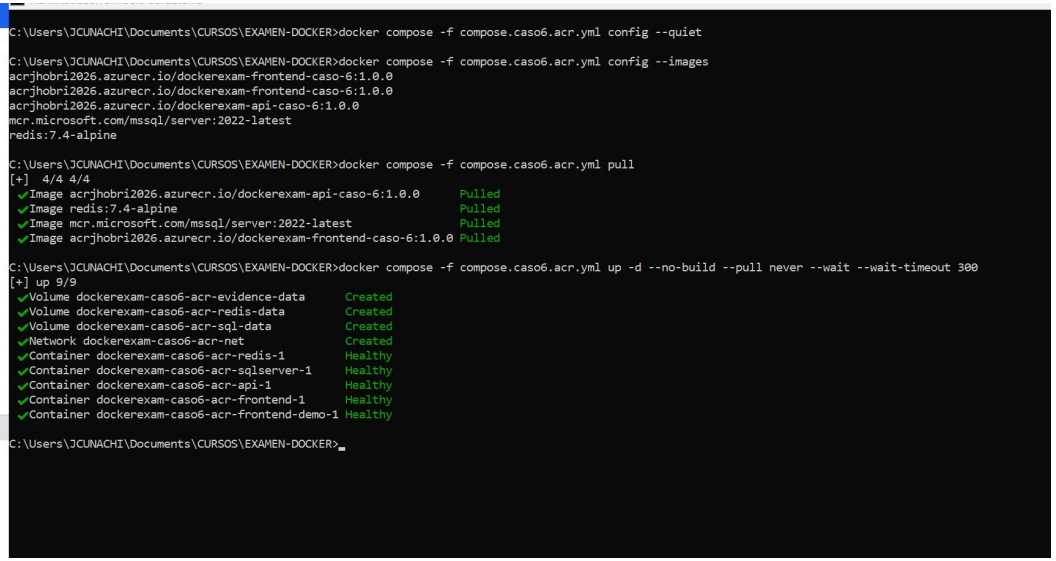
EVIDENCIA DE LAS IMAGENES SUBIDAD

CASO 06 · EVIDENCIA 1063

DOCKER AZURE MICROSFT

Cargada: 02 nov. 2026, 00:43 (Lima)





Descripción de la evidencia

Se deja evidencia de descarga de imágenes, crear contenedores y verificar su funcionamiento

CASO 07

Docker Compose

Listo para revisión · Avance guardado: 02 nov. 2026, 01:06

Lo solicitado

Crear un stack de servicios con al menos cuatro imágenes distintas propias y de terceros, cinco contenedores y uno expuesto en diferentes puertos.

Desarrollo y resultados

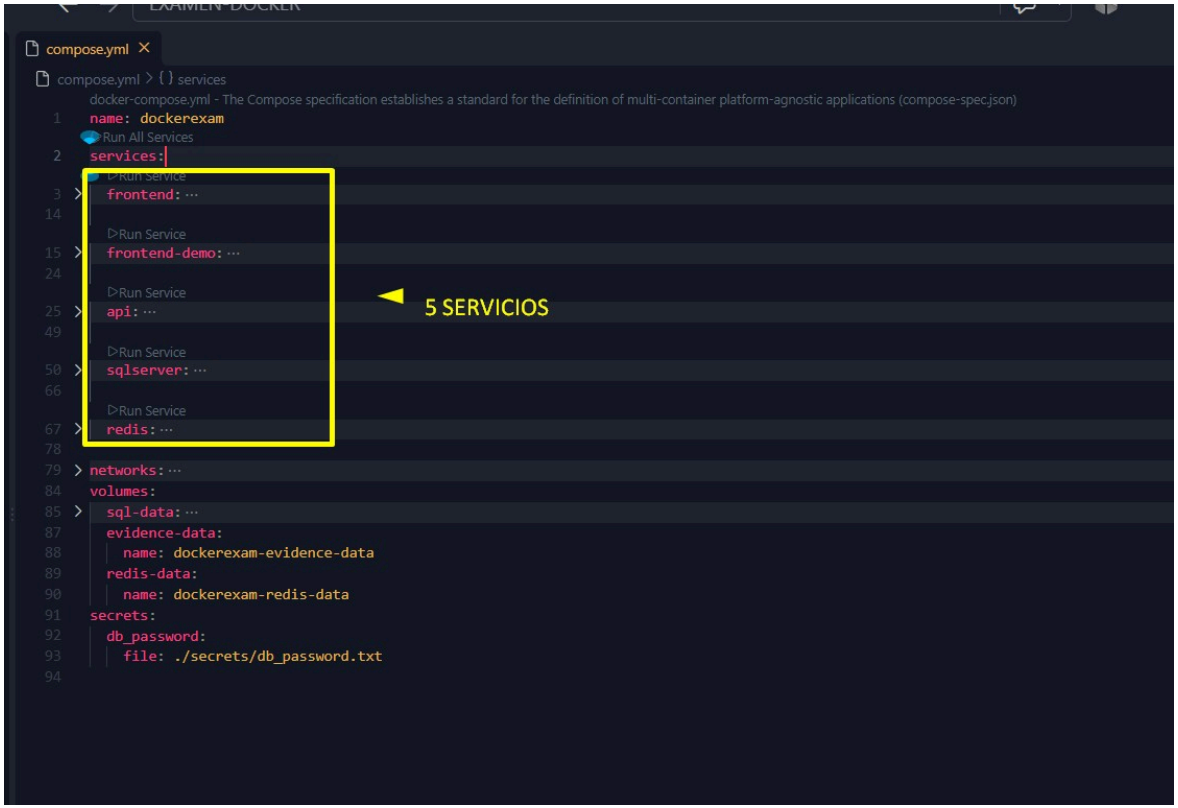
Ítem 7: Resuelto satisfactoriamente. s E CREARON SEGUN EL REQUERIMIENTO DE LA RUBRICA

Evidencias adjuntas: 2. Las capturas y sus descripciones se presentan a continuación.

CASO 07 · EVIDENCIA 1064

Configuración de arquitectura multicontenedor mediante Docker Compose, integrando Frontend, API, SQL Server y Redis.

Cargada: 02 nov. 2026, 00:59 (Lima)



Descripción de la evidencia

Archivo de configuración: `compose.yaml`

Descripción:

- `name: dockerexam`: Define el nombre del proyecto Docker Compose, utilizado para identificar y agrupar sus recursos.
- `services`: Permite definir los servicios que conforman la aplicación multicontenedor.
- `frontend`: Configura la aplicación web utilizando la imagen `dockerexam-frontend:local`, construida desde `./frontend`, y publica los puertos locales `18080` y `18081`.
- `frontend-demo`: Define una segunda instancia del Frontend, utilizando la misma imagen y publicando el puerto `18082`.
- `api`: Configura el servicio Backend utilizando la imagen `dockerexam-api:local`, construida desde `./backend`, con conexión a SQL Server y Redis.
- `sqlserver`: Implementa Microsoft SQL Server 2022 Developer como motor de base de datos, utilizando almacenamiento persistente y credenciales mediante Docker Secrets.
- `redis`: Configura Redis 7.4 Alpine como servicio de almacenamiento en memoria, habilitando persistencia mediante AOF.
- `image`: Especifica la imagen Docker que utilizará cada servicio.
- `build`: Define el directorio utilizado para construir las imágenes personalizadas.
- `ports`: Configura la publicación de puertos entre el equipo anfitrión y los contenedores.
- `environment`: Define las variables de entorno necesarias para configurar los servicios y sus conexiones.
- `depends_on`: Establece dependencias de inicio entre servicios, permitiendo esperar a que los servicios requeridos estén saludables.
- `healthcheck`: Configura verificaciones periódicas para comprobar el estado de funcionamiento de los servicios.
- `networks`: Define la red personalizada `dockerexam-net`, que permite la comunicación interna entre contenedores.
- `volumes`: Configura los volúmenes persistentes `dockerexam-sql-data`, `dockerexam-evidence-data` y `dockerexam-redis-data`.
- `secrets`: Permite proporcionar la contraseña de la base de datos mediante el archivo externo `./secrets/db_password.txt`, evitando declararla directamente en el archivo YAML.
- `restart: unless-stopped`: Configura el reinicio automático de los contenedores, excepto cuando hayan sido detenidos manualmente.

Resultado: Se define una arquitectura de cinco servicios Docker integrados mediante una red privada, con almacenamiento persistente, gestión de credenciales, dependencias de inicio y comprobaciones de salud para API, SQL Server y Redis.

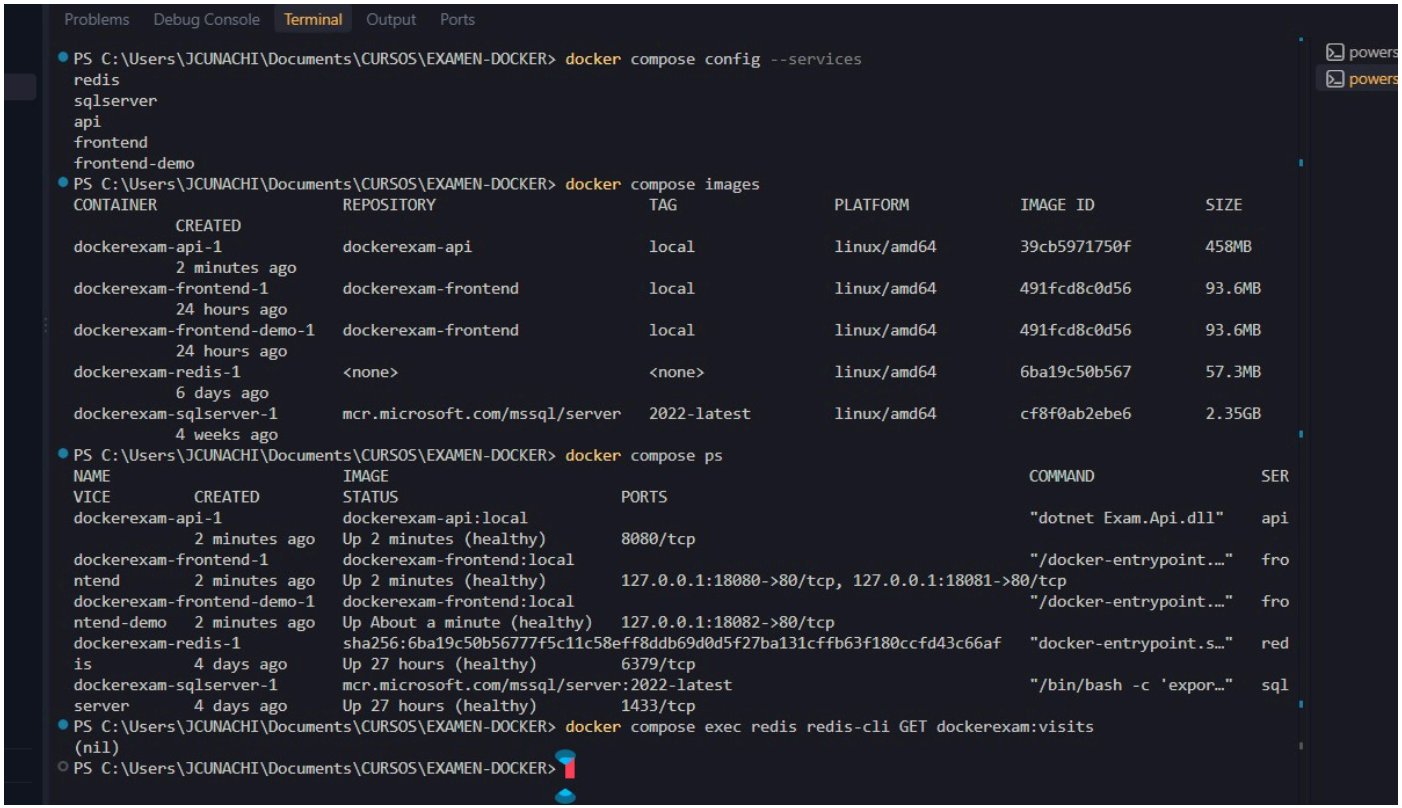
La configuración permite administrar el despliegue de la aplicación desde un único archivo Docker Compose, facilitando su ejecución, mantenimiento y administración.

Observación: El servicio `frontend-demo` depende de que `frontend` alcance el estado saludable, pero `frontend` no tiene un `healthcheck` definido explícitamente en este archivo. Para garantizar el funcionamiento de esa dependencia, debe existir una comprobación de salud incorporada en la imagen o agregarse una al servicio.

CASO 07 · EVIDENCIA 1065

Verificación de servicios, imágenes, estado de contenedores y consulta de datos en Redis mediante Docker Compose.

Cargada: 02 nov. 2026, 01:05 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose config --services
docker compose images
docker compose ps
docker compose exec redis redis-cli GET dockerexam:visits
```

Descripción:

- `compose`: Permite administrar aplicaciones compuestas por múltiples servicios Docker.
- `config --services`: Muestra los nombres de los servicios definidos en el archivo Docker Compose.
- `images`: Permite listar las imágenes utilizadas por los contenedores del proyecto, mostrando sus versiones, identificadores y tamaños.
- `ps`: Muestra los contenedores del proyecto, incluyendo su estado de ejecución, servicios y puertos publicados.
- `exec`: Permite ejecutar comandos dentro de un contenedor que se encuentra en funcionamiento.
- `redis`: Nombre del servicio Redis donde se ejecutará el comando.
- `redis-cli`: Herramienta de línea de comandos que permite interactuar con Redis.
- `GET`: Comando que permite consultar el valor almacenado en una clave de Redis.
- `dockerexam:visits`: Nombre de la clave que se desea consultar, utilizada para almacenar información relacionada con las visitas de la aplicación.

Resultado: Se verificó la configuración de los cinco servicios (`frontend`, `frontend-demo`, `api`, `sqlserver` y `redis`), confirmando que sus contenedores se encuentran en ejecución y con estado Healthy (5/5). Asimismo, se consultó la clave `dockerexam:visits` en Redis, obteniendo `(nil)`, lo que indica que la clave no contiene un valor disponible en ese momento.

CASO 08

Docker Compose con Build

Listo para revisión · Avance guardado: 02 nov. 2026, 02:22

Lo solicitado

Crear un stack con Dockerfile y context en Compose, modificar código de una aplicación o API y regenerar contenedores verificando el cambio.

Desarrollo y resultados

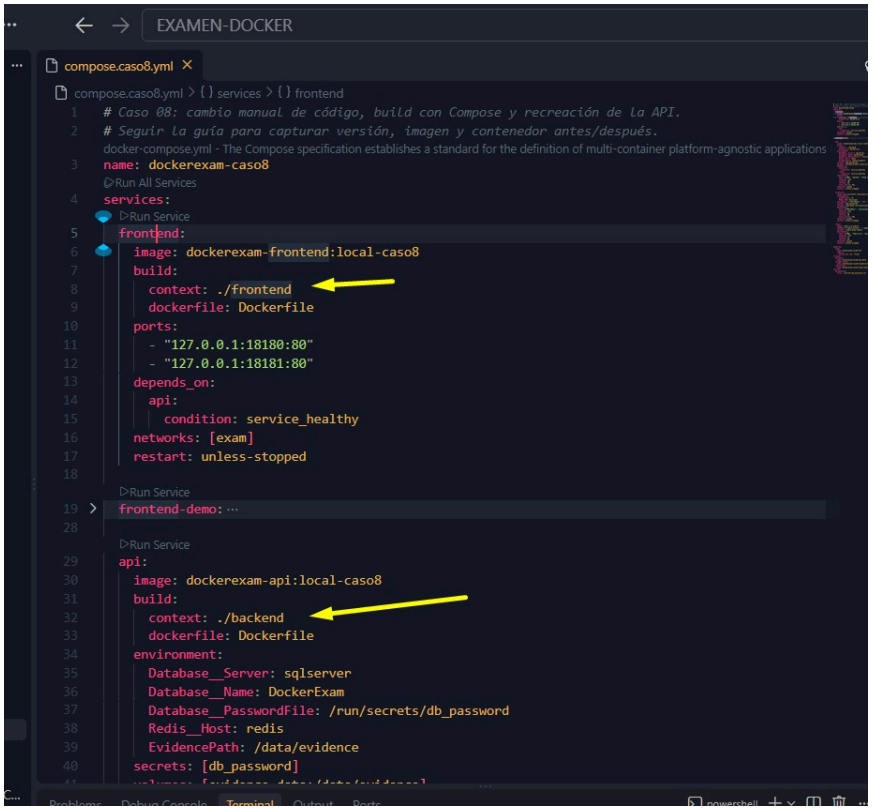
SE CUMPLIO CON EL CASO 8 SATISFACTORIAMENTE

Evidencias adjuntas: 3. Las capturas y sus descripciones se presentan a continuación.

CASO 08 · EVIDENCIA 1066

Uso de DOCKER FILE Y CONTECT

Cargada: 02 nov. 2026, 02:15 (Lima)



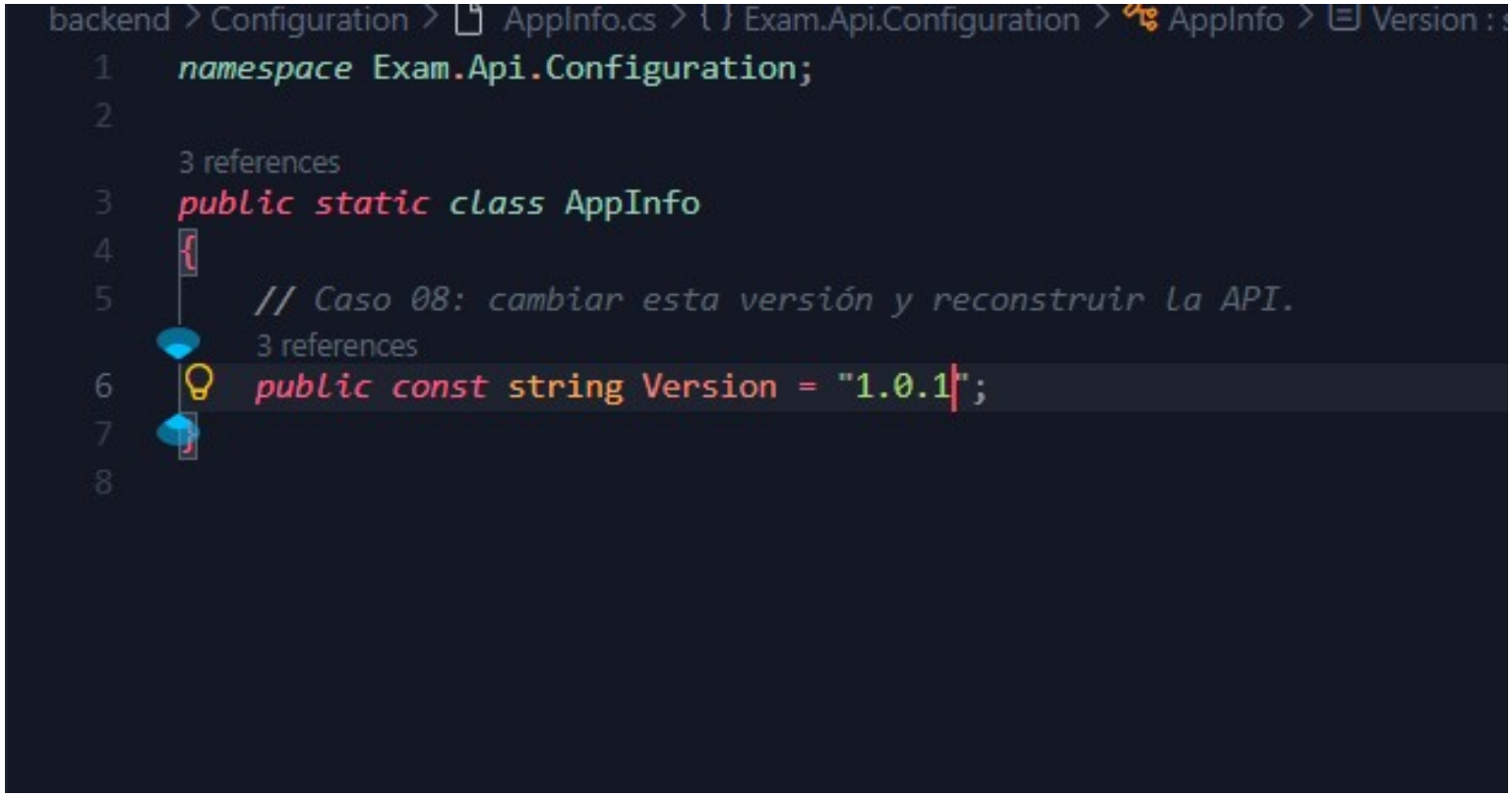
Descripción de la evidencia

stack de servicios relacionados, considerando la creación de imágenes en caliente (uso de Dockerfile y context).

CASO 08 · EVIDENCIA 1067

CAMBIO DE VERSION CODIGO C#

Cargada: 02 nov. 2026, 02:17 (Lima)



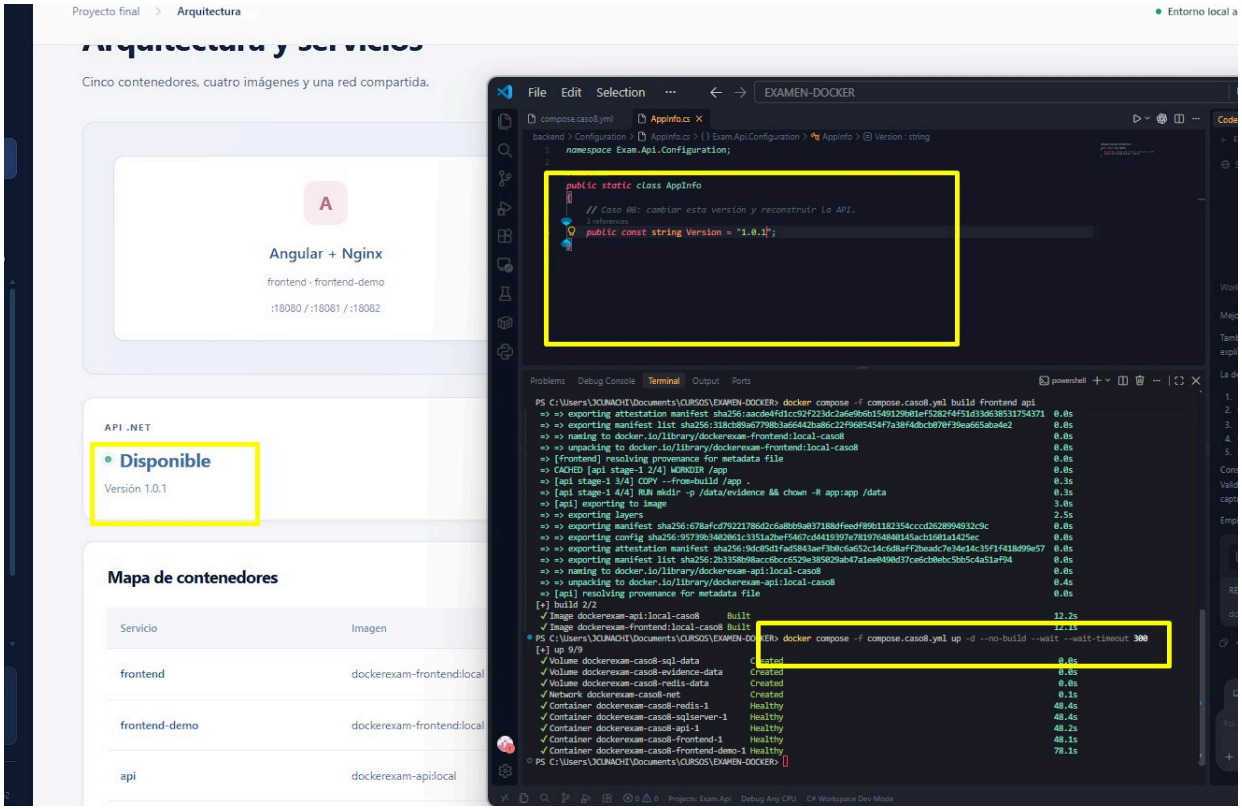
Descripción de la evidencia

SE CAMBIA LA VERSIÓN DEL API DE V1.0.0 A V1.0.1

CASO 08 · EVIDENCIA 1068

VALIDACION DE LOS CAMBIOS

Cargada: 02 nov. 2026, 02:22 (Lima)



Descripción de la evidencia

SE VERIFICA LA ACTUALIZACION DE LOS CAMBIOS REALZIADOS EN EL CONTENEDOR

CASO 09

Secrets y volúmenes

Listo para revisión · Avance guardado: 02 nov. 2026, 15:17

Lo solicitado

Crear y utilizar secrets con Compose o contenedores, usar volúmenes y demostrar persistencia después de eliminar contenedores y crear otros que reutilicen los volúmenes.

Desarrollo y resultados

ÍTEM COMPLETADO SATISFACTORIAMENTE. Se cumplió con todos los requerimientos establecidos, incluyendo la creación y utilización de **Docker Secrets** para la gestión segura de información sensible, así como la creación, configuración y uso de **volúmenes Docker** para el almacenamiento persistente de datos.

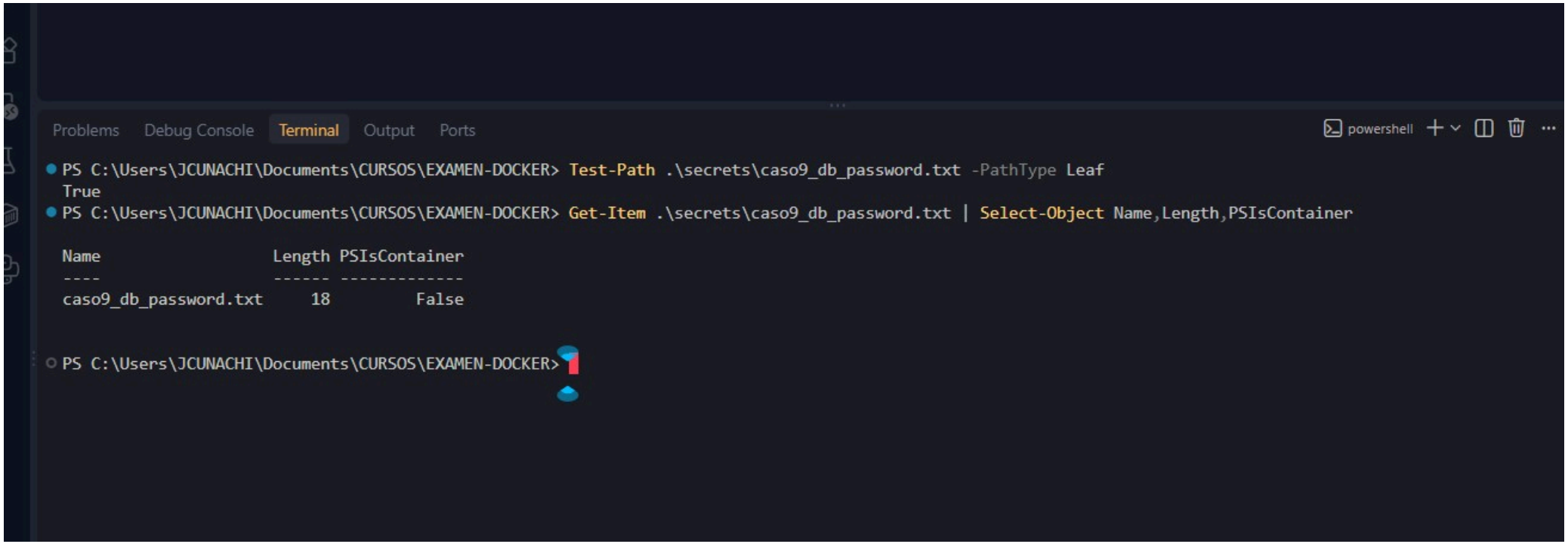
Asimismo, se demostró satisfactoriamente la **persistencia de la información después de eliminar los contenedores originales**, mediante la creación de nuevos contenedores y la reutilización de los volúmenes existentes, verificando que los datos permanecen íntegros y disponibles.

Evidencias adjuntas: 6. Las capturas y sus descripciones se presentan a continuación.

CASO 09 · EVIDENCIA 1069

Verificación de existencia y propiedades del secreto mediante PowerShell.

Cargada: 02 nov. 2026, 08:19 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
Test-Path .\secrets\caso9_db_password.txt -PathType Leaf
```

```
Get-Item .\secrets\caso9_db_password.txt | Select-Object Name,Length,PSIsContainer
```

Descripción:

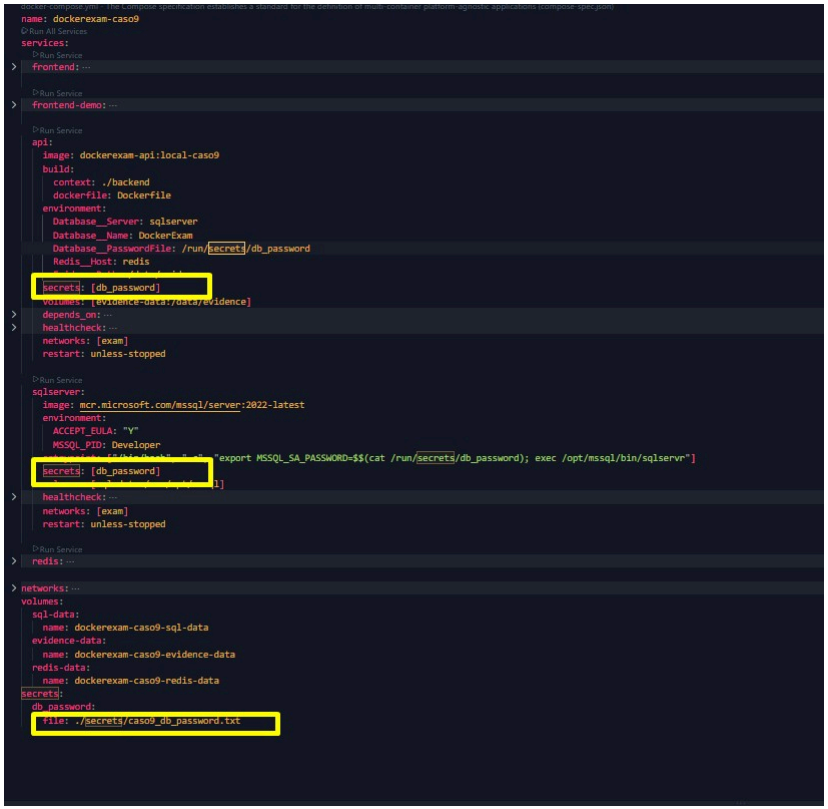
- `Test-Path`: Permite verificar si existe un archivo o directorio en la ruta especificada.
- `.\secrets\caso9_db_password.txt`: Ruta relativa del archivo que contiene la contraseña de la base de datos para el caso práctico 09.
- `-PathType Leaf`: Verifica que la ruta corresponda a un archivo y no a un directorio.
- `Get-Item`: Permite consultar las propiedades de un archivo existente.
- `Select-Object`: Selecciona las propiedades específicas que se desean visualizar.
- `Name`: Muestra el nombre del archivo.
- `Length`: Indica el tamaño del archivo en bytes.
- `PSIsContainer`: Indica si el elemento es un directorio (`True`) o un archivo (`False`).

Resultado: Se confirmó que el archivo `caso9_db_password.txt` existe mediante el resultado `True`. Asimismo, se verificó que tiene un tamaño de 18 bytes y que corresponde a un archivo (`PSIsContainer: False`), sin necesidad de exponer su contenido confidencial.

CASO 09 · EVIDENCIA 1070

Configuración y uso de Docker Secrets para la gestión segura de credenciales de SQL Server y API.

Cargada: 02 nov. 2026, 08:26 (Lima)



Descripción de la evidencia

Archivo de configuración: `compose.caso9.yml`

Descripción:

- `secrets`: Permite definir y administrar información sensible, como contraseñas y credenciales, evitando incluirlas directamente en las variables de entorno del archivo Docker

Compose.

- `db_password`: Nombre del secreto utilizado para almacenar la contraseña de conexión a SQL Server.
- `file: ./secrets/caso9_db_password.txt`: Especifica el archivo local que contiene la contraseña utilizada por los servicios.
- `secrets: [db_password]`: Autoriza a los contenedores `api` y `sqlserver` a acceder al secreto configurado.
- `/run/secrets/db_password`: Ruta interna donde Docker Compose monta el archivo del secreto dentro de los contenedores autorizados.
- `Database__PasswordFile`: Variable que indica a la API la ubicación del archivo desde el cual debe leer la contraseña.
- `MSSQL_SA_PASSWORD`: Variable utilizada por SQL Server para configurar la contraseña del administrador, obteniéndola del archivo de secreto durante el inicio del servicio.

Resultado: Se configuró el uso de Docker Secrets para proporcionar las credenciales de la base de datos a los contenedores `api` y `sqlserver`, evitando escribir la contraseña directamente en el archivo YAML. Esta implementación mejora la organización y protección de las credenciales, permitiendo que ambos servicios accedan al secreto mediante un archivo montado en `/run/secrets/db_password`.

Nota: En Docker Compose local, los secretos basados en archivos se montan dentro de los contenedores; no constituyen por sí solos un almacén cifrado de secretos. Por seguridad, el archivo de origen debe mantenerse protegido y excluido del repositorio Git.

CASO 09 · EVIDENCIA 1071

LEVANTAMIENTO DEL CASO 9

Cargada: 02 nov. 2026, 08:49 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose -f compose.caso9.yml up -d --no-build --wait --wait-timeout 300
```

```
docker compose -f compose.caso9.yml ps
```

Descripción:

- `compose`: Permite administrar aplicaciones compuestas por múltiples servicios Docker.
- `-f compose.caso9.yml`: Especifica el archivo de configuración correspondiente al caso práctico 09.
- `up`: Permite crear e iniciar los contenedores definidos en Docker Compose.
- `-d`: Ejecuta los contenedores en segundo plano.
- `--no-build`: Utiliza las imágenes existentes sin reconstruirlas.
- `--wait`: Espera hasta que los servicios alcancen el estado requerido de ejecución o salud.
- `--wait-timeout 300`: Establece un tiempo máximo de espera de 300 segundos.
- `ps`: Permite consultar el estado de ejecución y salud de los contenedores.

Resultado: Se inició el despliegue de los servicios del caso práctico 09. Redis alcanzó el estado `Healthy`, mientras que SQL Server permaneció en estado `Unhealthy`, provocando el mensaje `timeout waiting for dependencies`. Debido a las dependencias configuradas, los servicios API y Frontend no completaron su inicio.

Observación: Se requiere revisar el `healthcheck`, los registros y la configuración de credenciales de SQL Server para identificar la causa del problema.

Comandos recomendados para el diagnóstico:

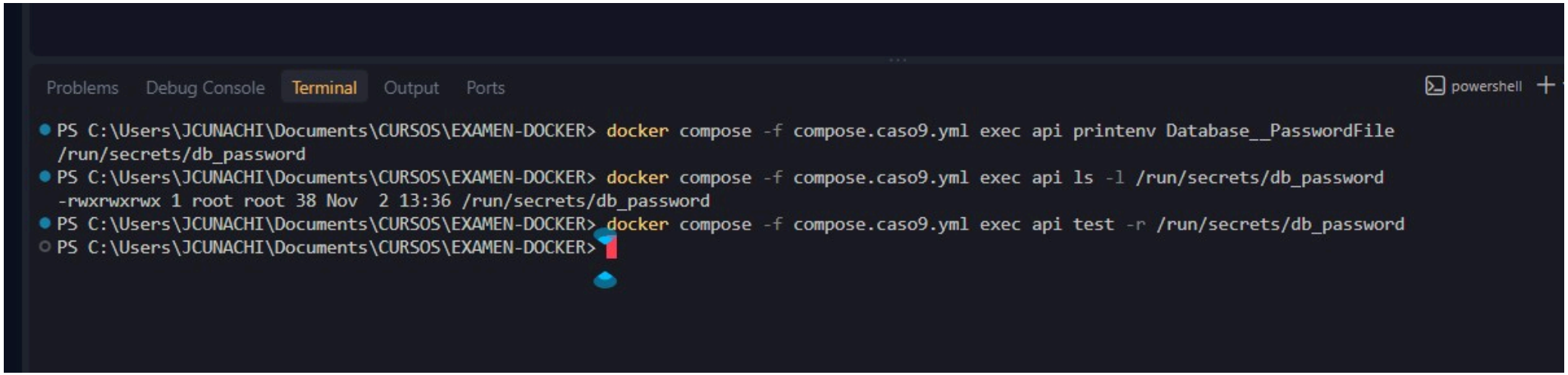
```
docker logs --tail 100 dockerexam-caso9-sqlserver-1
docker inspect dockerexam-caso9-sqlserver-1 --format '{{json .State.Health}}'
```

Esto permitirá determinar si el problema está relacionado con el inicio de SQL Server, la autenticación o la verificación de salud configurada.

CASO 09 · EVIDENCIA 1072

Verificación de configuración, acceso y permisos de Docker Secrets en el contenedor API.

Cargada: 02 nov. 2026, 12:03 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose -f compose.caso9.yml exec api printenv Database__PasswordFile
```

```
docker compose -f compose.caso9.yml exec api ls -l /run/secrets/db_password

docker compose -f compose.caso9.yml exec api test -r /run/secrets/db_password
```

Descripción:

- `compose -f compose.caso9.yml`: Permite administrar los servicios definidos en el archivo Docker Compose del caso práctico 09.
- `exec api`: Ejecuta comandos dentro del contenedor correspondiente al servicio API.
- `printenv Database__PasswordFile`: Consulta la variable de entorno que contiene la ruta del archivo de contraseña.
- `ls -l`: Muestra la existencia, propietario, tamaño y permisos del archivo secreto.
- `/run/secrets/db_password`: Ruta interna donde se encuentra montado el archivo de credenciales.
- `test -r`: Verifica si el archivo de contraseña tiene permisos de lectura para el usuario que ejecuta el comando.

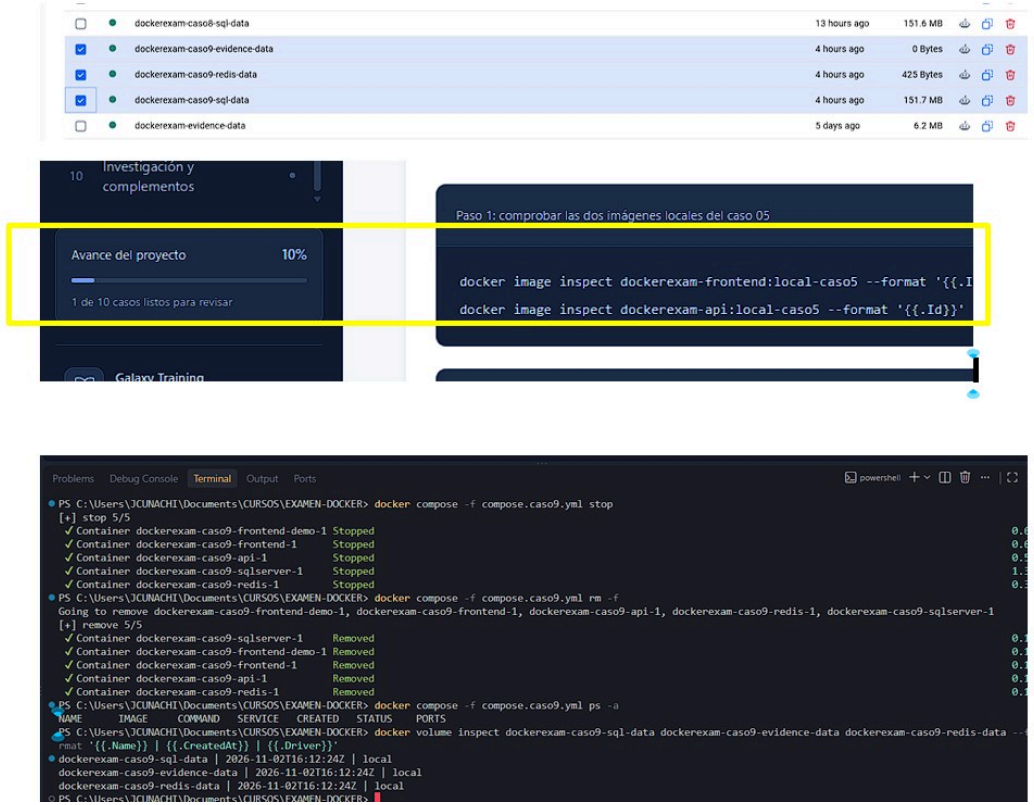
Resultado: Se confirmó que la API tiene configurada la ruta `/run/secrets/db_password` y que el archivo secreto existe dentro del contenedor, con un tamaño de 38 bytes y permisos de lectura. La ejecución de `test -r` finalizó sin errores, lo que indica que el usuario del contenedor puede leer el archivo.

Estas verificaciones permiten comprobar que Docker Compose ha montado correctamente el secreto y que la API tiene acceso al archivo de credenciales, sin exponer su contenido.

CASO 09 · EVIDENCIA 1073

Detención y eliminación de contenedores Docker, verificando la conservación de volúmenes persistentes.

Cargada: 02 nov. 2026, 15:03 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose -f compose.caso9.yml stop
docker compose -f compose.caso9.yml rm -f
docker compose -f compose.caso9.yml ps -a
docker volume inspect dockerexam-caso9-sql-data dockerexam-caso9-evidence-data dockerexam-caso9-redis-data --format '{{.Name}} | {{.CreatedAt}} | {{.Driver}}'
```

Descripción:

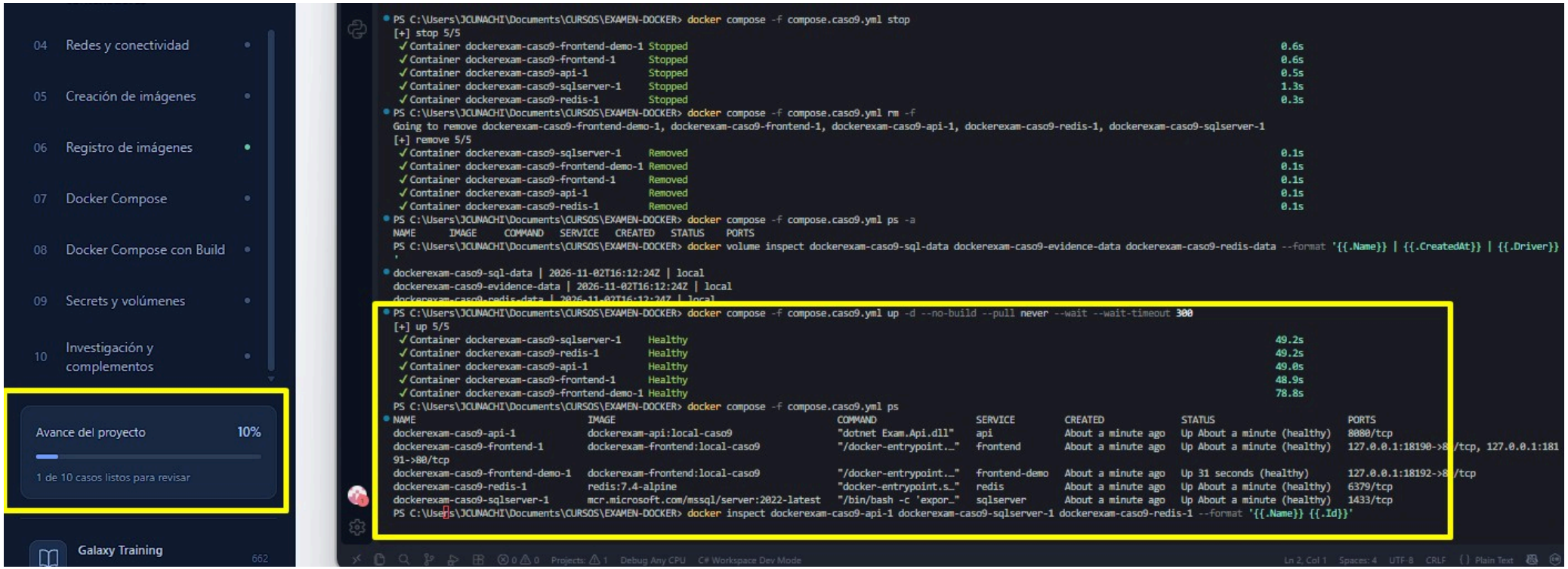
- `compose -f compose.caso9.yml`: Permite administrar los servicios definidos en el archivo Docker Compose del caso práctico 09.
- `stop`: Detiene los contenedores de manera controlada sin eliminarlos.
- `rm -f`: Elimina los contenedores detenidos sin solicitar confirmación, conservando los volúmenes nombrados.
- `ps -a`: Permite listar los contenedores del proyecto, incluidos los detenidos, para verificar su eliminación.
- `volume inspect`: Consulta información detallada de los volúmenes Docker existentes.
- `dockerexam-caso9-sql-data`: Volumen destinado al almacenamiento persistente de SQL Server.
- `dockerexam-caso9-evidence-data`: Volumen destinado al almacenamiento de evidencias de la aplicación.
- `dockerexam-caso9-redis-data`: Volumen utilizado para conservar los datos persistentes de Redis.
- `--format`: Permite seleccionar la información que se mostrará de cada volumen.
- `{{.Name}} | {{.CreatedAt}} | {{.Driver}}`: Muestra el nombre, fecha de creación y controlador de almacenamiento de cada volumen.

Resultado: Se detuvieron y eliminaron satisfactoriamente los cinco contenedores del caso práctico 09. Posteriormente, se verificó que los volúmenes de SQL Server, Redis y evidencias permanecen disponibles, demostrando que los datos persistentes pueden conservarse independientemente de la eliminación de los contenedores, siempre que sus volúmenes no sean eliminados.

CASO 09 · EVIDENCIA 1074

Demostración de persistencia de datos mediante volúmenes Docker.

Cargada: 02 nov. 2026, 15:08 (Lima)



Descripción de la evidencia

Descripción:
Se realizó la demostración de persistencia deteniendo y eliminando los contenedores del caso 09 mediante `docker compose stop` y `docker compose rm -f`, verificando previamente que los volúmenes `dockerexam-caso9-sql-data`, `dockerexam-caso9-evidence-data` y `dockerexam-caso9-redis-data` permanecían creados y disponibles en Docker. Posteriormente, se volvió a desplegar la solución con `docker compose -f compose.caso9.yml up -d --no-build --pull never --wait --wait-timeout 300`, confirmando que los servicios `sqlserver`, `redis`, `api`, `frontend`

y `frontend-demo` iniciaron correctamente y alcanzaron el estado **Healthy**. Esto demuestra que, aunque los contenedores fueron eliminados, la información persistente se conservó en los volúmenes y fue reutilizada automáticamente por los nuevos contenedores al recrearse el entorno.

CASO 10

Investigación y complementos

Listo para revisión · Avance guardado: 02 nov. 2026, 17:38

Lo solicitado

Incluir otros comandos, aplicaciones o usos de Docker y Compose investigados como complemento del entrenamiento.

Desarrollo y resultados

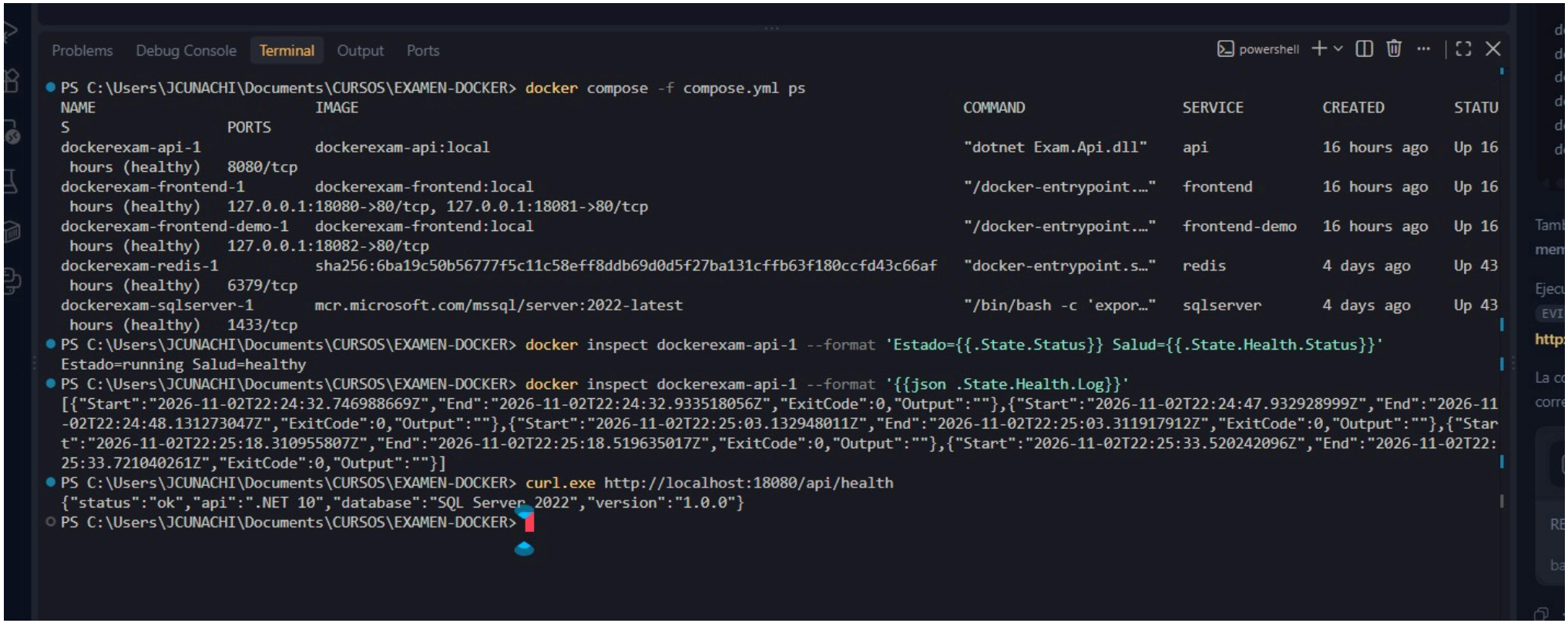
Se conluyo con el ITEM DEL CASO 10

Evidencias adjuntas: 6. Las capturas y sus descripciones se presentan a continuación.

CASO 10 · EVIDENCIA 1075

Salud del stack y detalle del healthcheck

Cargada: 02 nov. 2026, 17:27 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose -f compose.yml ps
```

```
docker inspect dockerexam-api-1 --format 'Estado={{.State.Status}} Salud={{.State.Health.Status}}'

docker inspect dockerexam-api-1 --format '{{json .State.Health.Log}}'

curl.exe http://localhost:18080/api/health
```

Descripción:

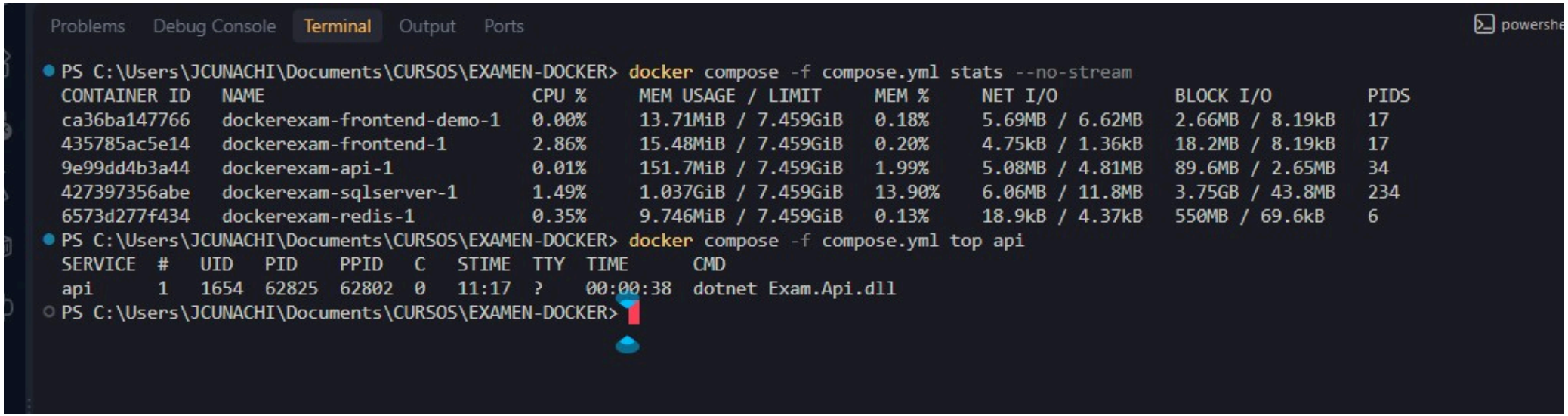
- `compose ps`: Permite consultar el estado de los servicios Docker, verificando que los cinco contenedores se encuentren en ejecución y saludables.
- `inspect`: Permite consultar información detallada sobre el estado y funcionamiento del contenedor API.
- `.State.Status`: Muestra el estado de ejecución del contenedor, obteniendo `running`.
- `.State.Health.Status`: Permite verificar el estado de salud del contenedor, obteniendo `healthy`.
- `.State.Health.Log`: Muestra el historial de las verificaciones de salud realizadas, incluyendo fechas, resultados y códigos de salida.
- `--format`: Permite filtrar la información de inspección para mostrar únicamente los datos requeridos.
- `curl.exe`: Realiza una solicitud HTTP al endpoint `/api/health` para comprobar la disponibilidad y respuesta de la API.
- `ExitCode: 0`: Indica que las verificaciones de salud registradas finalizaron correctamente.

Resultado: Se verificó satisfactoriamente el funcionamiento de los cinco servicios Docker (`api`, `frontend`, `frontend-demo`, `redis` y `sqlserver`), confirmando su estado Healthy (5/5). Asimismo, se comprobó que la API se encuentra en estado `running`, con verificaciones de salud exitosas (`ExitCode: 0`). Finalmente, el endpoint `/api/health` respondió con `"status":"ok"`, confirmando la disponibilidad de la API y que la comprobación reporta correctamente el uso de .NET 10 y SQL Server 2022.

CASO 10 · EVIDENCIA 1076

Monitoreo del consumo de recursos y verificación de procesos en ejecución de los contenedores Docker.

Cargada: 02 nov. 2026, 17:29 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose -f compose.yml stats --no-stream

docker compose -f compose.yml top api
```

Descripción:

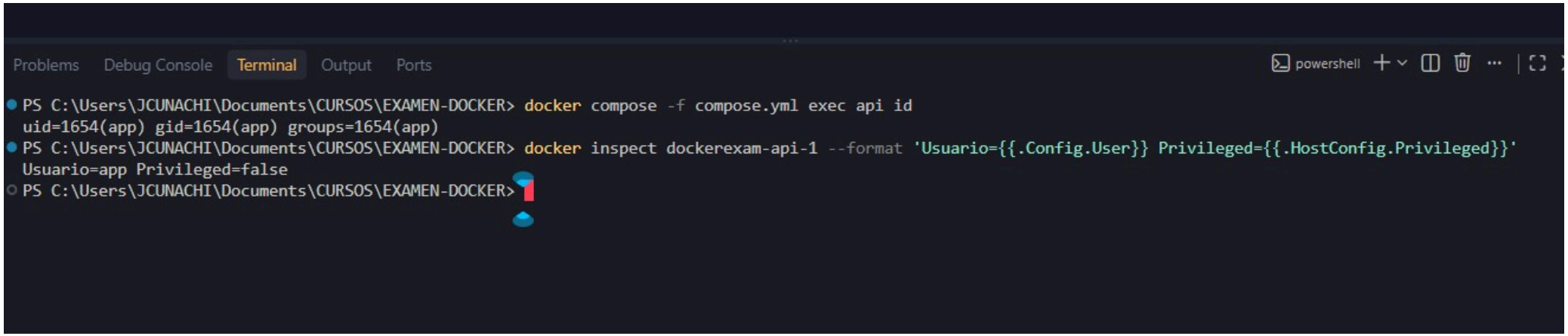
- `compose`: Permite administrar y supervisar los servicios definidos en un archivo Docker Compose.
- `-f compose.yml`: Especifica el archivo de configuración utilizado para administrar los contenedores.
- `stats`: Permite consultar estadísticas de consumo de recursos de los contenedores, incluyendo CPU, memoria, red y operaciones de disco.
- `--no-stream`: Muestra una única captura de las estadísticas, sin actualizar continuamente los resultados.
- `CPU %`: Indica el porcentaje de utilización del procesador por cada contenedor.
- `MEM USAGE / LIMIT`: Muestra la memoria utilizada y el límite disponible.
- `NET I/O`: Presenta el tráfico de red recibido y transmitido.
- `BLOCK I/O`: Muestra las operaciones de lectura y escritura realizadas en dispositivos de almacenamiento.
- `PIDS`: Indica la cantidad de procesos o tareas existentes dentro de cada contenedor.
- `top api`: Permite consultar los procesos en ejecución del servicio API, identificando sus PID y comandos asociados.

Resultado: Se verificó el consumo de recursos de los cinco contenedores (`frontend`, `frontend-demo`, `api`, `sqlserver` y `redis`). SQL Server presentó el mayor consumo de memoria, aproximadamente 1.037 GiB, mientras que los demás servicios registraron consumos menores. Asimismo, se confirmó que el servicio API ejecuta el proceso `dotnet Exam.Api.dll`, demostrando que la aplicación .NET se encuentra en ejecución y permitiendo supervisar el comportamiento y utilización de recursos del entorno Docker.

CASO 10 · EVIDENCIA 1077

Verificación de seguridad del contenedor API mediante usuario no privilegiado.

Cargada: 02 nov. 2026, 17:31 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose -f compose.yml exec api id
```

```
docker inspect dockerexam-api-1 --format 'Usuario={{.Config.User}} Privileged={{.HostConfig.Privileged}}'
```

Descripción:

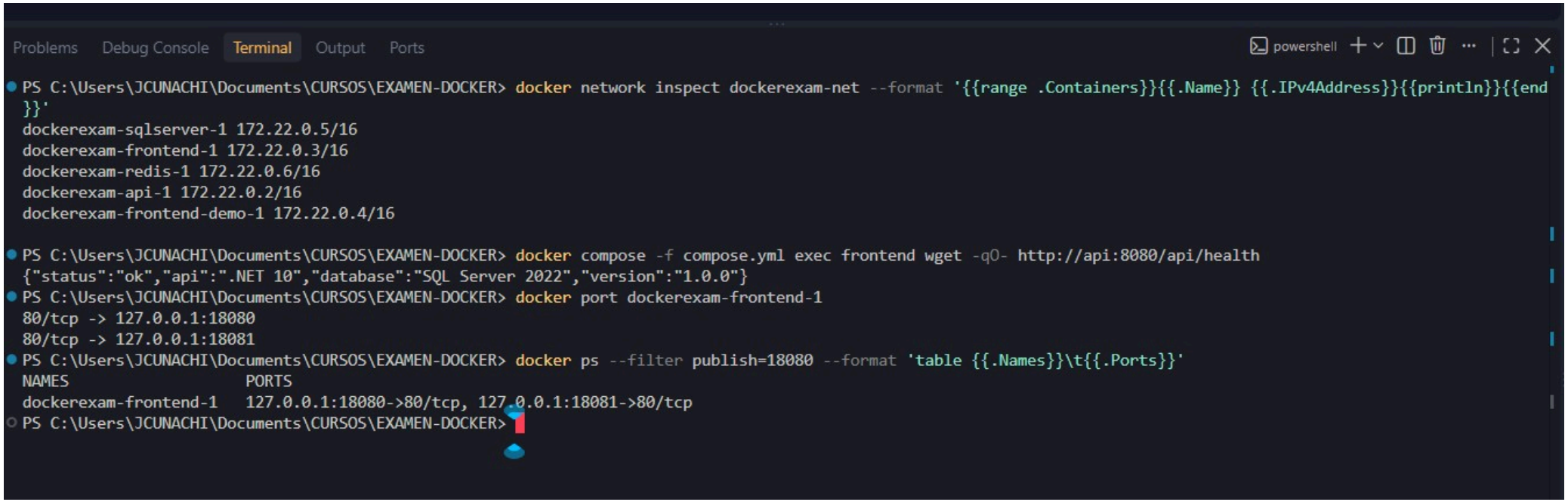
- `compose exec api`: Permite ejecutar comandos dentro del contenedor correspondiente al servicio API.
- `id`: Muestra la identidad del usuario que ejecuta los procesos dentro del contenedor, incluyendo su identificador (UID), grupo (GID) y grupos asociados.
- `inspect`: Permite consultar información detallada sobre la configuración y propiedades de seguridad del contenedor.
- `--format`: Permite filtrar y mostrar únicamente las propiedades solicitadas.
- `{{.Config.User}}`: Identifica el usuario configurado para ejecutar la aplicación dentro del contenedor.
- `{{.HostConfig.Privileged}}`: Permite verificar si el contenedor se ejecuta en modo privilegiado (`true`) o no privilegiado (`false`).

Resultado: Se verificó que el contenedor `dockerexam-api-1` ejecuta sus procesos mediante el usuario `app` (UID 1654, GID 1654), en lugar del usuario administrador `root`. Asimismo, se confirmó que la configuración `Privileged=false` mantiene deshabilitado el modo privilegiado de Docker, aplicando buenas prácticas de seguridad y reduciendo los permisos otorgados al contenedor.

CASO 10 · EVIDENCIA 1078

Red interna, DNS y puertos publicados

Cargada: 02 nov. 2026, 17:33 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker network inspect dockerexam-net --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}{{println}}{{end}}'
```

```
docker compose -f compose.yml exec frontend wget -qO- http://api:8080/api/health

docker port dockerexam-frontend-1

docker ps --filter publish=18080 --format 'table {{.Names}}\t{{.Ports}}'
```

Descripción:

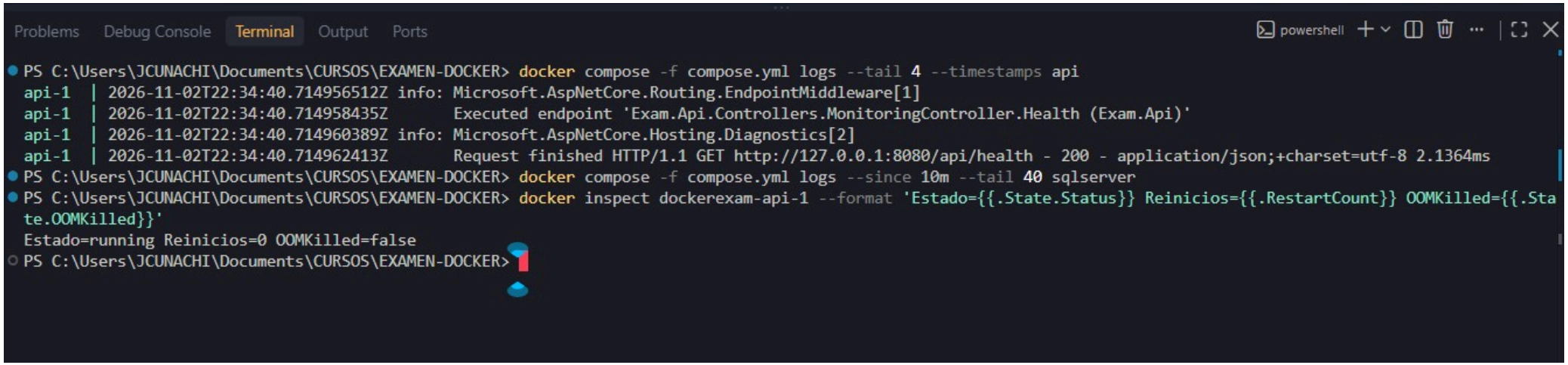
- `network inspect`: Permite consultar la configuración de una red Docker y los contenedores conectados.
- `dockerexam-net`: Nombre de la red personalizada que permite la comunicación entre los servicios.
- `--format`: Permite mostrar información específica mediante plantillas de Go.
- `.Containers`: Recorre los contenedores conectados a la red.
- `.Name` y `.IPv4Address`: Muestran el nombre y la dirección IP interna de cada contenedor.
- `compose exec frontend`: Permite ejecutar comandos dentro del contenedor Frontend.
- `wget -qO-`: Realiza una solicitud HTTP y muestra la respuesta del servicio, sin imprimir información adicional.
- `http://api:8080/api/health`: Endpoint utilizado para verificar la conectividad interna entre el Frontend y la API mediante el nombre del servicio Docker.
- `docker port`: Muestra los puertos internos del contenedor y sus correspondientes puertos publicados en el equipo anfitrión.
- `docker ps --filter publish=18080`: Filtra los contenedores que tienen publicado el puerto `18080`.
- `{{.Names}}` y `{{.Ports}}`: Permiten visualizar únicamente los nombres de los contenedores y sus puertos configurados.

Resultado: Se verificó que los cinco contenedores se encuentran conectados a la red `dockerexam-net`, con direcciones IP internas dentro del rango `172.22.0.0/16`. Asimismo, se comprobó exitosamente la comunicación entre el Frontend y la API, obteniendo una respuesta con `"status":"ok"`. Finalmente, se confirmó la publicación de los puertos locales `18080` y `18081` hacia el puerto interno `80` del contenedor Frontend, demostrando la correcta configuración de la red y accesibilidad de los servicios.

CASO 10 · EVIDENCIA 1079

Logs, reinicios y memoria insuficiente

Cargada: 02 nov. 2026, 17:35 (Lima)



Descripción de la evidencia

Comandos ejecutados:

```
docker compose -f compose.yml logs --tail 4 --timestamps api
```

```
docker compose -f compose.yml logs --since 10m --tail 40 sqlserver

docker inspect dockerexam-api-1 --format 'Estado={{.State.Status}} Reinicios={{.RestartCount}} OOMKilled={{.State.OOMKilled}}'
```

Descripción:

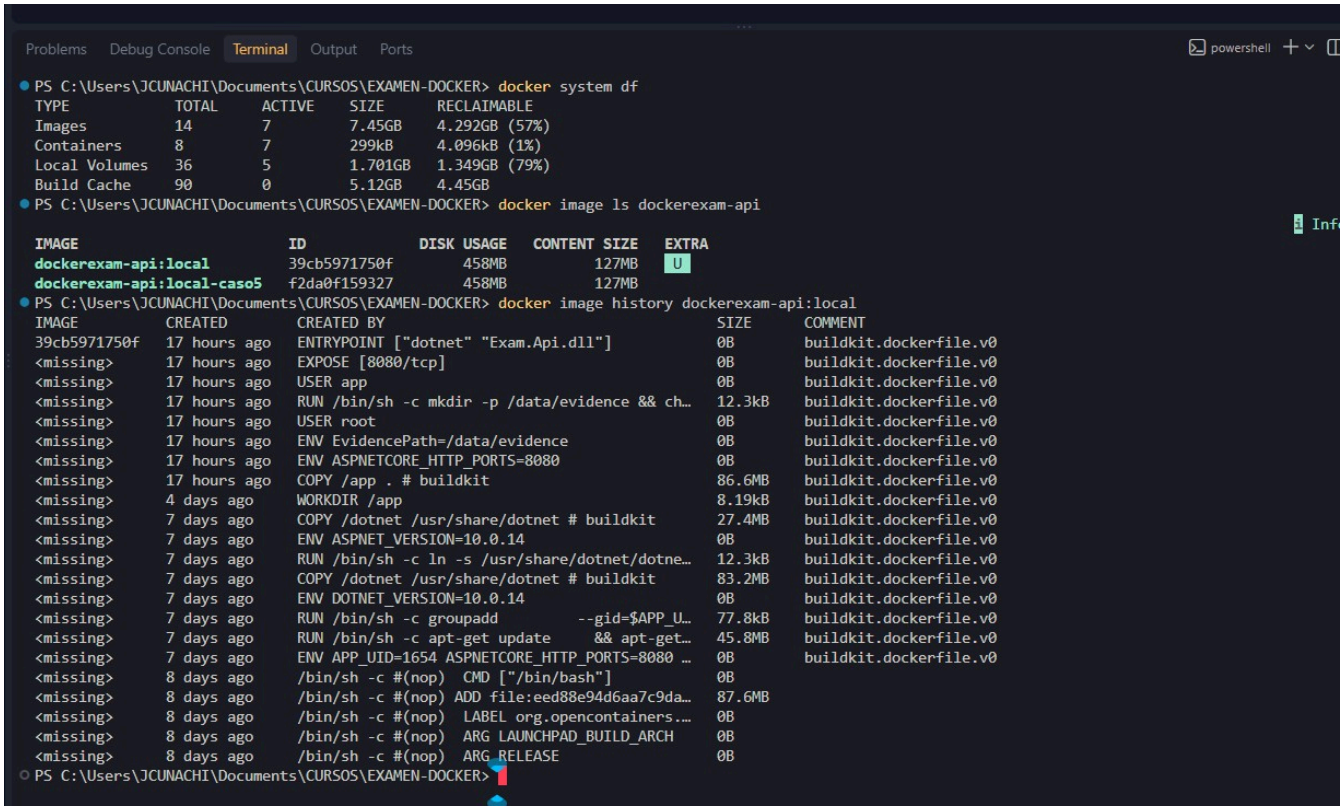
- `compose logs`: Permite consultar los registros generados por los servicios definidos en Docker Compose.
- `--tail 4`: Muestra únicamente las últimas cuatro líneas de los registros.
- `--timestamps`: Incluye la fecha y hora de cada evento registrado.
- `api`: Identifica el servicio backend cuyos registros se desean consultar.
- `--since 10m`: Filtra los registros generados durante los últimos diez minutos.
- `--tail 40`: Limita la consulta a las últimas 40 líneas disponibles.
- `sqlserver`: Identifica el servicio de base de datos cuyos registros se desean revisar.
- `inspect`: Permite consultar información detallada sobre el estado y comportamiento del contenedor.
- `.State.Status`: Muestra el estado actual de ejecución del contenedor.
- `.RestartCount`: Indica la cantidad de reinicios registrados por Docker.
- `.State.OOMKilled`: Permite identificar si el contenedor fue finalizado debido a un consumo excesivo de memoria.

Resultado: Se verificaron los registros de la API, confirmando una solicitud HTTP exitosa al endpoint `/api/health`, con código de respuesta 200 OK y un tiempo de procesamiento aproximado de 2.14 ms. Asimismo, se comprobó que el contenedor `dockerexam-api-1` permanece en estado `running`, con 0 reinicios y `OOMKilled=false`, evidenciando estabilidad operativa y ausencia de finalizaciones por falta de memoria.

CASO 10 · EVIDENCIA 1080

Espacio utilizado e historial de la imagen

Cargada: 02 nov. 2026, 17:37 (Lima)



Descripción de la evidencia

Título de la evidencia: Verificación del consumo de almacenamiento Docker, listado de imágenes y análisis del historial de construcción de la API.

Comandos ejecutados:

```
docker system df

docker image ls dockerexam-api

docker image history dockerexam-api:local
```

Descripción:

- `system df`: Permite consultar el espacio en disco utilizado por Docker, incluyendo imágenes, contenedores, volúmenes y caché de construcción.
- `TOTAL`: Indica la cantidad total de recursos Docker existentes.
- `ACTIVE`: Muestra la cantidad de recursos que se encuentran en uso.
- `SIZE`: Indica el espacio de almacenamiento ocupado por cada categoría.
- `RECLAIMABLE`: Muestra el espacio potencialmente recuperable mediante operaciones de limpieza.
- `image ls`: Permite listar las imágenes Docker disponibles localmente, mostrando sus identificadores, etiquetas y tamaños.
- `dockerexam-api`: Nombre de la imagen correspondiente al servicio Backend o API.
- `image history`: Permite visualizar el historial de construcción de una imagen Docker, incluyendo sus capas, instrucciones ejecutadas y tamaños.
- `dockerexam-api:local`: Identifica la imagen local de la API cuyo historial se desea inspeccionar.

Resultado: Se verificó el almacenamiento utilizado por Docker, identificando 14 imágenes (7.45 GB), 8 contenedores, 36 volúmenes (1.701 GB) y 90 registros de caché de construcción (5.12 GB). Asimismo, se confirmó la existencia de las imágenes `dockerexam-api:local` y `dockerexam-api:local-caso5`, ambas con un uso de disco de 458 MB.

Finalmente, se revisó el historial de construcción de la API, identificando las instrucciones de configuración de .NET 10, la exposición del puerto `8080`, la creación del usuario `app` y la ejecución de `Exam.Api.dll`. Esto permite evaluar el consumo de almacenamiento, la estructura de la imagen y las oportunidades de optimización de los recursos Docker.